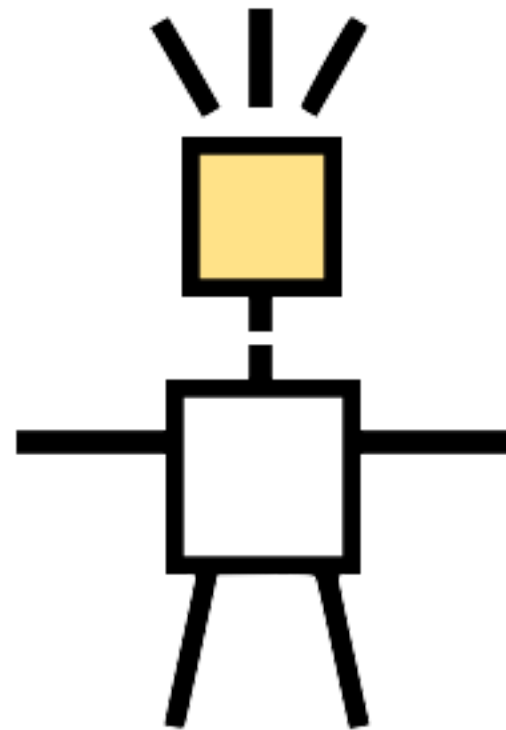


Welcome to the ITensor School!

2026 Edition



Upcoming Workshop Weds-Fri

New Methods and Applications of Tensor Networks

Wednesday: "Harnessing Physical Structure for Efficient Simulation"

Mike Zaletel, Sam Garratt, Mari Carmen Banuls, Carolin Wille

Thursday: "Tensor Networks for Classical Simulation"

Todd Gingrich, Kate Murphy, Yuehaw Khoo, Juan Jose Arenas, Hessam Babaei

Friday: "The Future of Tensor Network Algorithms"

Chris Camaño, Garnet Chan, Glen Evenbly, Dima Abanin

Goals of the School

- Fundamental **theory** (tensor networks, algorithms)
- Understanding **strengths** and **weaknesses** of tensor networks
- Learning the **ITensor software** – hands on experience

Goals of the School

This is for you!

Please ask questions if
anything is unclear...

Goals of the School

A note on AI usage...

You wouldn't use a forklift
to lift weight at the gym

So don't let AI substitute
for learning



But after spending, say, 45 minutes going "no AI"
for the hands-on tutorials, feel free to use it to
"push" your code or develop visualizations

Schedule

Day One:

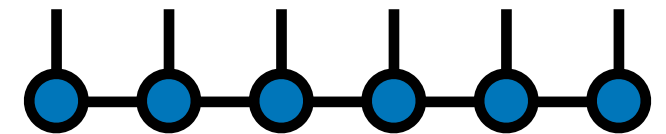
Talk 1: Tensor Networks, MPS, and ITensor

Talk 2: DMRG algorithm and ITensorMPS

Hands-on: Using DMRG, writing measurement code

Talk 3: Time evolution with MPS

Hands-on: Time Evolution of MPS, Sampling, METTS



$$e^{-iHt} |\psi(0)\rangle = |\psi(t)\rangle$$

Day Two:

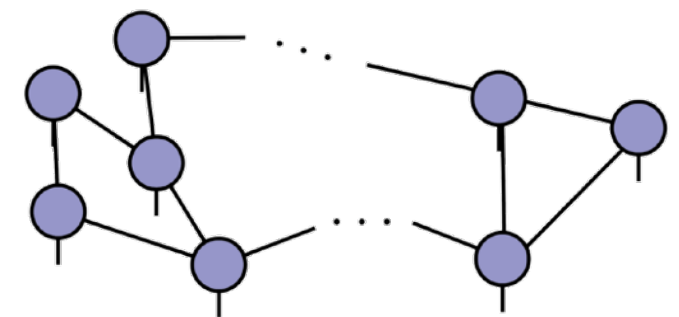
Talk 4: Quantics and MPS beyond wave functions

Hands-on: QTT functions

Talk 5: Tensor networks in higher dimensions

Hands-on: Belief Propagation on 2D, 3D TN's

$$f(x, y)$$



Schedule

Featured lecturers:

Lukas Devos

Easy to Write, Fast to Build:

Operator Sums to MPOs with OpSum.jl



Jack Dunham

*Parallelizing and Distributing Tensor
Network Algorithms*



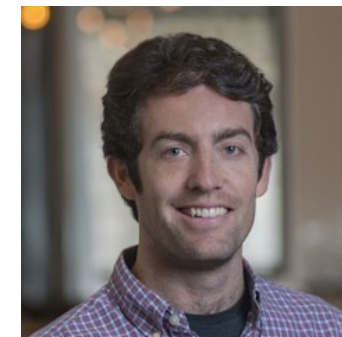
Schedule

Main lecturers:

Matt Fishman



Miles Stoudenmire



Joey Tindall



This Talk

- Tensors
- Tensor Networks (Matrix Product States)
- The ITensor software

Tensors and Tensor Networks

Is exponential compression possible?

Tensor Networks

General wavefunction of n qubits

$$|\Psi\rangle = \sum_{s_1 s_2 s_3 \cdots s_n} \Psi^{s_1 s_2 s_3 \cdots s_n} |s_1 s_2 s_3 \cdots s_n\rangle \quad s_j \in 0, 1$$

Amplitudes form a big tensor!

$$\Psi^{s_1 s_2 s_3 s_4 s_5 s_6} =$$


The diagram shows a blue rounded rectangle representing a tensor. Six vertical lines extend upwards from the top edge of the rectangle, each labeled with an index: s_1 , s_2 , s_3 , s_4 , s_5 , and s_6 from left to right.

Tensor Networks

What is a tensor?

vector

$$v = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

$$v_2 = 3$$

matrix

$$M = \begin{bmatrix} 5 & 7 \\ 8 & 9 \end{bmatrix}$$

$$M_{12} = 7$$

order-3
tensor

$$T = \begin{bmatrix} 3 & \begin{bmatrix} 5 & 4 \end{bmatrix} & 7 \\ 1 & \begin{bmatrix} 3 & 2 \end{bmatrix} & 5 \end{bmatrix}$$

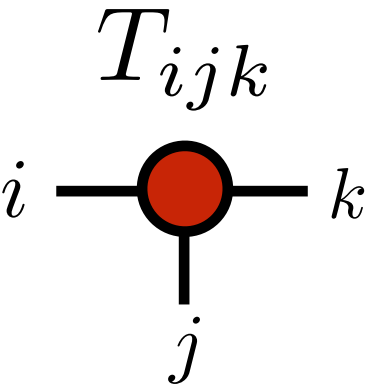
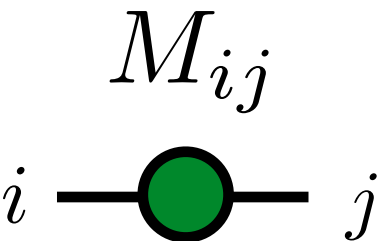
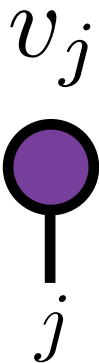
$$T_{112} = 5$$

Tensor Networks

N-index tensor = shape with N lines

$$T^{s_1 s_2 s_3 \cdots s_N} = \text{Diagram of a tensor with } N \text{ lines labeled } s_1, s_2, s_3, s_4, \dots, s_N$$

Low-order examples:



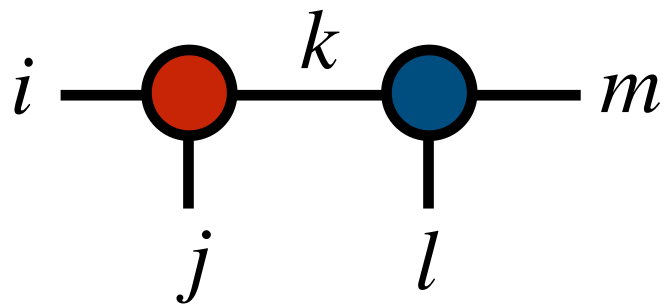
Joining wires means contraction:



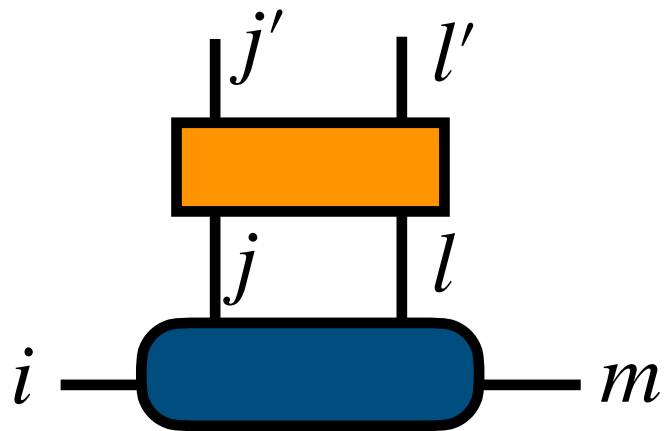
$$\sum_j M_{ij} v_j = w_i$$

Tensor Networks

Other contraction examples



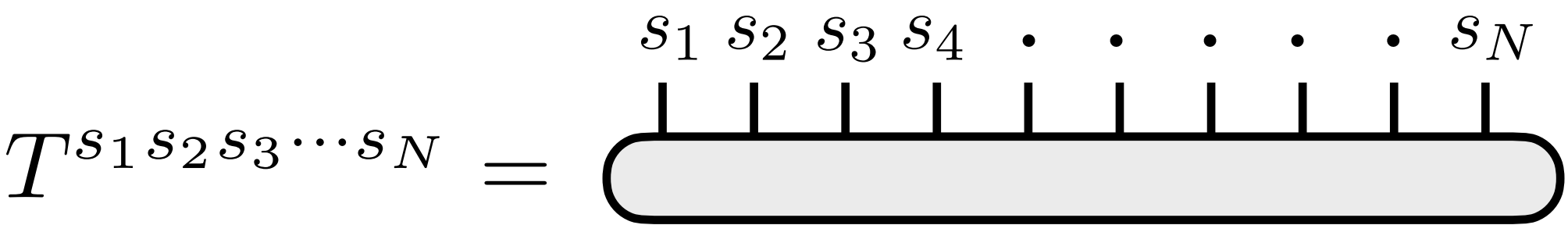
$$\sum_k A_{ijk} B_{klm} = C_{ijlm}$$



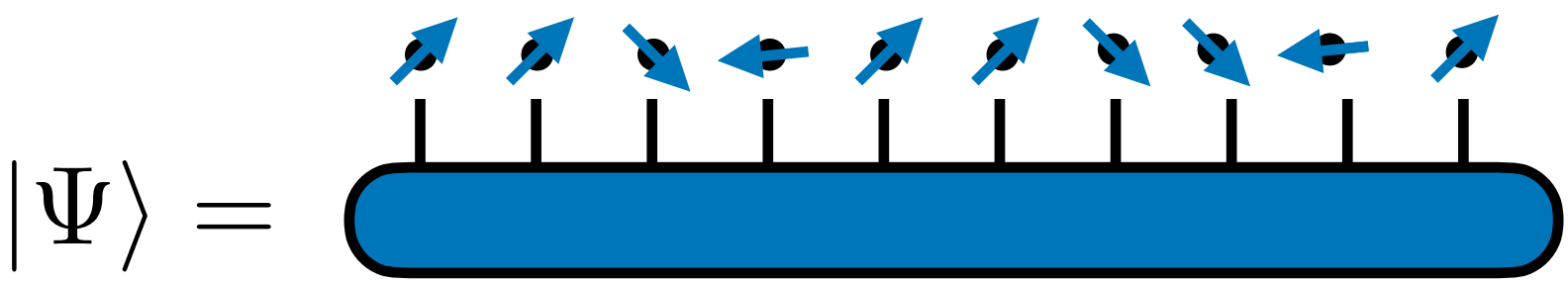
$$\sum_{jl} A_{ijlm} G_{jj'll'} = B_{ij'l'm}$$

Tensor Networks

Problem: N-index tensor **exponential** to store

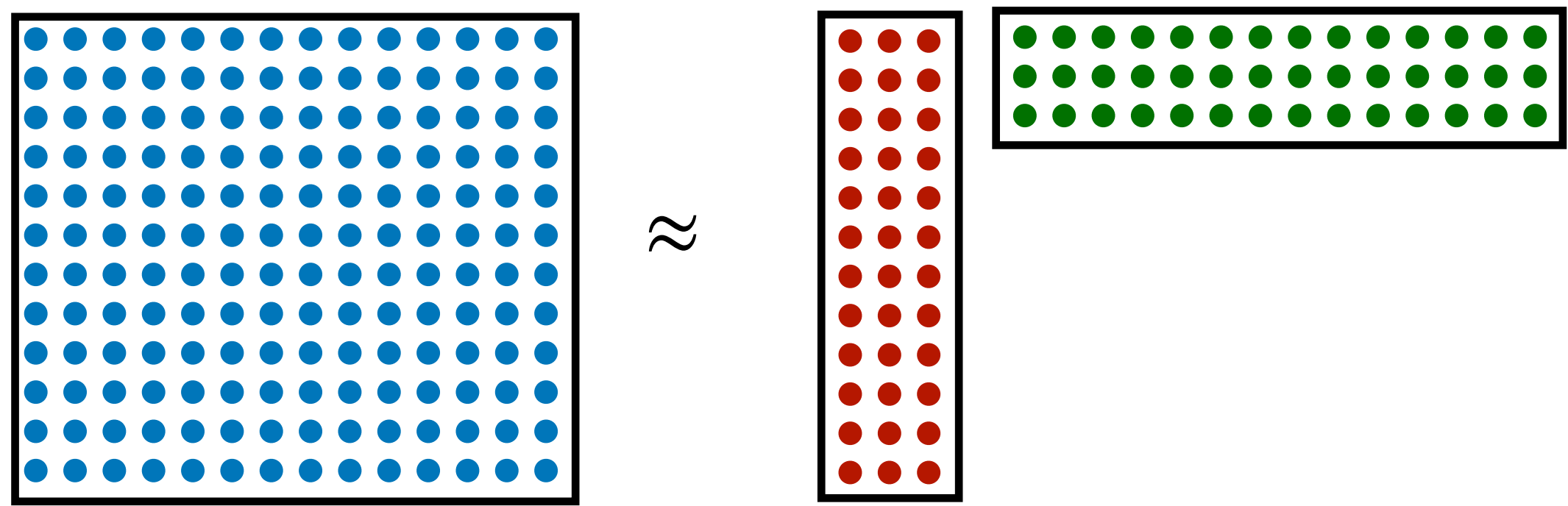


A version of the "**many-body problem**"



Tensor Networks

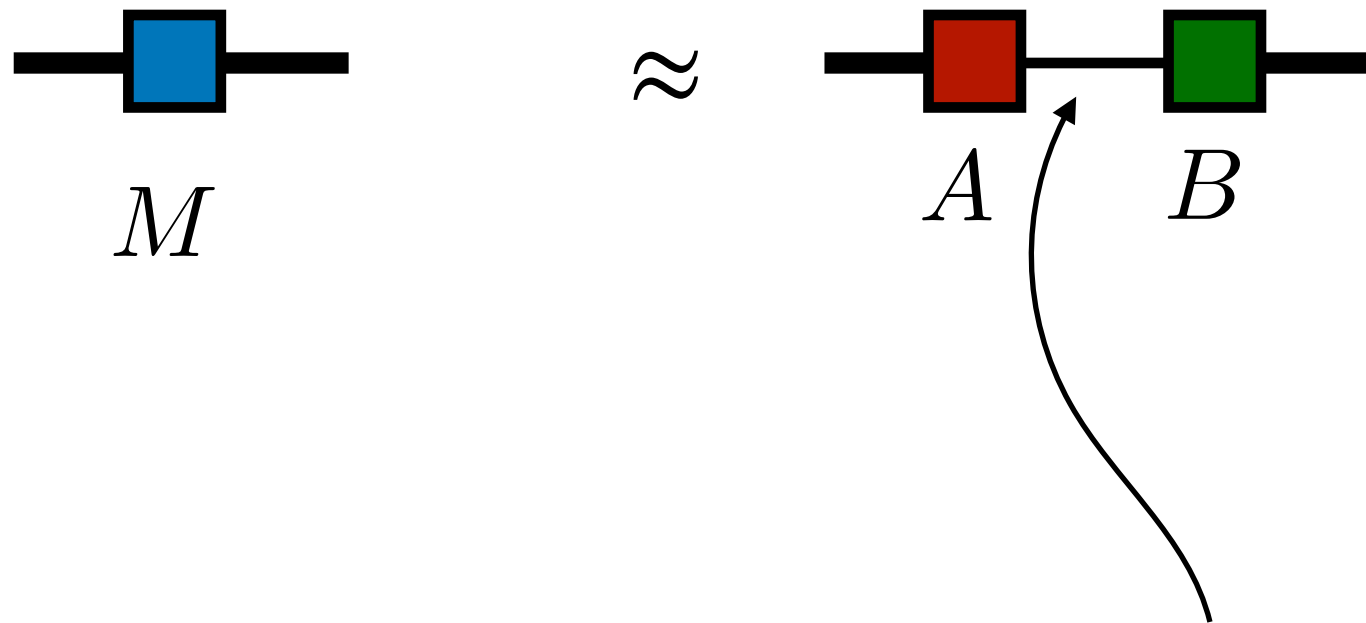
For matrices, can perform low-rank compression



Less memory and fewer operations

Tensor Networks

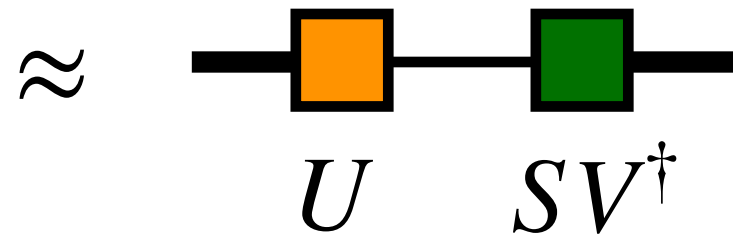
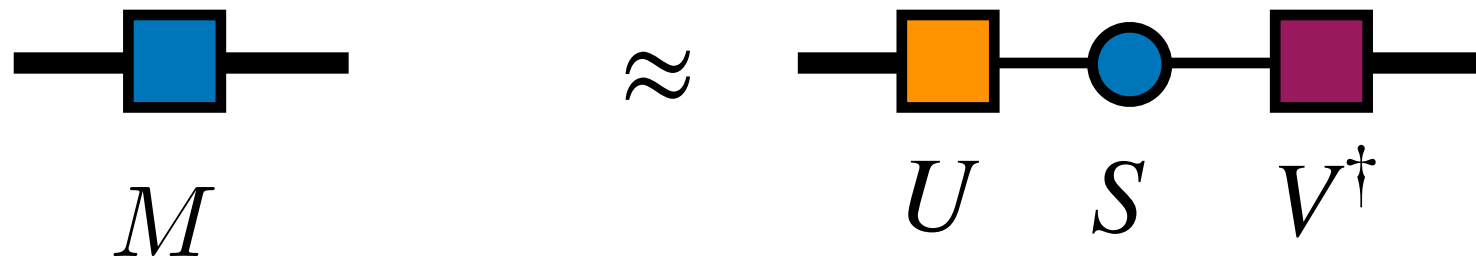
For matrices, can perform low-rank compression



*size of shared index called the
"rank" of the factorization*

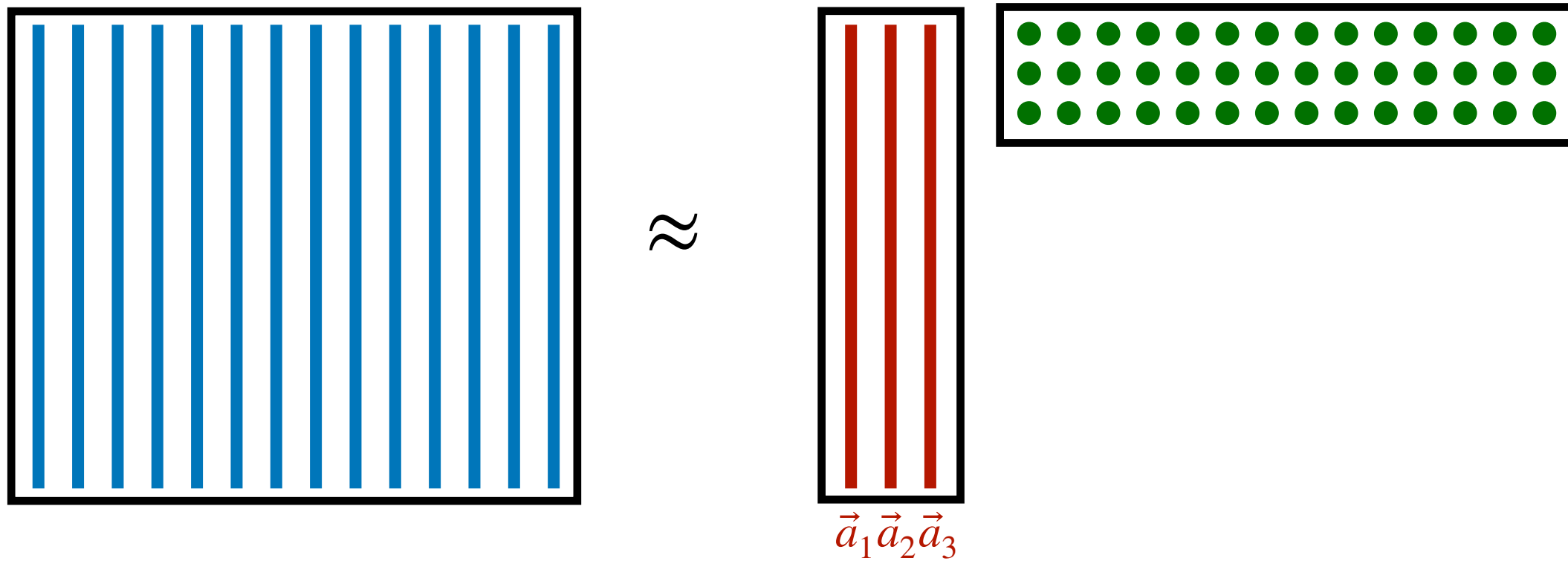
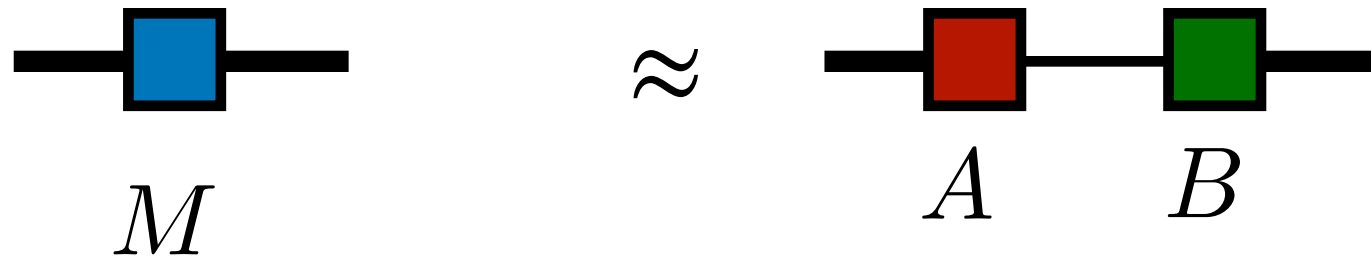
Tensor Networks

Optimal solution (smallest rank) found by SVD



Tensor Networks

Low-rank form means columns of M
linear combinations of columns of A



Likewise, rows of M combinations of rows of B

Low-rank: Test Your Knowledge

Is this matrix low rank? Or high rank?

Why?

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Low-rank: Test Your Knowledge

Is this matrix low rank? Or high rank?
Why?

$$\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} [1 \quad 1 \quad 1 \quad 1]$$

product of Nx1 and 1xN matrices

Low rank

Columns linearly dependent

Low-rank: Test Your Knowledge

Is this matrix low rank? Or high rank?

Why?

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Low-rank: Test Your Knowledge

Is this matrix low rank? Or high rank?
Why?

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix}$$

Low rank

Outer product of two vectors

Low-rank: Test Your Knowledge

Is this matrix low rank? Or high rank?

Why?

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Low-rank: Test Your Knowledge

Is this matrix low rank? Or high rank?
Why?

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

High rank (full rank)

All columns orthogonal / maximally linearly independent

Low-rank: Test Your Knowledge

Numerical example

Is this matrix low rank? Or high rank?

Why?

$$\begin{bmatrix} 0.16 & 0.110 & 0.117 & 0.084 \\ 0.73 & 0.873 & 0.304 & 0.287 \\ 0.16 & 0.110 & 0.117 & 0.084 \\ 0.73 & 0.873 & 0.304 & 0.287 \end{bmatrix}$$

Low-rank: Test Your Knowledge

Numerical example

Is this matrix low rank? Or high rank?

Why?

$$\begin{bmatrix} 0.16 & 0.110 & 0.117 & 0.084 \\ 0.73 & 0.873 & 0.304 & 0.287 \\ 0.16 & 0.110 & 0.117 & 0.084 \\ 0.73 & 0.873 & 0.304 & 0.287 \end{bmatrix} \approx \begin{bmatrix} 1.0 & -1.0 \\ 1.0 & 1.0 \\ 1.0 & -1.0 \\ 1.0 & 1.0 \end{bmatrix} \begin{bmatrix} 0.445 & 0.492 & 0.211 & 0.186 \\ 0.285 & 0.381 & 0.094 & 0.101 \end{bmatrix}$$

Low rank

All columns a sum of two vectors

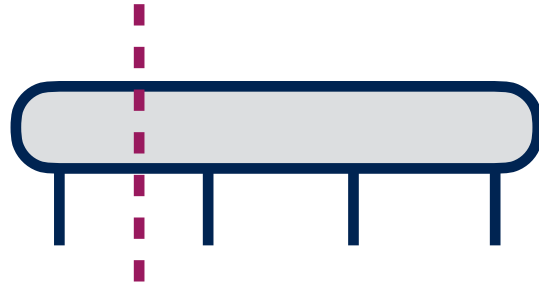
How to generalize SVD to tensors?

Reshape as a matrix:



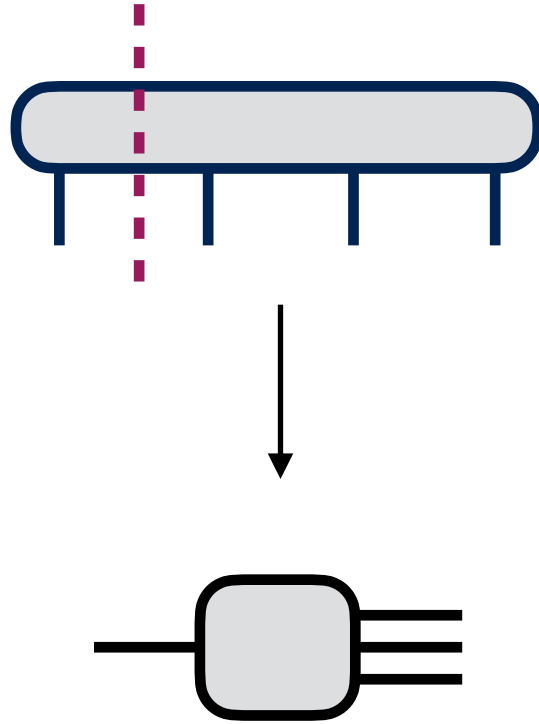
How to generalize SVD to tensors?

Reshape as a matrix:



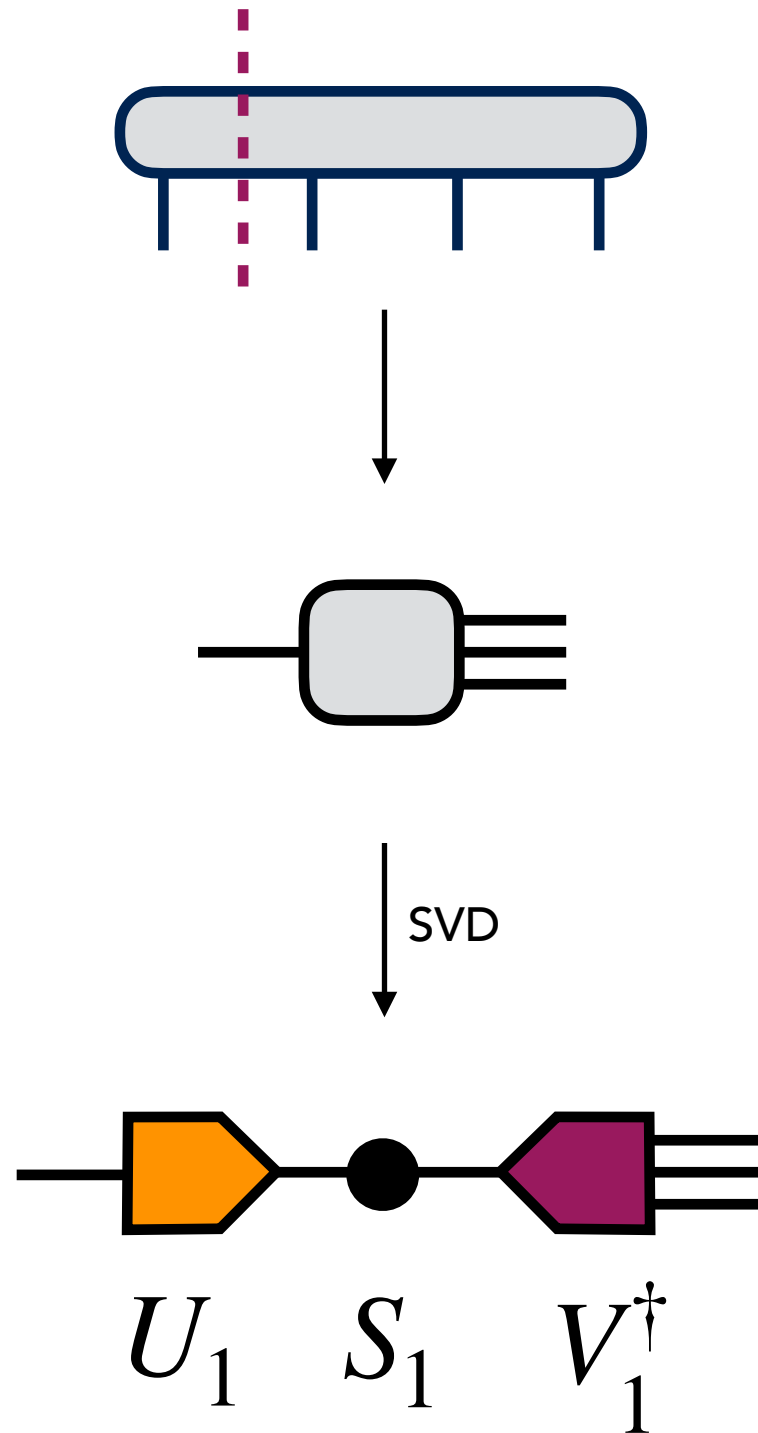
How to generalize SVD to tensors?

Reshape as a matrix:



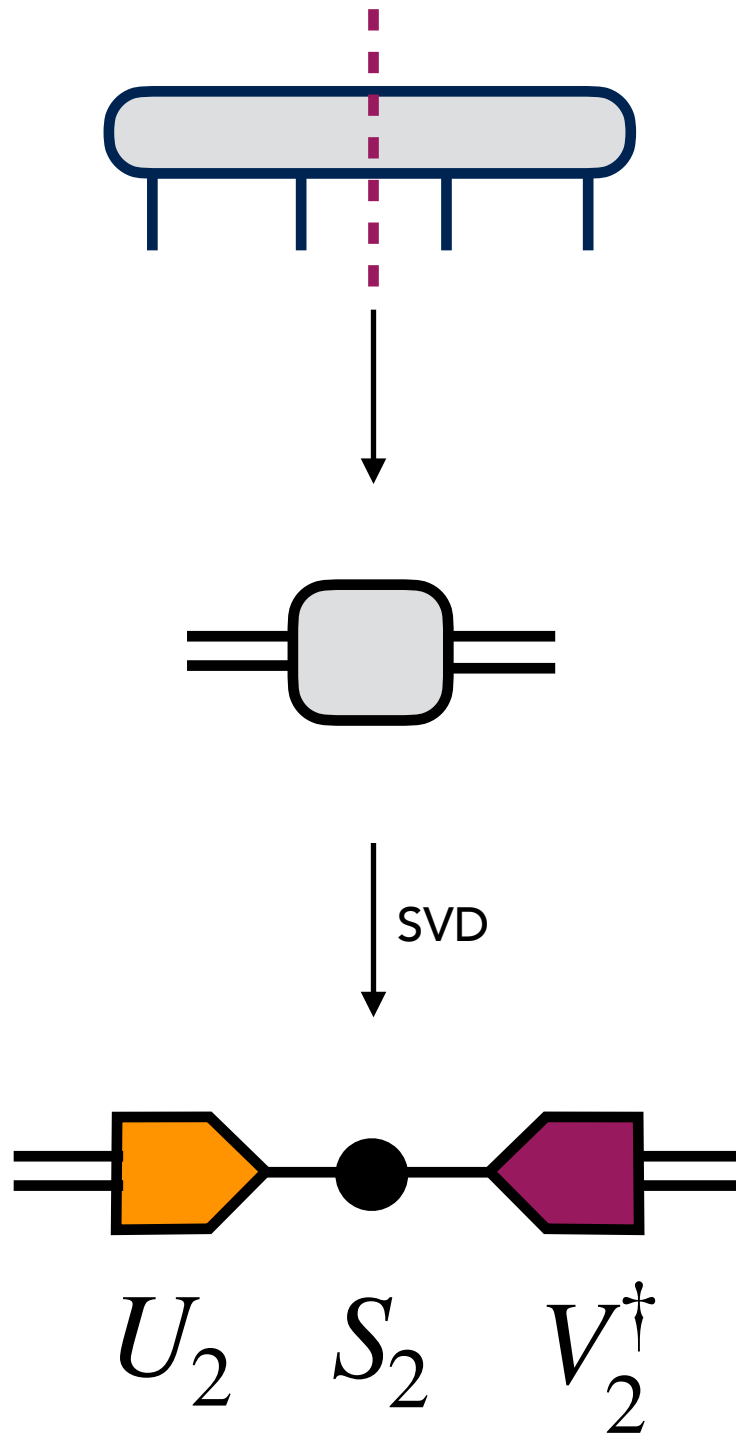
Generalizing SVD to tensors

Reshape as a matrix:



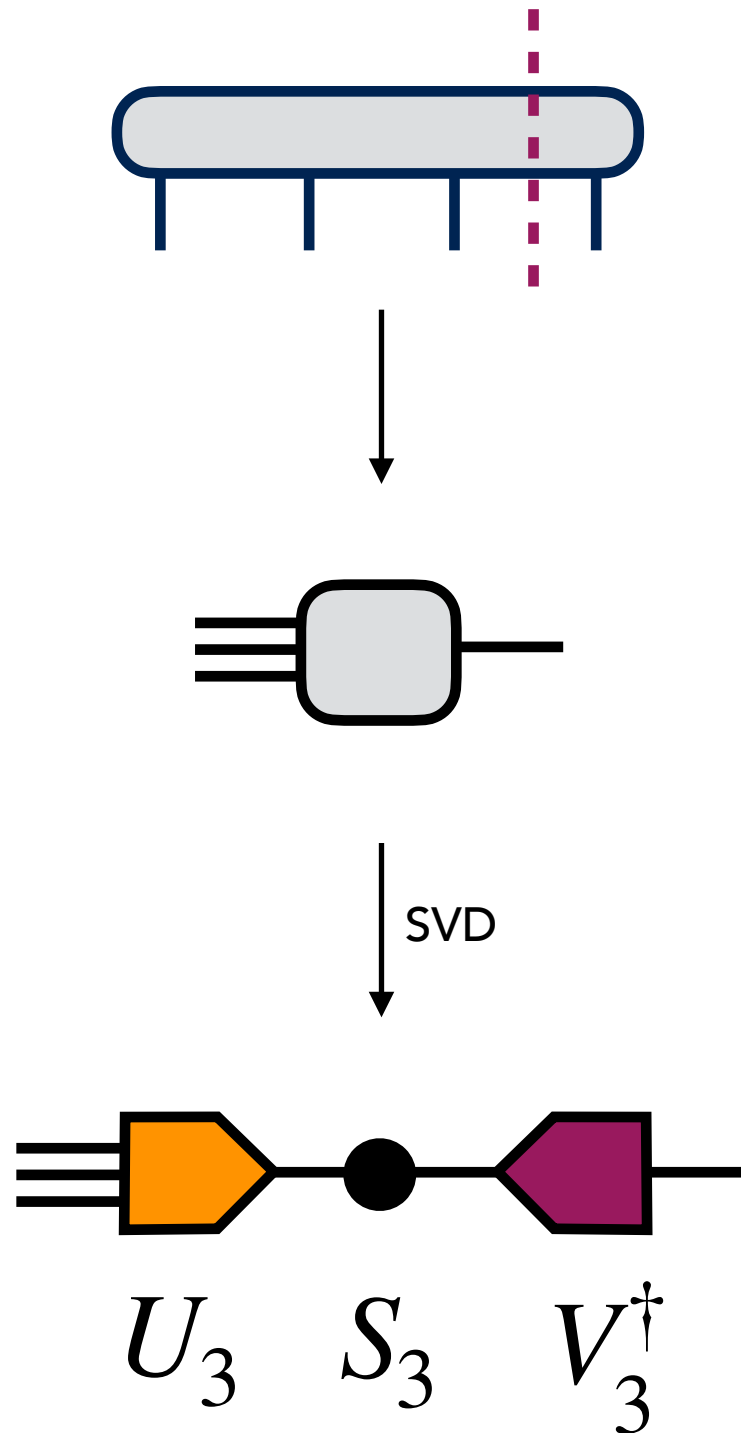
Generalizing SVD to tensors

Other partitions:

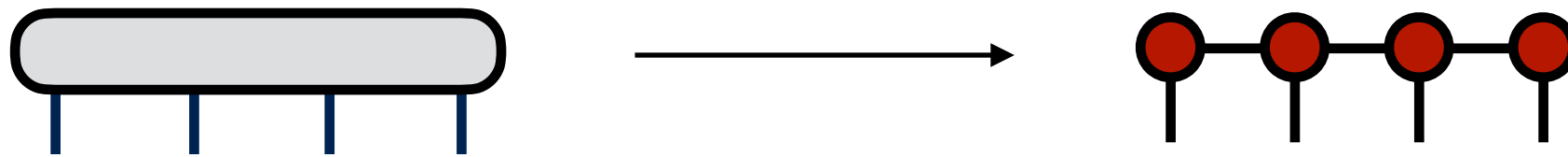


Generalizing SVD to tensors

Other partitions:



Factorizations lead to tensor networks

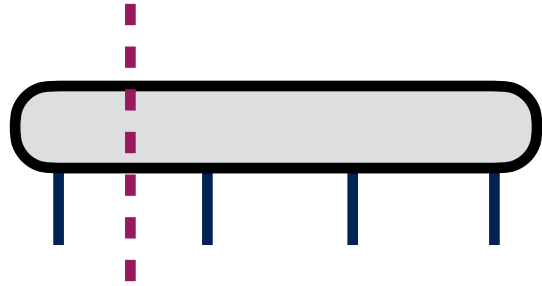


Consider sequence of SVD's...

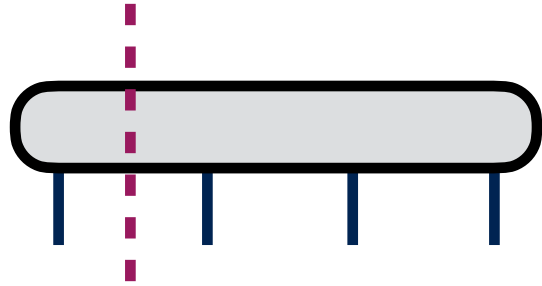
1. SVD first index from rest



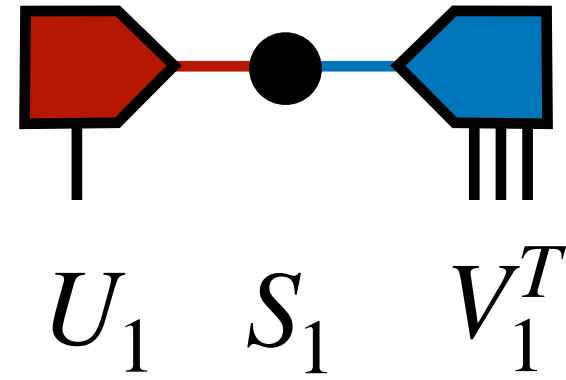
1. SVD first index from rest



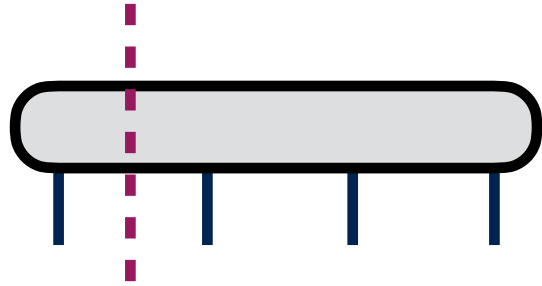
1. SVD first index from rest



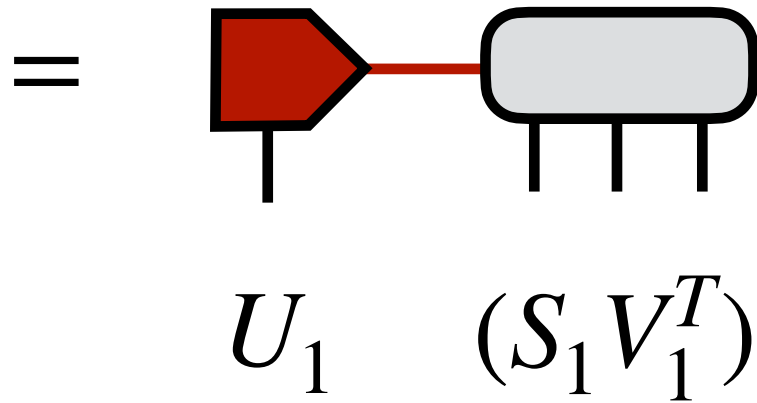
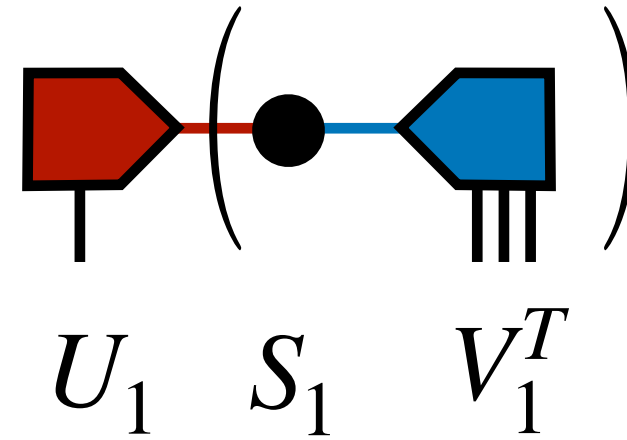
SVD
 $\approx$



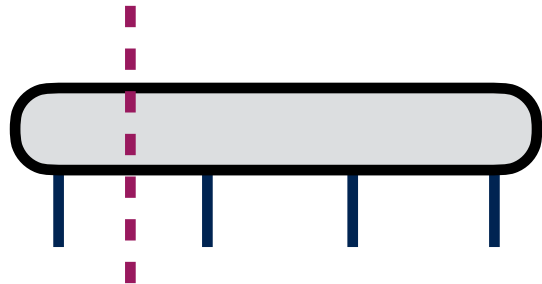
2. Multiply S_1 into V_1^T



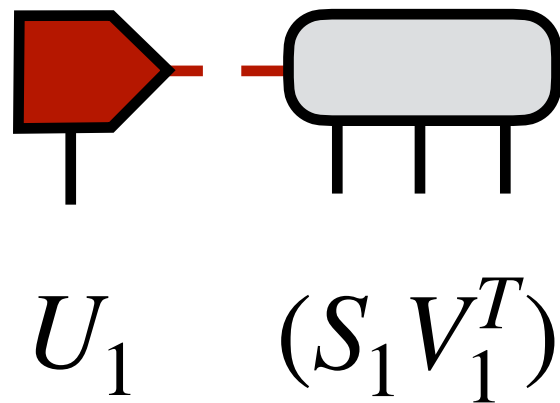
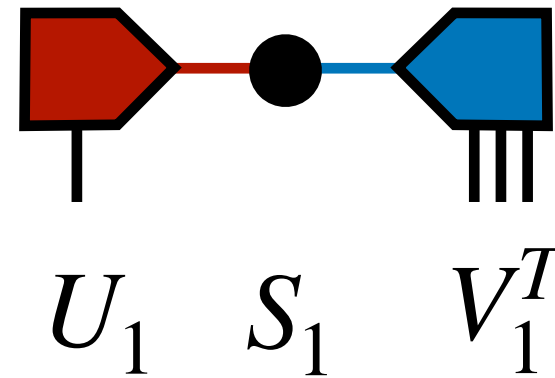
SVD
 $\approx$



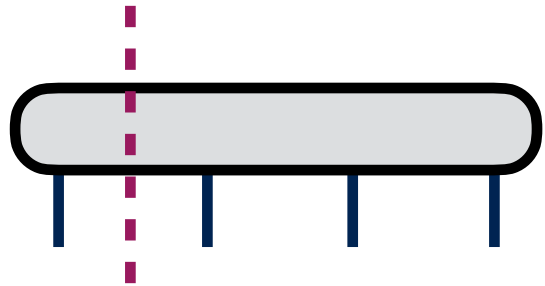
3. SVD this new tensor $(= S_1 V_1^T)$



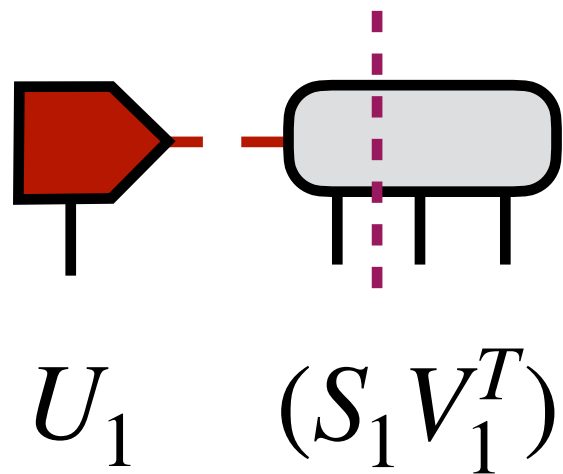
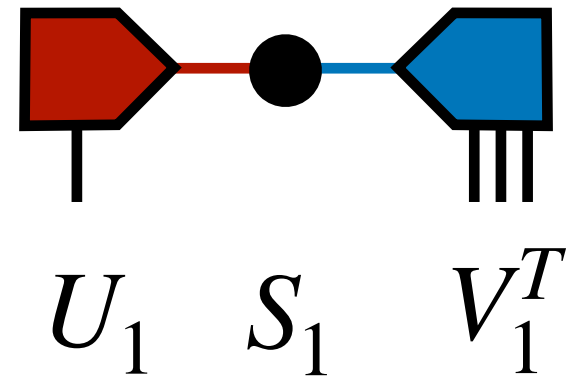
SVD
 $\approx$



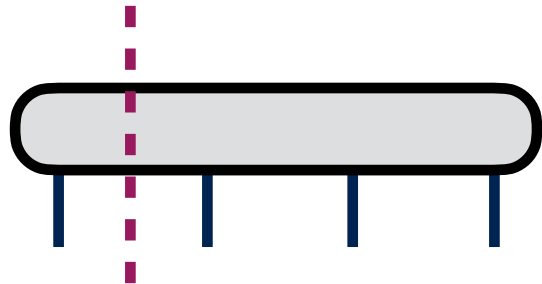
3. SVD this new tensor $(= S_1 V_1^T)$



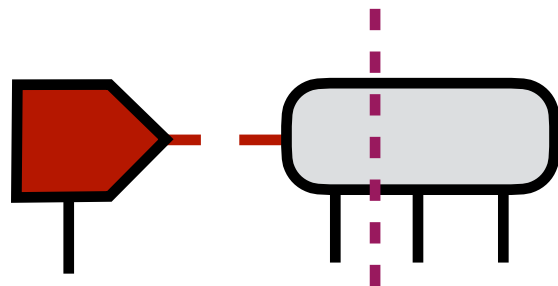
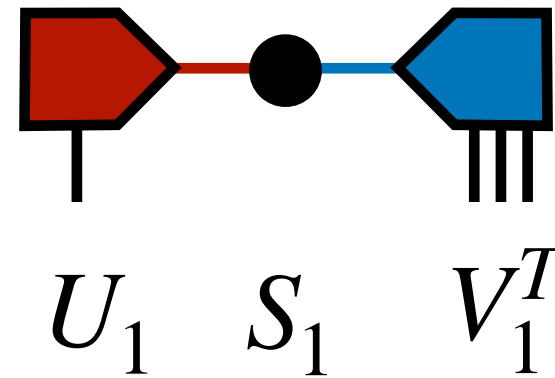
SVD
 $\approx$



3. SVD this new tensor $(= S_1 V_1^T)$

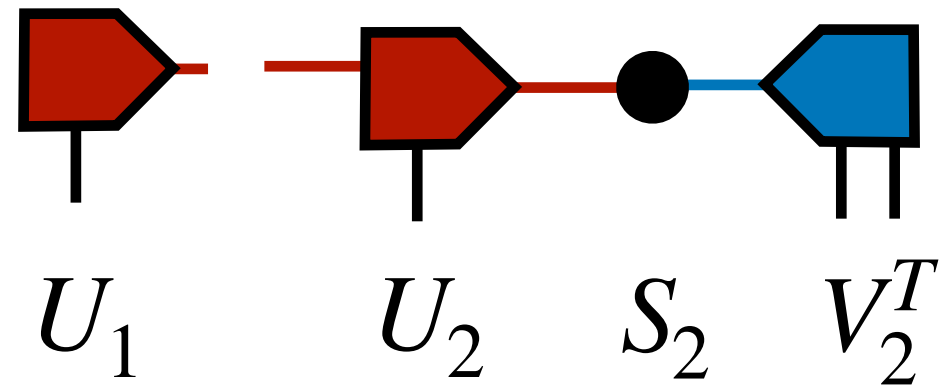


SVD
 $\approx$

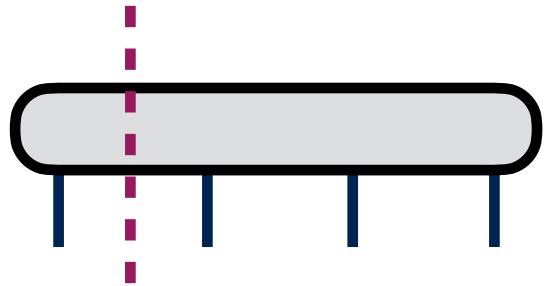


SVD
 $\approx$

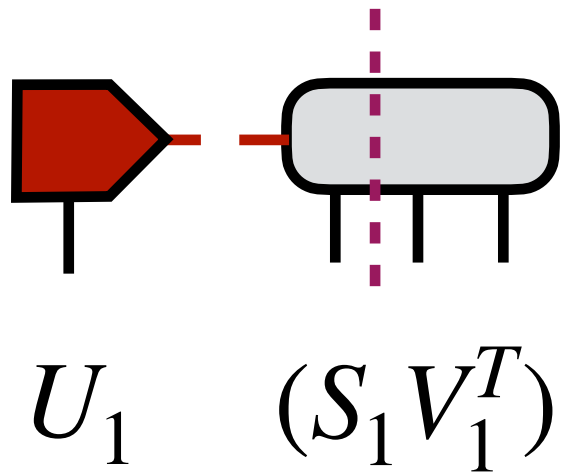
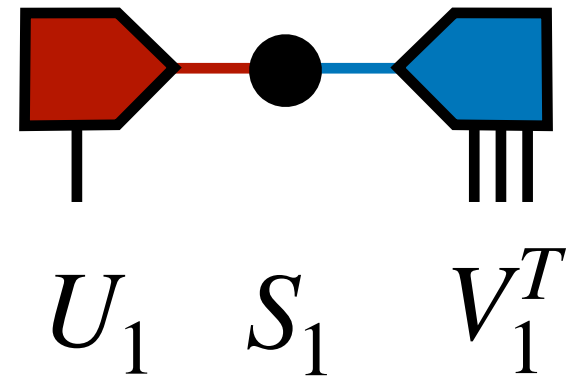
$U_1 \quad (S_1 V_1^T)$



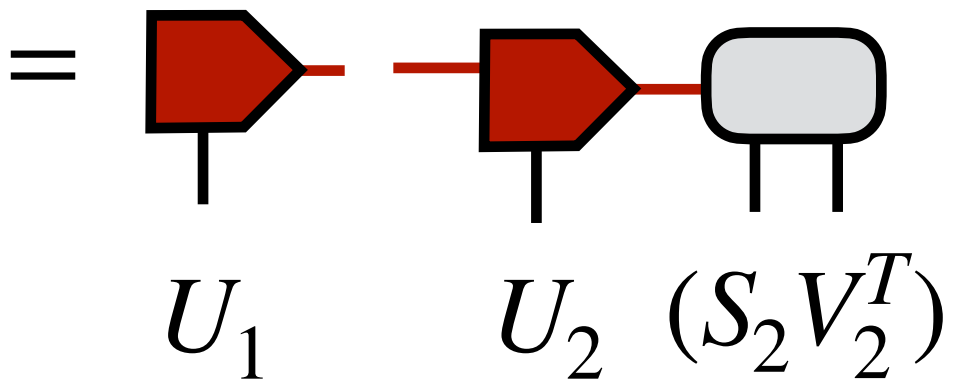
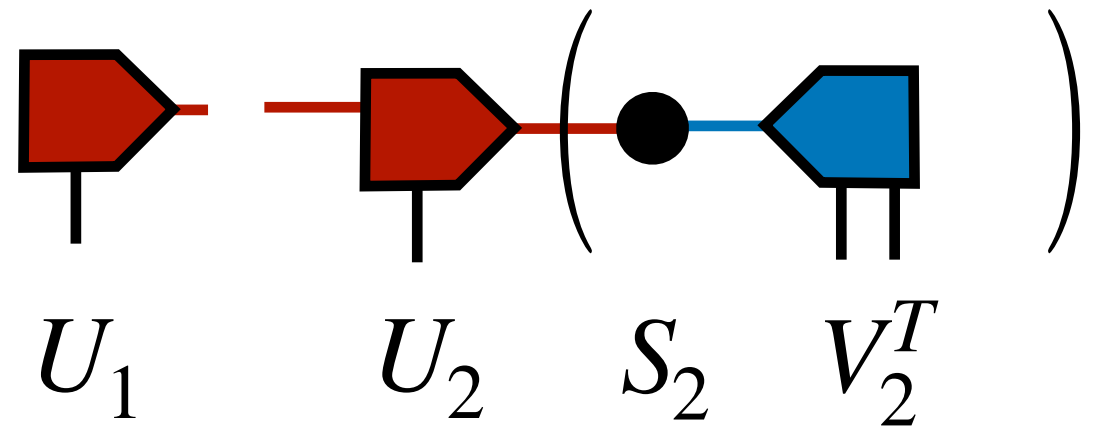
4. Multiply S_2 into V_2



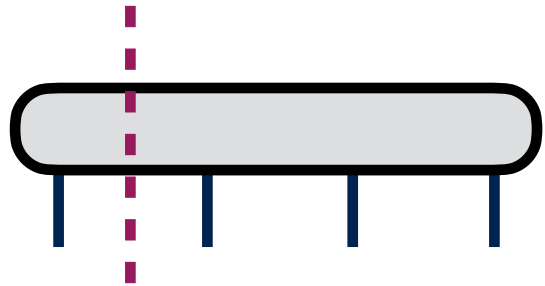
SVD
 $\approx$



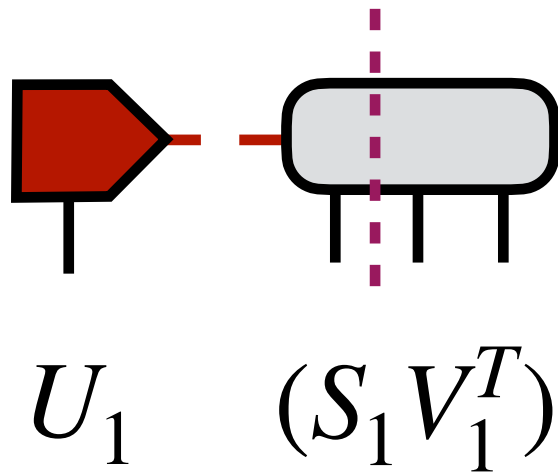
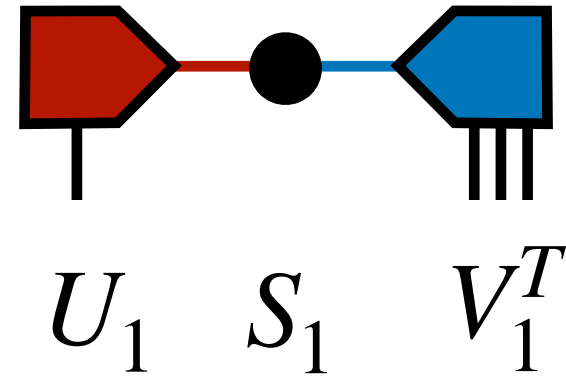
SVD
 $\approx$



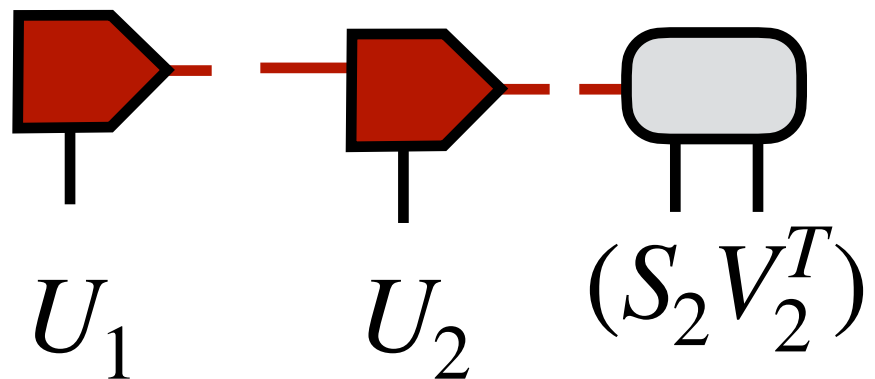
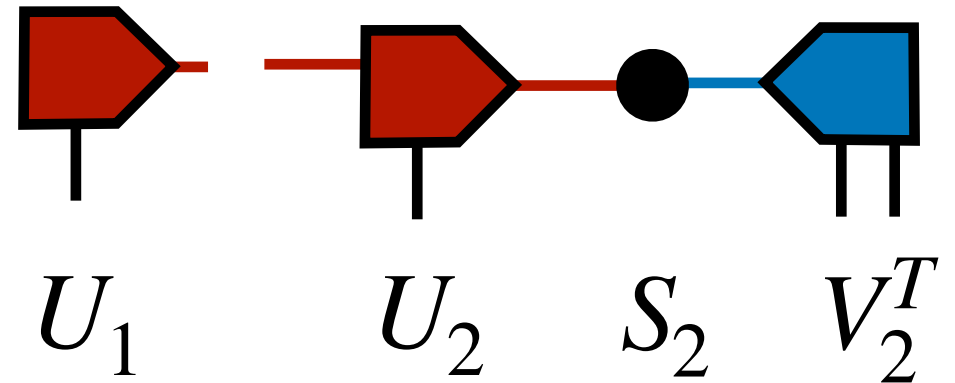
5. Finally SVD ($S_2 V_2^T$)



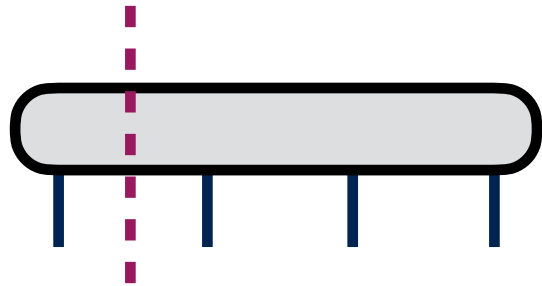
SVD
 $\approx$



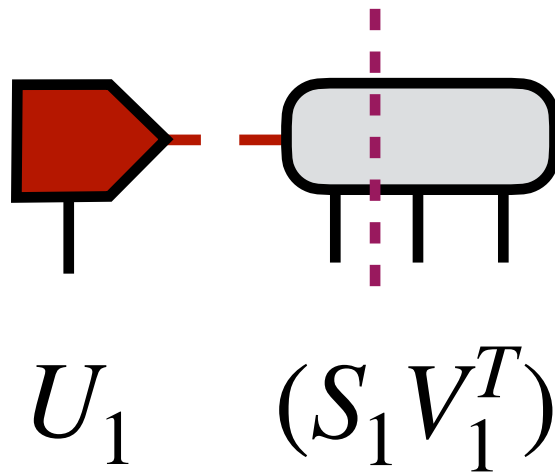
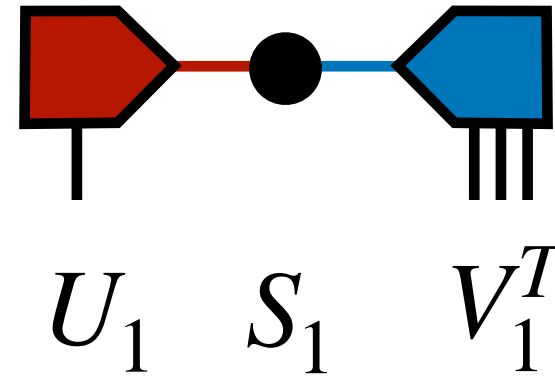
SVD
 $\approx$



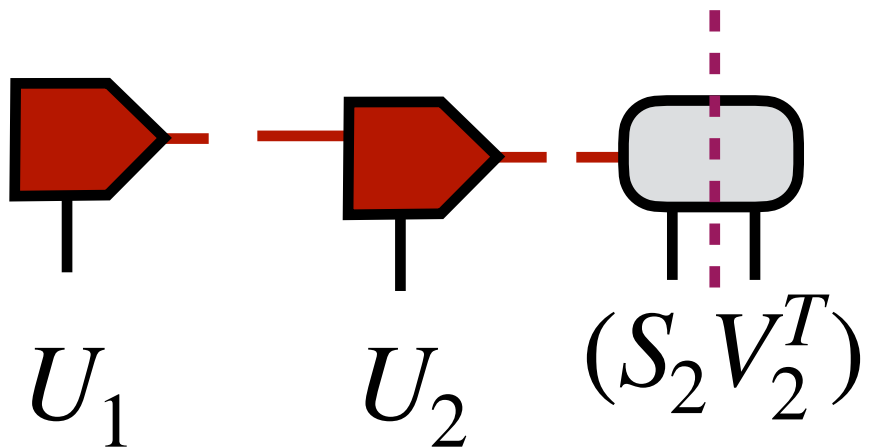
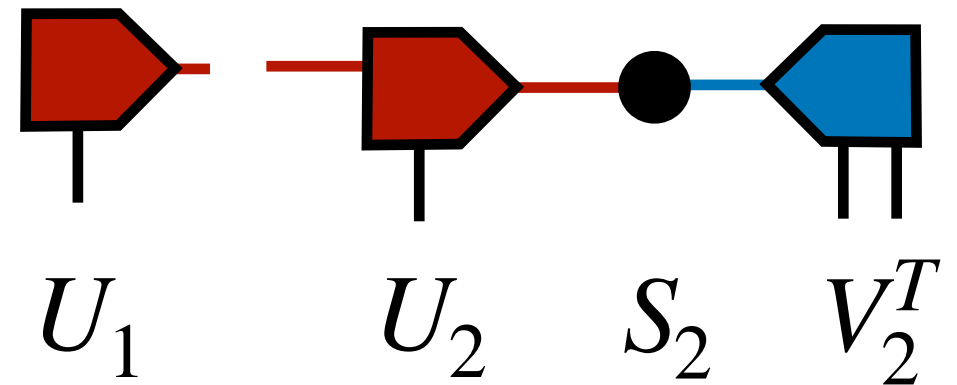
5. Finally SVD ($S_2 V_2^T$)



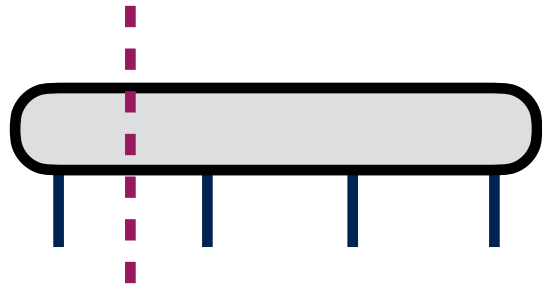
SVD
 $\approx$



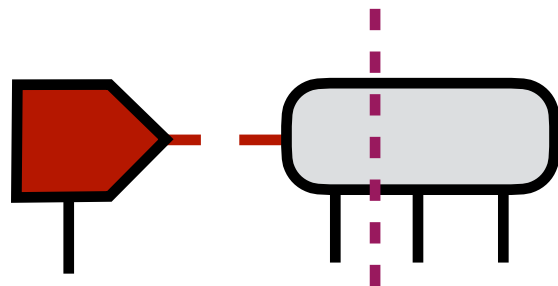
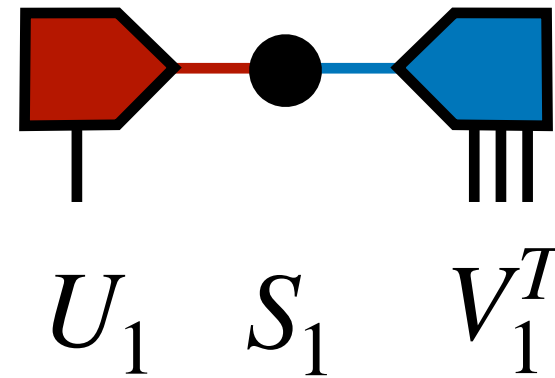
SVD
 $\approx$



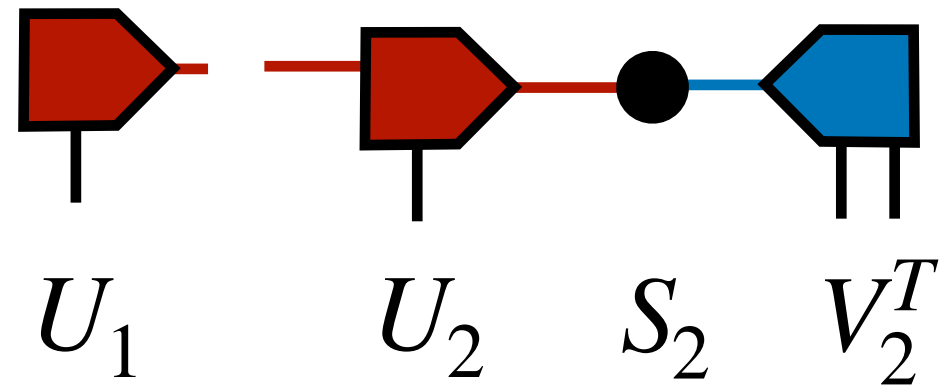
5. Finally SVD ($S_2 V_2^T$)



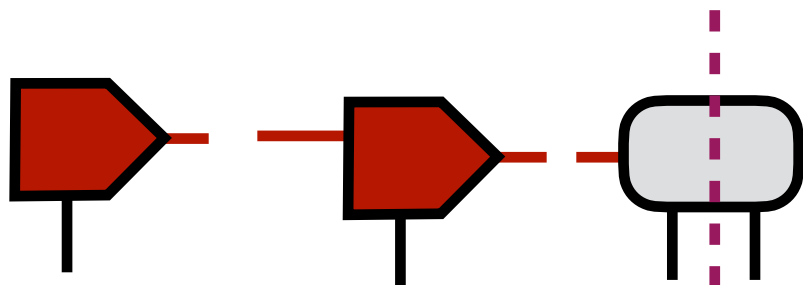
SVD
 $\approx$



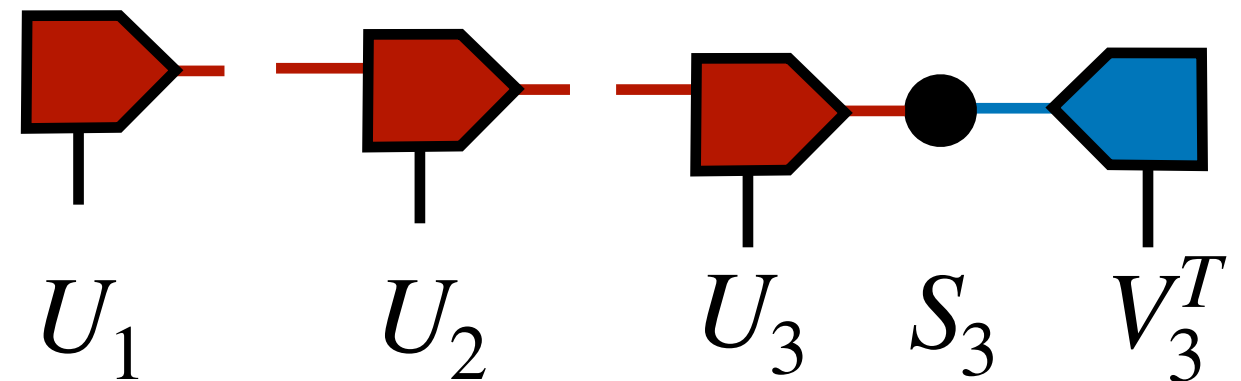
SVD
 $\approx$



$U_1 \quad (S_1 V_1^T)$

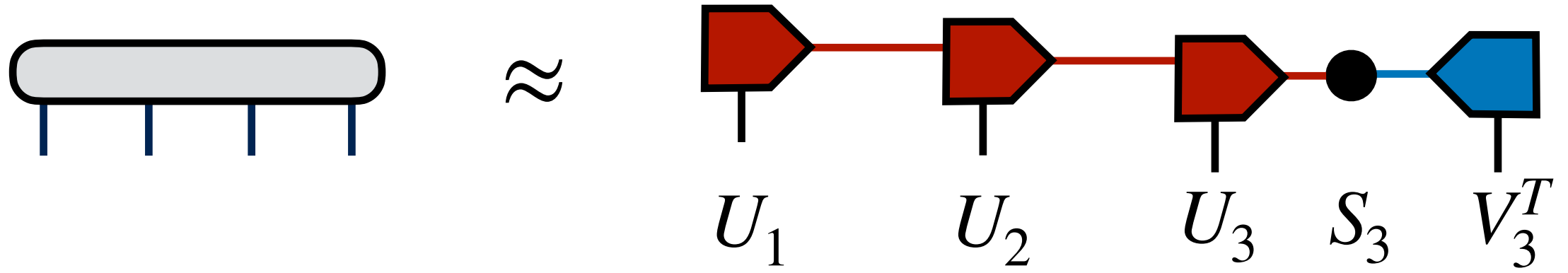


SVD
 $\approx$

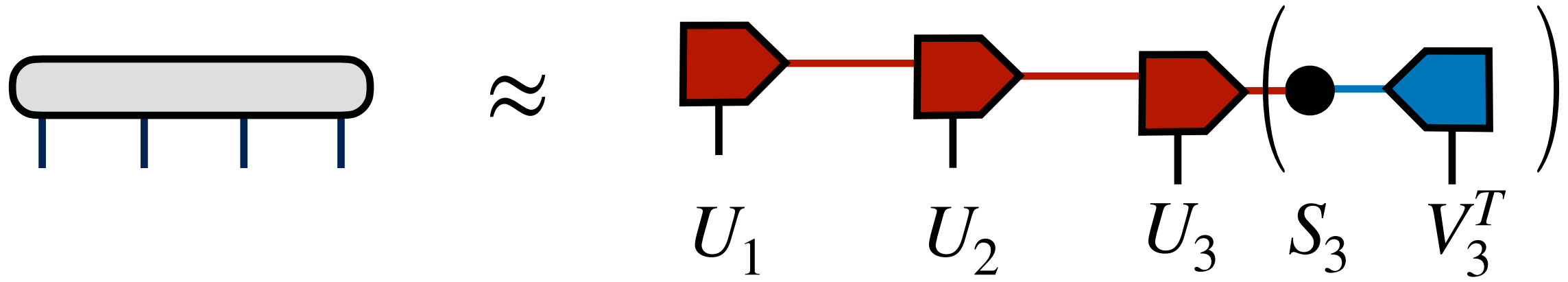


$U_1 \quad U_2 \quad (S_2 V_2^T)$

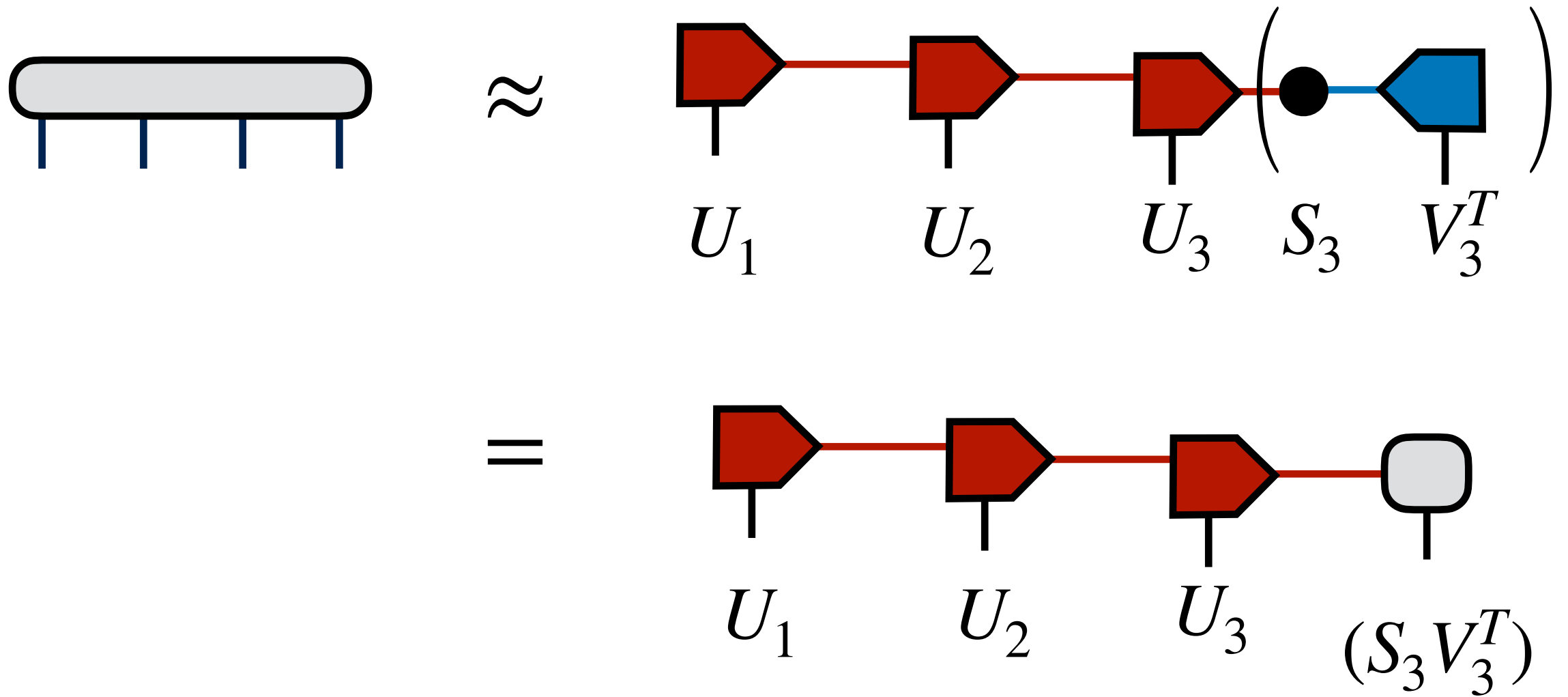
6. Interpret result as a tensor network



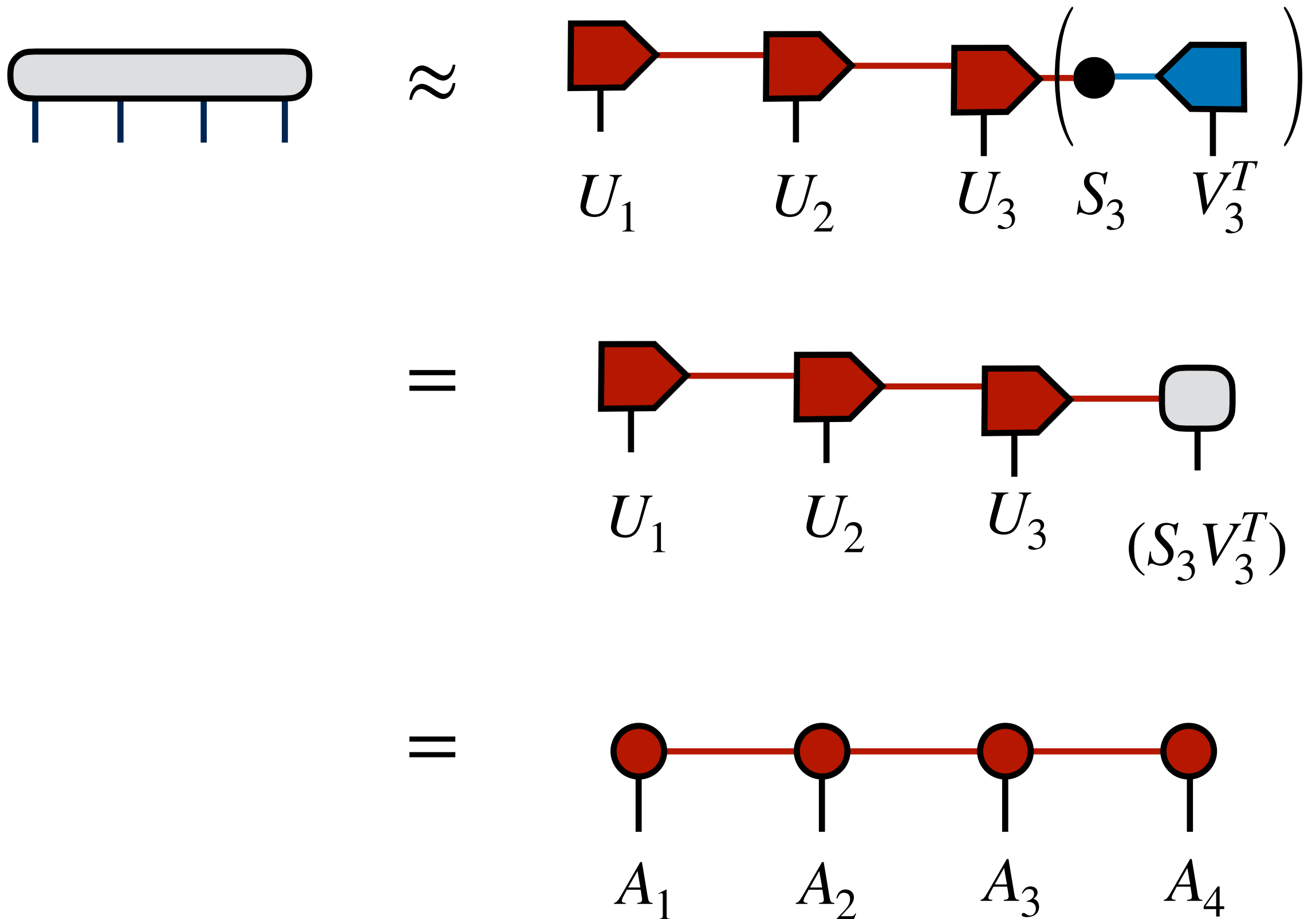
6. Interpret result as a tensor network



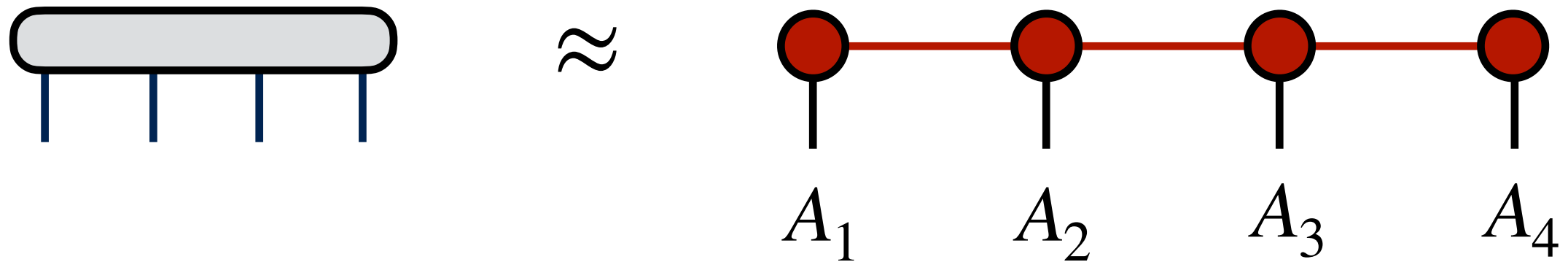
6. Interpret result as a tensor network



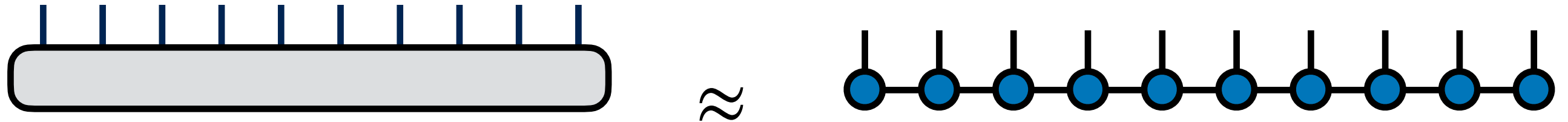
6. Interpret result as a tensor network



This decomposition is called a
"matrix product state" (MPS)



Two views of matrix product states



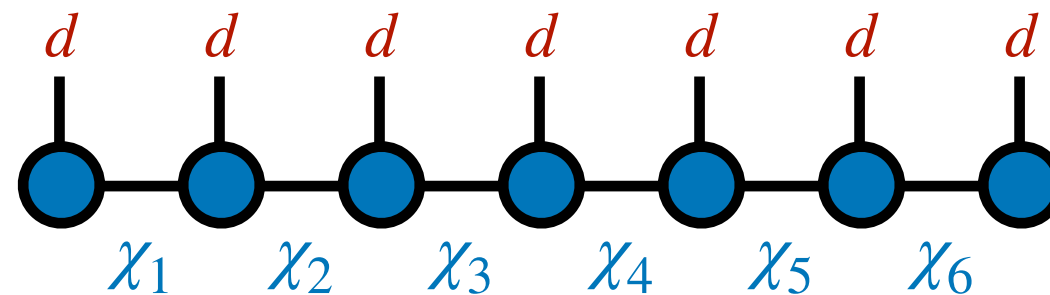
factorization of a tensor



*special subset of tensors
(low-rank tensor manifold)*

Matrix product state (MPS) tensor network

Size of a matrix product state (MPS)



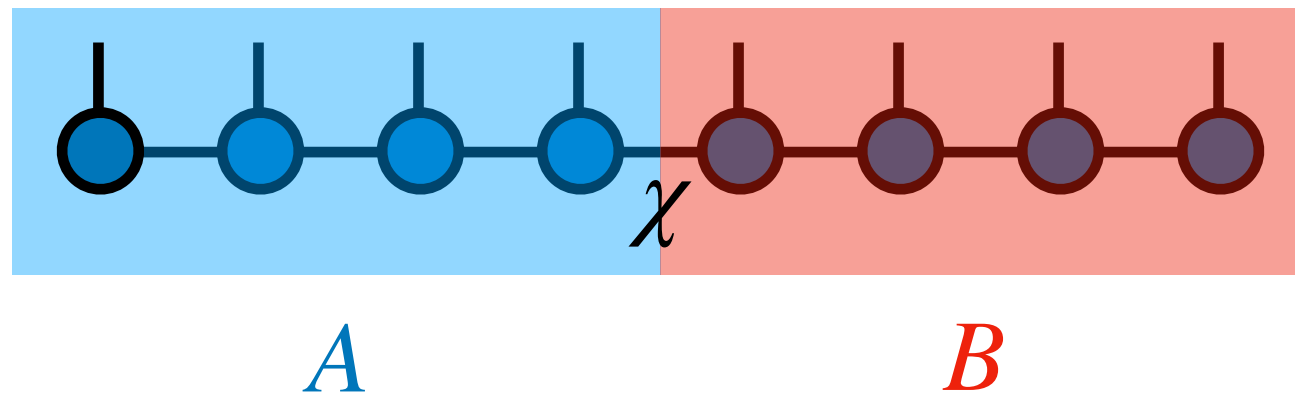
N tensors of size $\chi \cdot d \cdot \chi = d\chi^2$

Memory cost: $N d \chi^2 \quad (\ll d^N)$

Matrix product state (MPS) tensor network

What governs bond dimensions?

Divide sites into regions **A** and **B**



Can show $\chi \geq \exp(S)$ or $S \leq \log(\chi)$

Where S is the entanglement entropy* between **A** and **B**

More entanglement $\implies$ larger bond dimensions

* von Neumann entanglement entropy

Tensor Networks

Many efficient algorithms for tensor networks

Applying quantum gates (time evolution)

$$U|\Psi\rangle = \text{Diagram 1} \approx \text{Diagram 2}$$

The diagram shows the application of a quantum gate U to a state $|\Psi\rangle$. On the left, a horizontal chain of five blue circles (tensors) is shown. An orange rectangular box, representing the gate U , is connected to the third and fourth tensors from the left. To the right of this chain is an approximation symbol $\approx$. On the right, a similar horizontal chain of five tensors is shown, but the third and fourth tensors are now green, indicating a contraction or simplification of the network.

"Perfect sampling" from MPS tensor networks

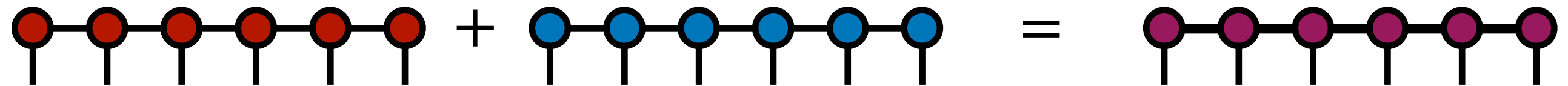
$$\text{Diagram 3} = \text{Diagram 4}$$

The diagram illustrates "Perfect sampling" from MPS tensor networks. On the left, a tensor network is shown with a vertical stack of two blue circles connected by a loop. A grey die icon is positioned above the top circle. Two arrows point from the right side of the loop to the labels $|0\rangle$ and $|1\rangle$. Below the vertical stack is the label ρ_1 . To the right of this is an equals sign. On the right side of the equals sign, a horizontal chain of five tensors is shown. The first tensor is a box containing $|1\rangle$, and the second tensor is a box containing $\langle 1|$. The remaining three tensors in the chain are blue circles. The entire chain is connected by horizontal lines, representing the contraction of the tensor network.

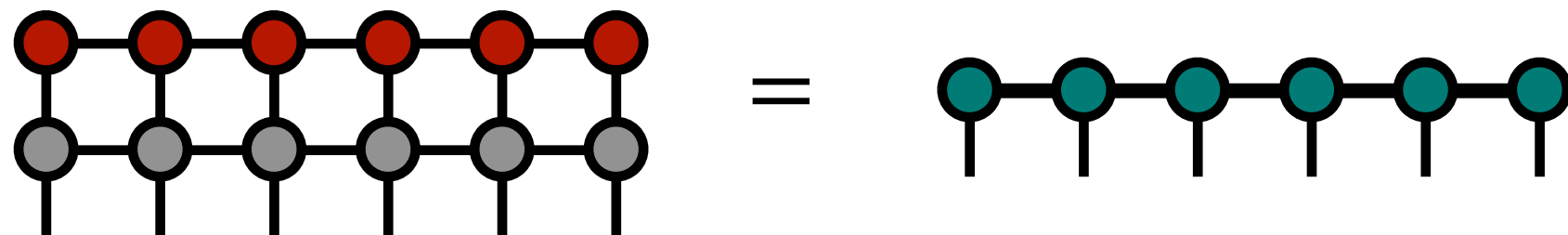
Tensor Networks

Many efficient algorithms for tensor networks

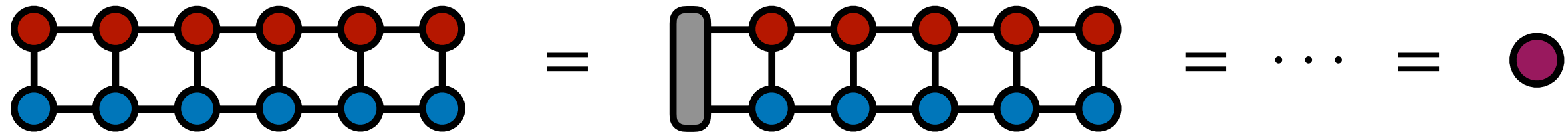
Summing MPS in compressed form



Multiply by other networks (MPO operator $\times$ MPS)



Inner product of two MPS tensors



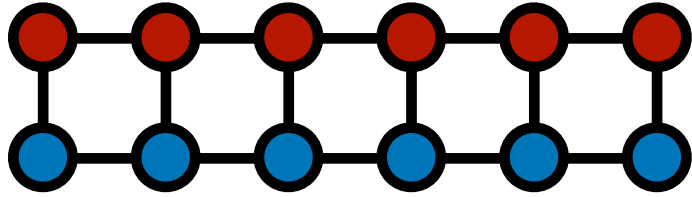
If each MPS represents tensor with 40 indices of dimension 2

Then above algorithm computes dot product of two 'vectors' of a *trillion* entries each

Takes $\sim 10\text{ms}$ for bond dimension $\chi = 100$

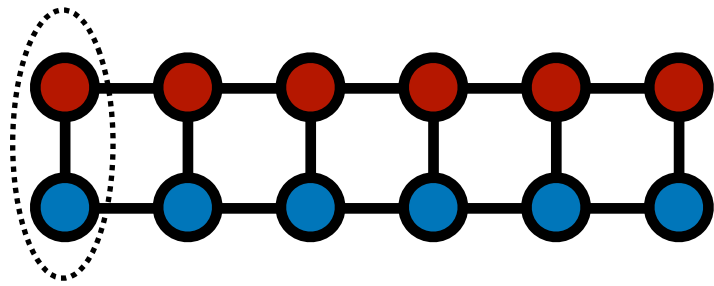
More detailed tensor network algorithm

Inner product of two MPS tensors



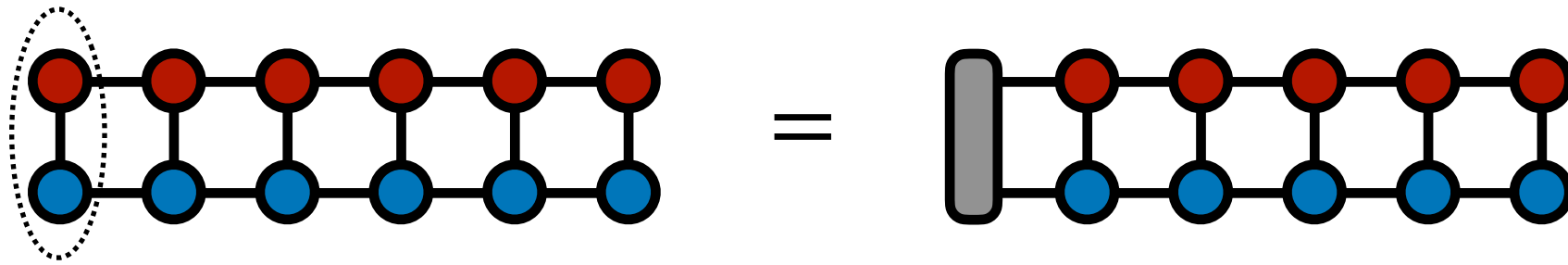
More detailed tensor network algorithm

Inner product of two MPS tensors



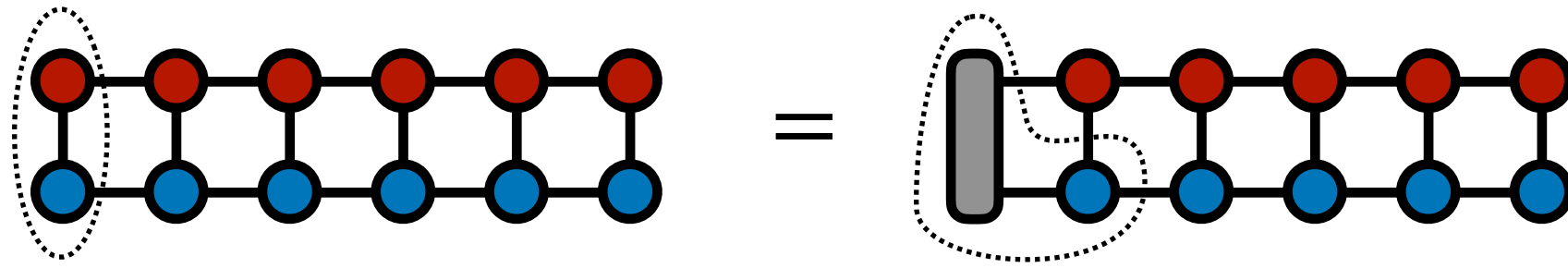
More detailed tensor network algorithm

Inner product of two MPS tensors



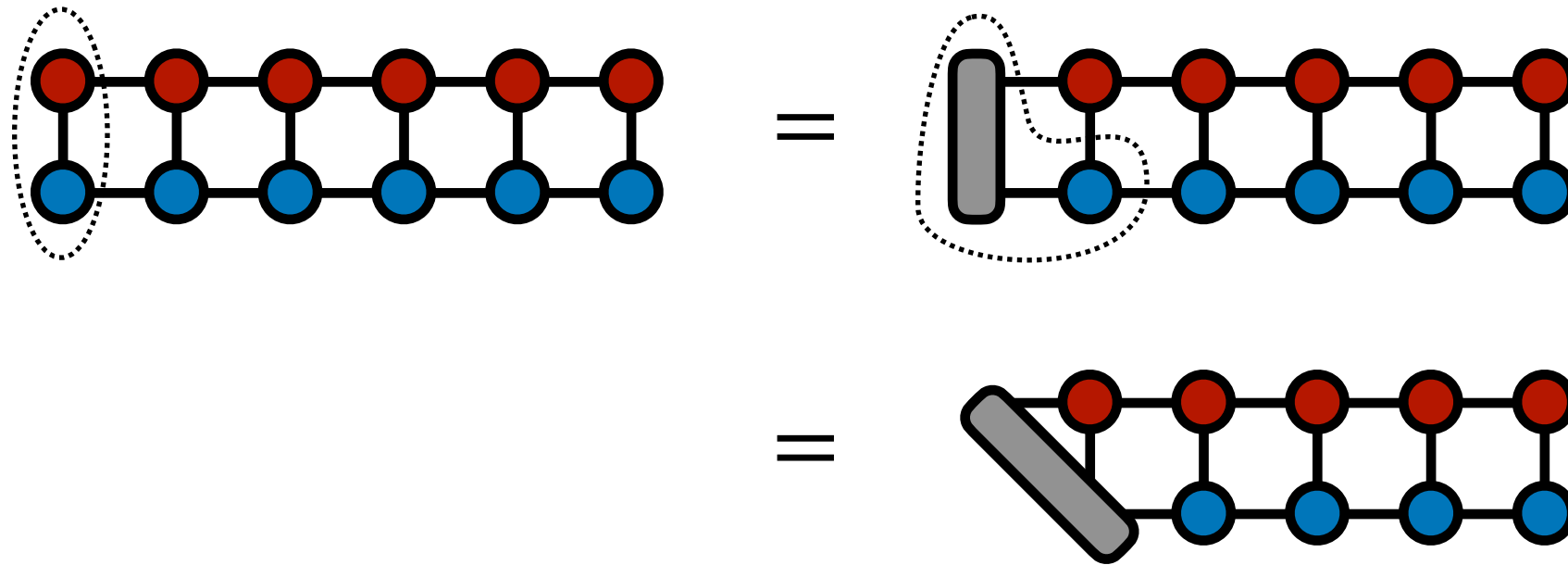
More detailed tensor network algorithm

Inner product of two MPS tensors



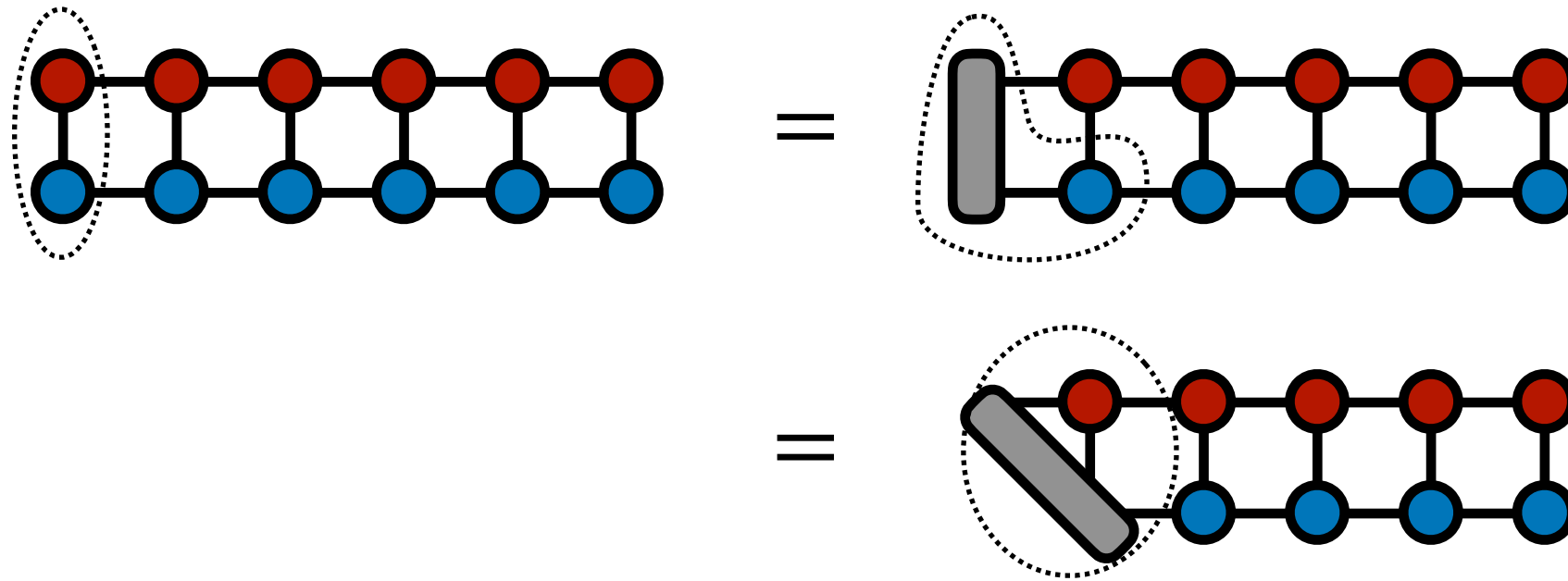
More detailed tensor network algorithm

Inner product of two MPS tensors



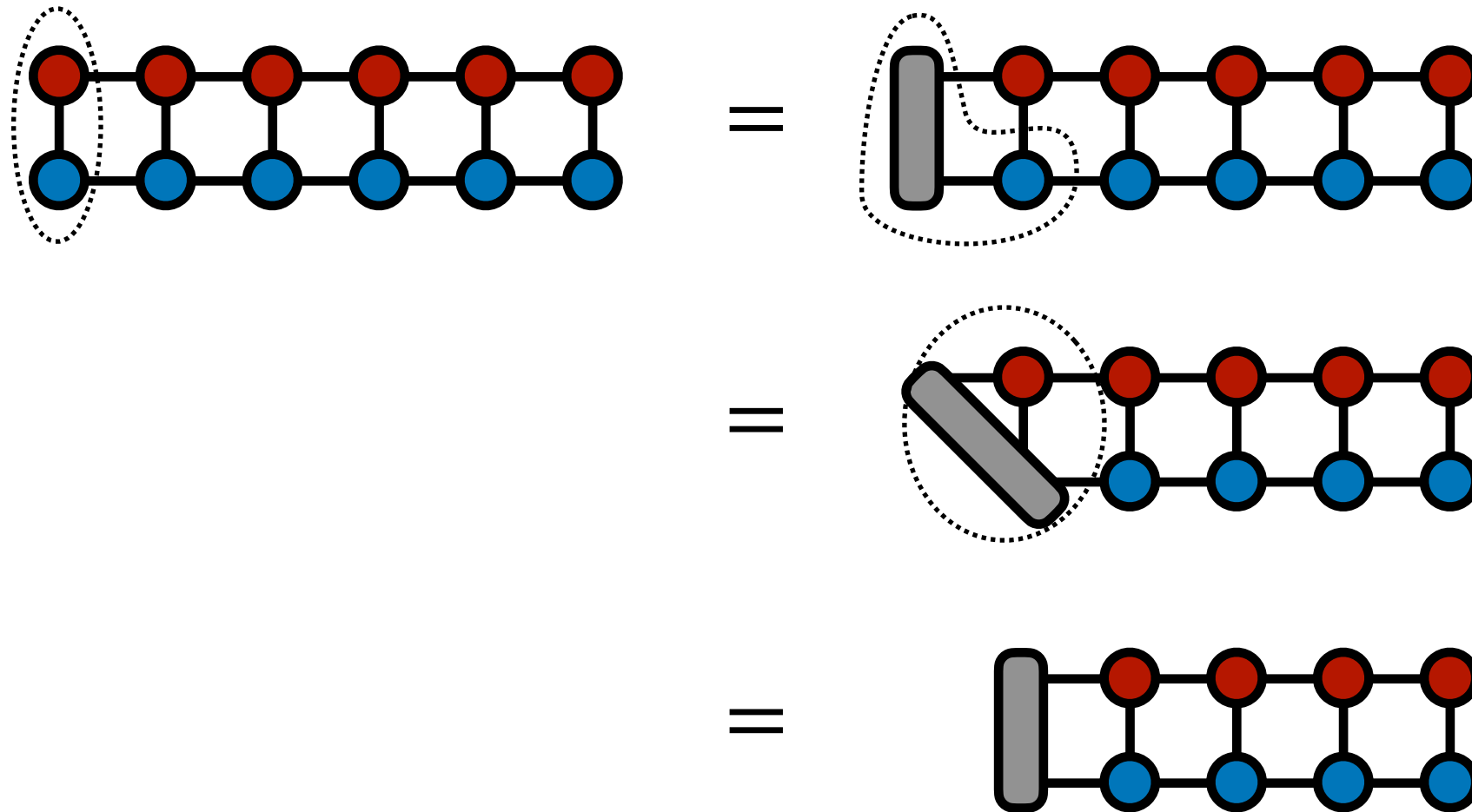
More detailed tensor network algorithm

Inner product of two MPS tensors



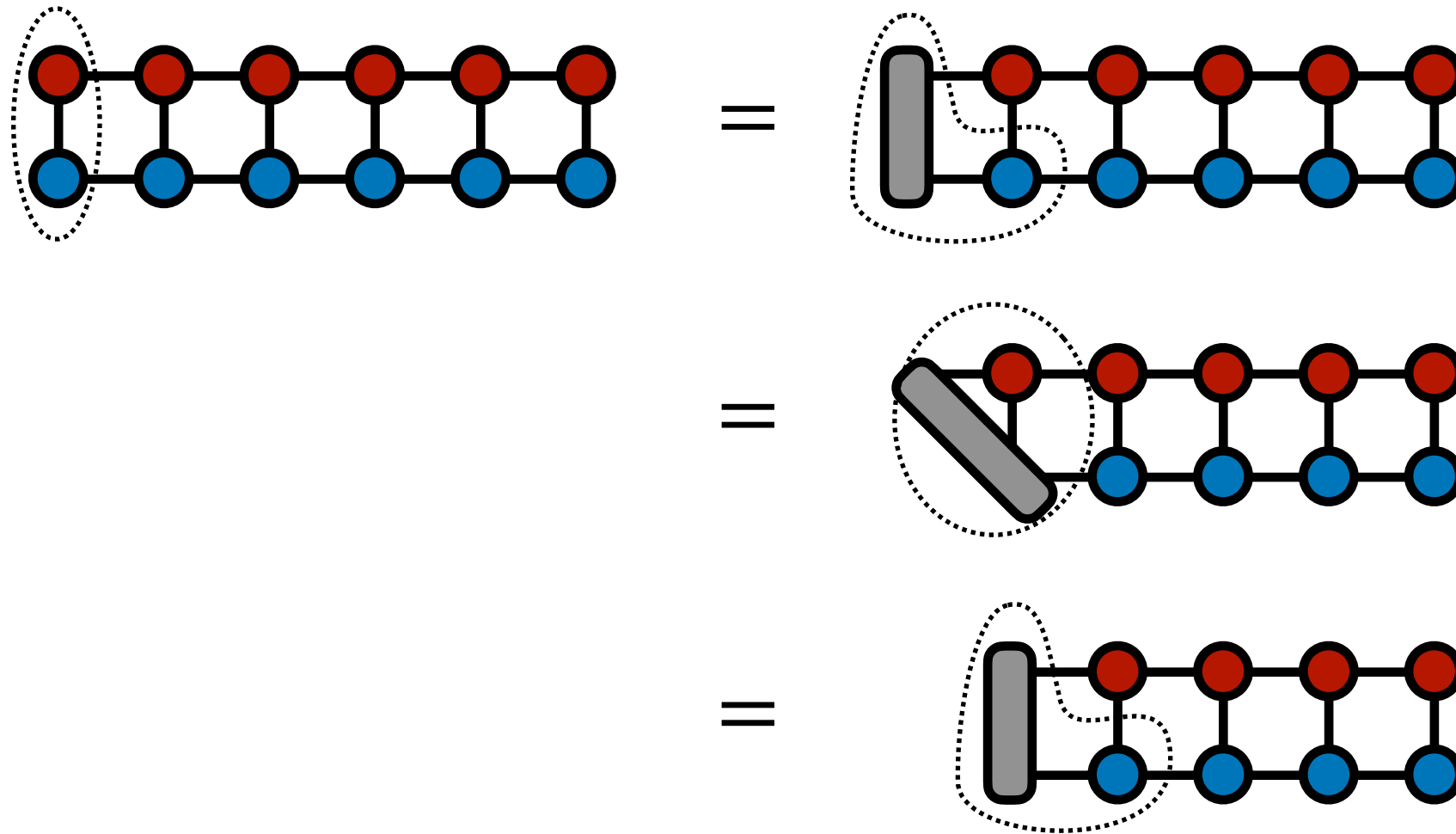
More detailed tensor network algorithm

Inner product of two MPS tensors



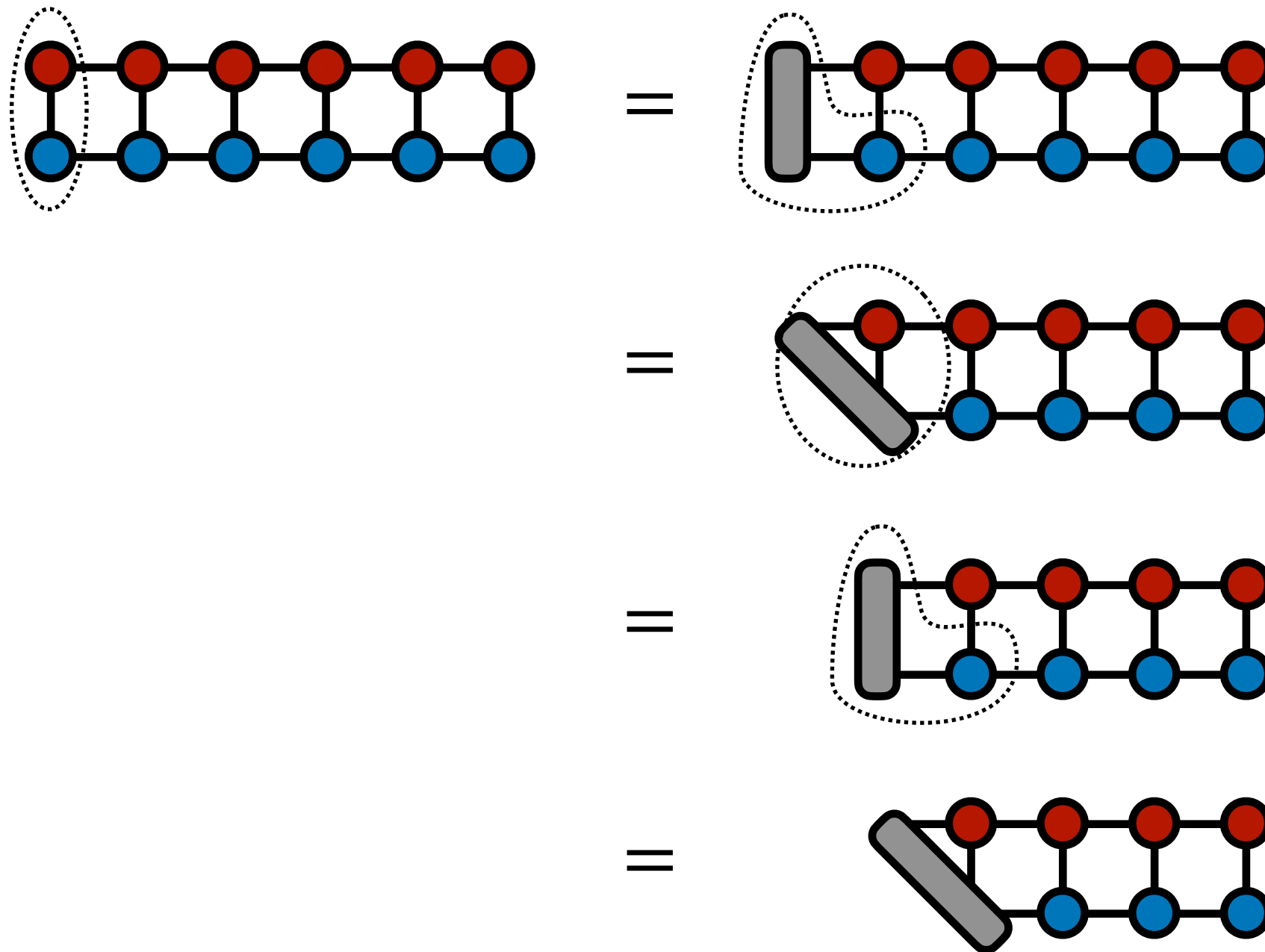
More detailed tensor network algorithm

Inner product of two MPS tensors



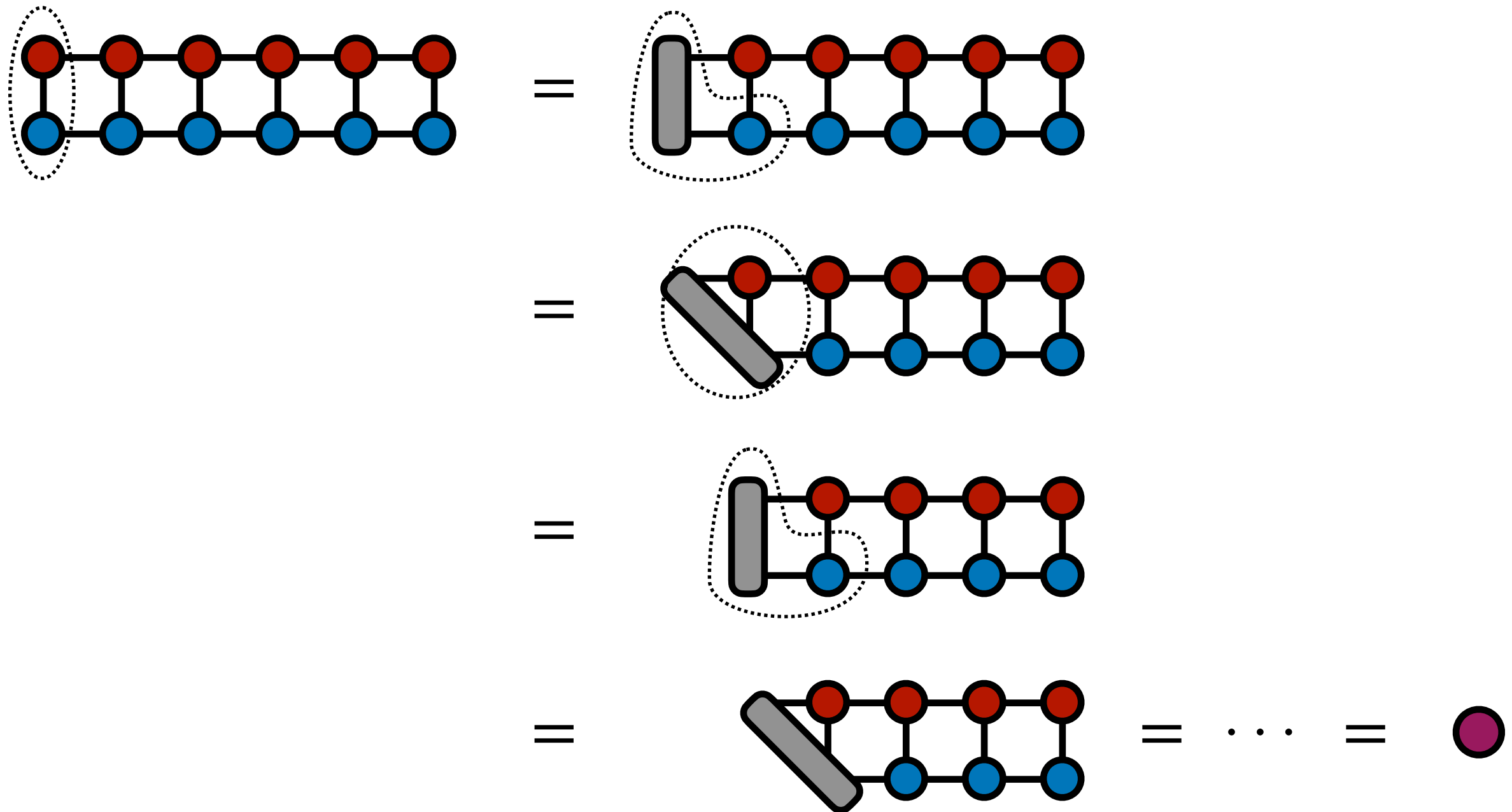
More detailed tensor network algorithm

Inner product of two MPS tensors



More detailed tensor network algorithm

Inner product of two MPS tensors



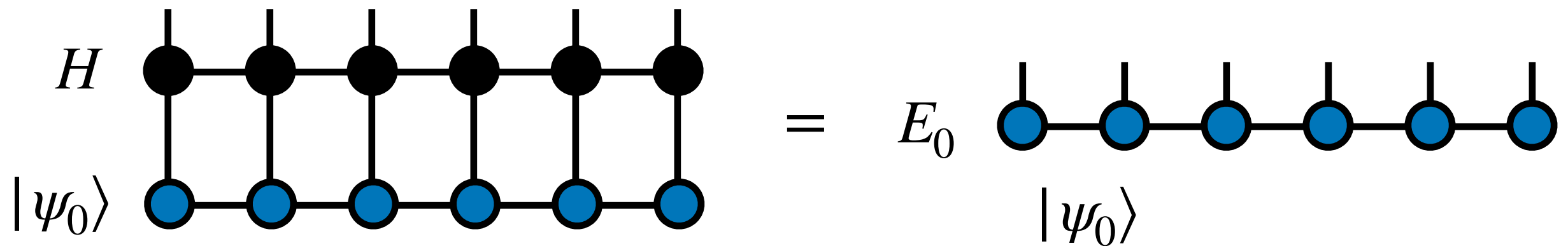
Cost $\sim \chi^3$, memory usage $\sim \chi^2$

Tensor Networks

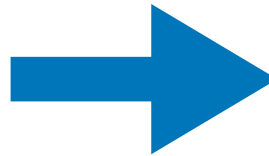
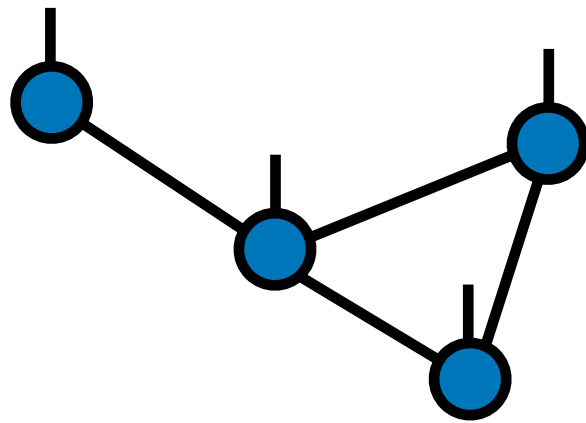
Many efficient algorithms for tensor networks

In the next talk we will study an important algorithm more...

DMRG algorithm



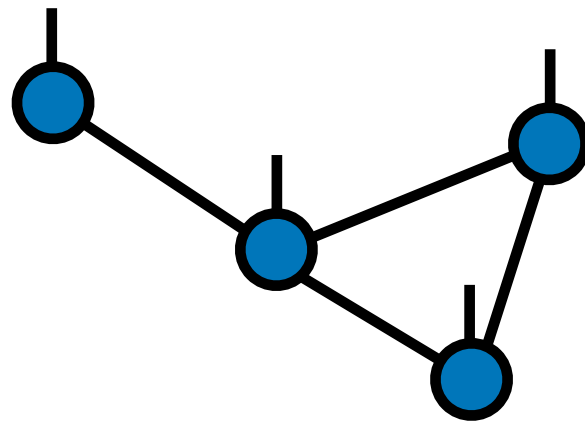
The ITensor Library



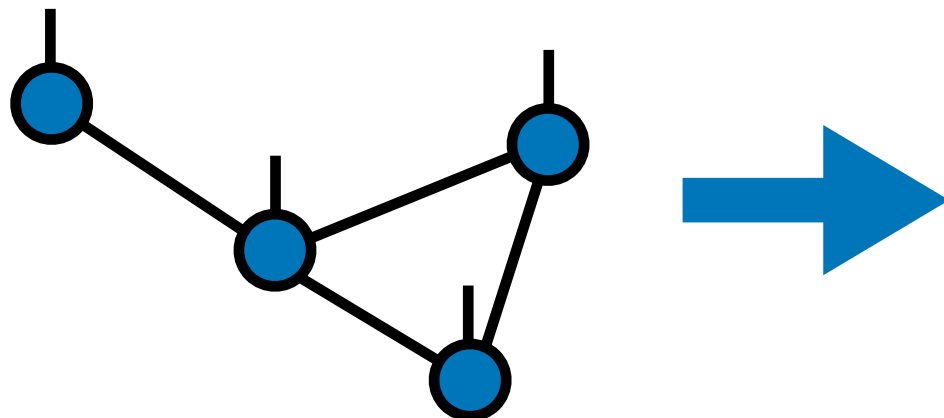
```
function hubbard_chain(sites; N, t=1.0, U=4.0)
  terms = OpSum()
  for j in 1:(N - 1)
    terms -= t, "Cdagup", j, "Cup", j+1
    terms -= t, "Cdagup", j+1, "Cup", j
    terms -= t, "Cdagdn", j, "Cdn", j+1
    terms -= t, "Cdagdn", j+1, "Cdn", j
  end
  [...]
end
```

Motivation

Tensor diagrams are a powerful notation for tensor networks and tensor contractions

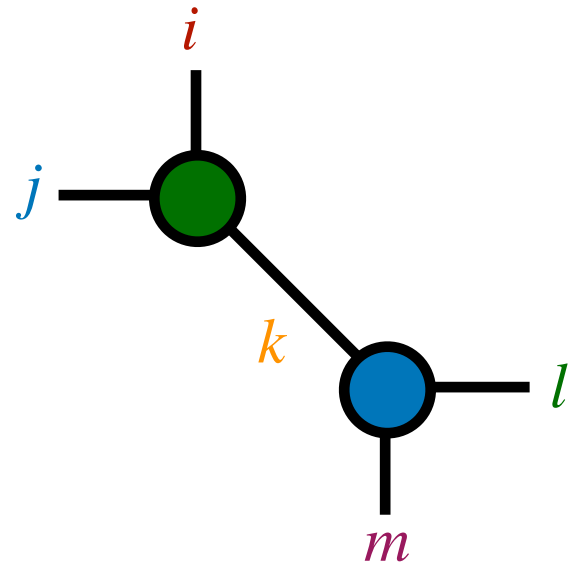


What if there was software modeled on tensor diagrams?



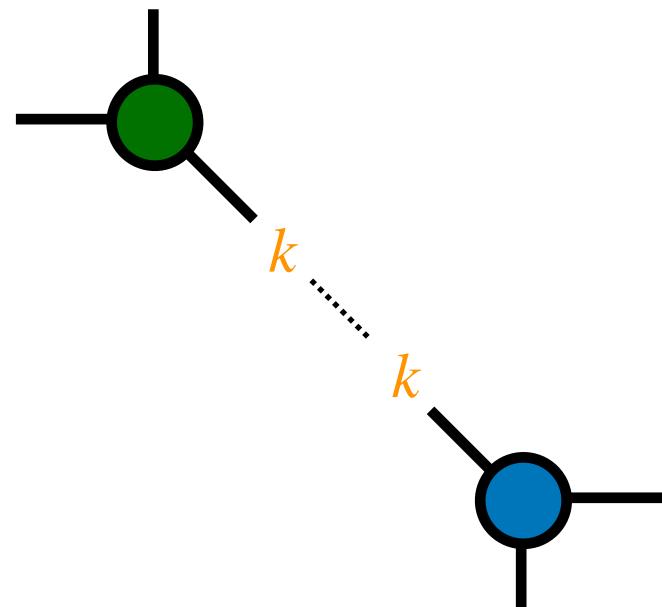
```
function hubbard_chain(sites; N, t=1.0, U=4.0)
  terms = OpSum()
  for j in 1:(N - 1)
    terms -= t, "Cdagup", j, "Cup", j+1
    terms -= t, "Cdagup", j+1, "Cup", j
    terms -= t, "Cdagdn", j, "Cdn", j+1
    terms -= t, "Cdagdn", j+1, "Cdn", j
  end
  [...]
end
```

Reminder: connecting index line means contraction

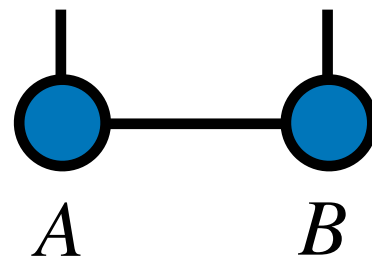


Which indices can connect?

Should be the **same** index – same size and meaning



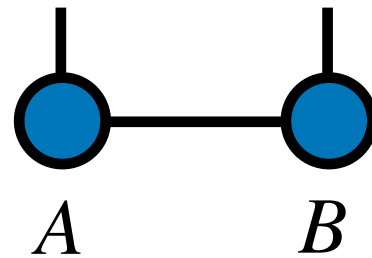
Some tensor libraries use index positions...



```
contract(A,{1,2},B,{2,3})
```

Must know contracted index is:
second index of A , first index of B

String identifiers help...



```
contract(A,{'i','j'},B,{'j','k'})
```

But the strings are temporary

Not unique – might repeat later

The ITensor Software

*Can indices "remember"
their identity?*

Before making an ITensor, we make indices

```
julia> i = Index(3)  
          (dim=3|id=807)
```

i
|

Before making an ITensor, we make indices

```
julia> i = Index(3)  
          (dim=3|id=807)
```

i
|

↑ ↑
dimension unique identifier

Identifier lets index "recognize" copies of itself

"Intelligent" index

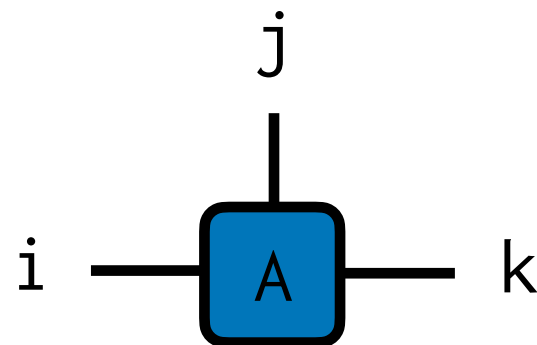
Making some indices

```
julia> i = Index(3)
julia> j = Index(2)
julia> k = Index(4)
```

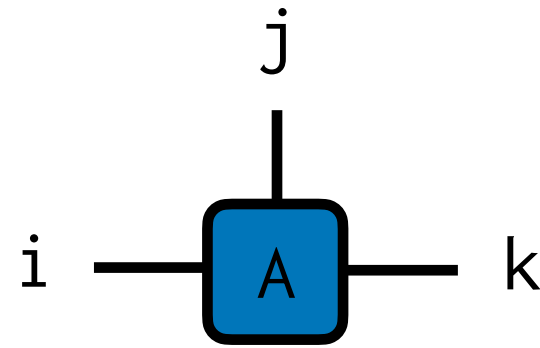
i j k
| | |

Now we can make tensors

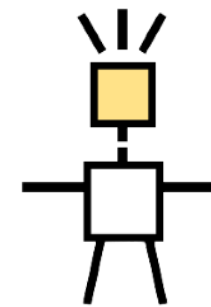
```
julia> A = ITensor(i,j,k)
```



```
julia> A = ITensor(i,j,k)
```



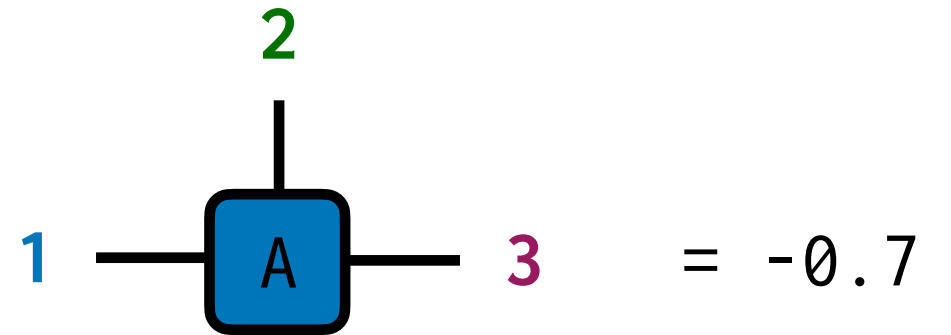
Indices are intelligent –
an intelligent tensor or **ITensor** *



* Not iTensor or i-Tensor ... 😊

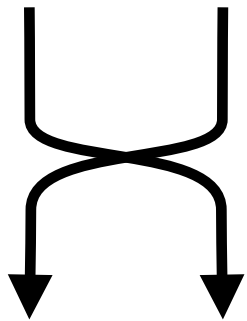
Setting element of an ITensor

```
julia> A[i=>1,j=>2,k=>3] = -0.7
```

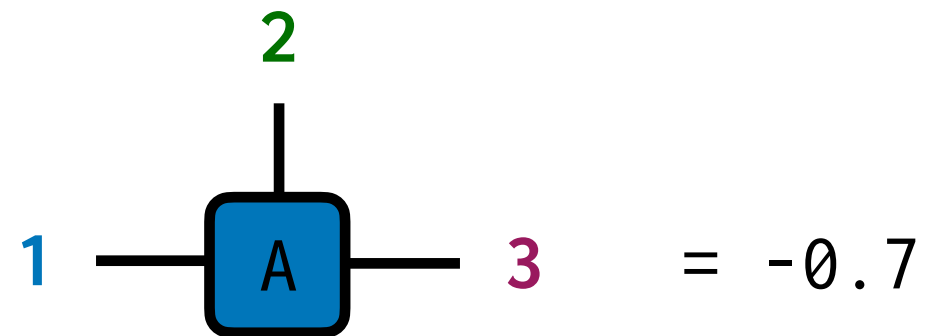


Any order allowed since indices known

```
julia> A[i=>1,j=>2,k=>3] = -0.7
```



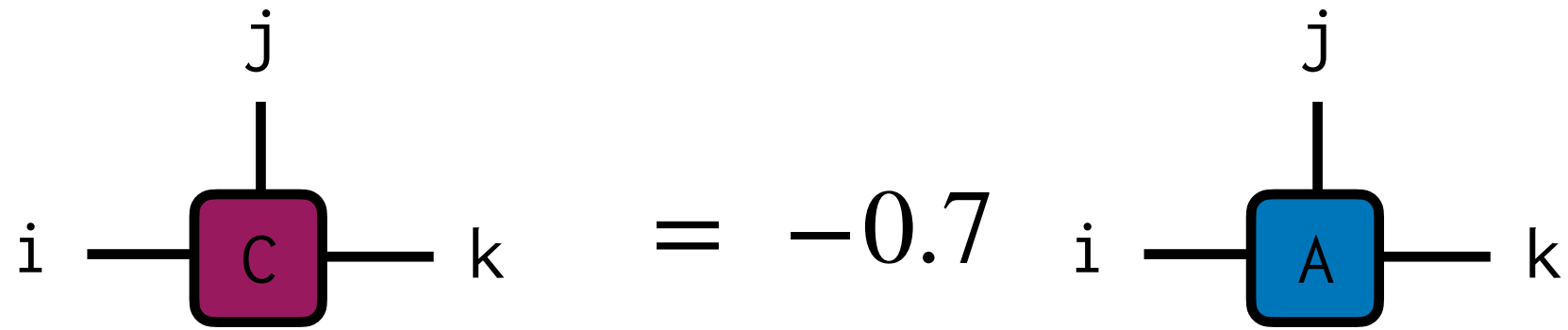
```
julia> A[j=>2,i=>1,k=>3] = -0.7
```



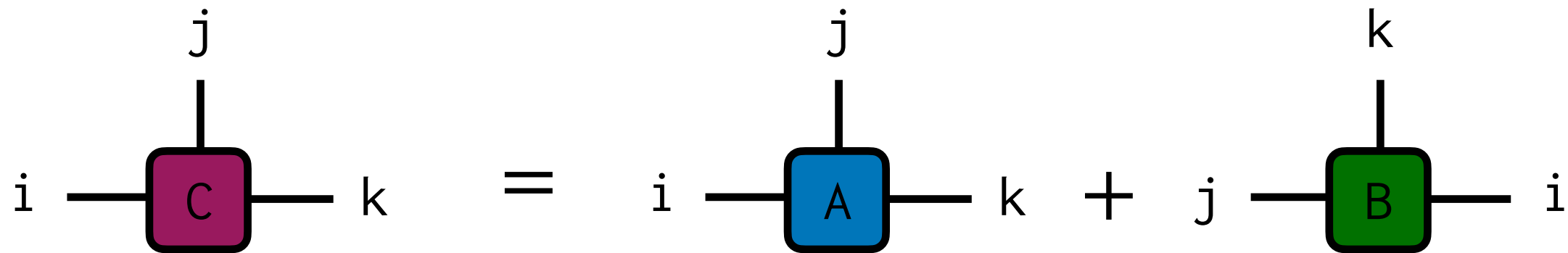
Both commands have same effect

Operations with ITensors

Naturally, ITensors also support vector operators like multiplication by scalar and addition


$$\begin{array}{c} j \\ | \\ i - \boxed{C} - k \end{array} = -0.7 \begin{array}{c} j \\ | \\ i - \boxed{A} - k \end{array}$$

```
julia> C = -0.7 * A
```


$$\begin{array}{c} j \\ | \\ i - \boxed{C} - k \end{array} = \begin{array}{c} j \\ | \\ i - \boxed{A} - k \end{array} + \begin{array}{c} k \\ | \\ j - \boxed{B} - i \end{array}$$

```
julia> C = A + B
```

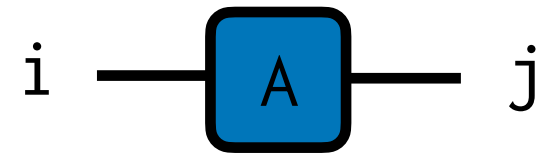
Indices must match, but different order ok

Contracting ITensors

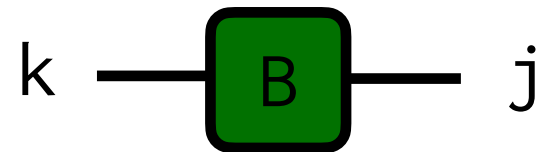
What about **tensor contraction**?

Given two ITensors

```
julia> A = ITensor(i,j)
```

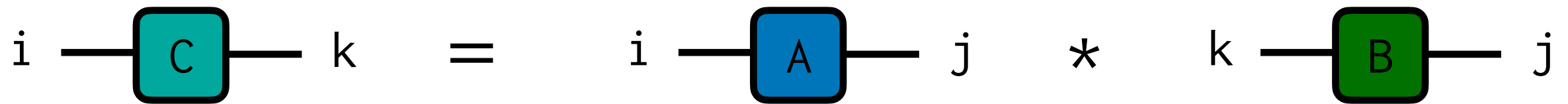


```
julia> B = ITensor(k,j)
```



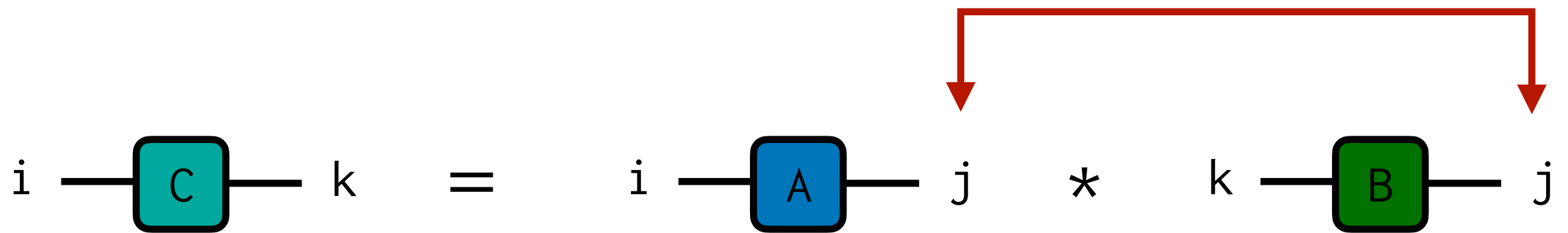
Contract ITensors with "*"

```
julia> C = A * B
```



Contract ITensors with "*"

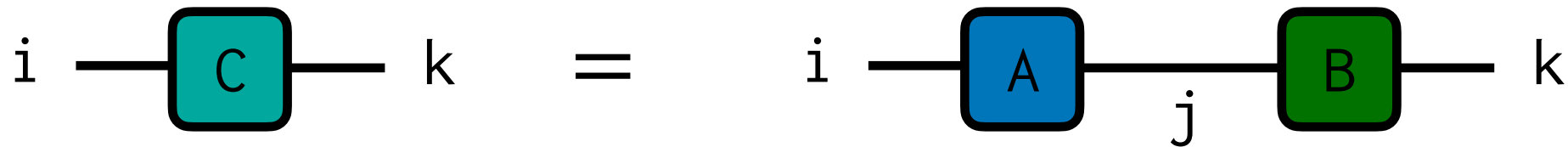
```
julia> C = A * B
```



Matching indices recognized

Contract ITensors with "*"

```
julia> C = A * B
```

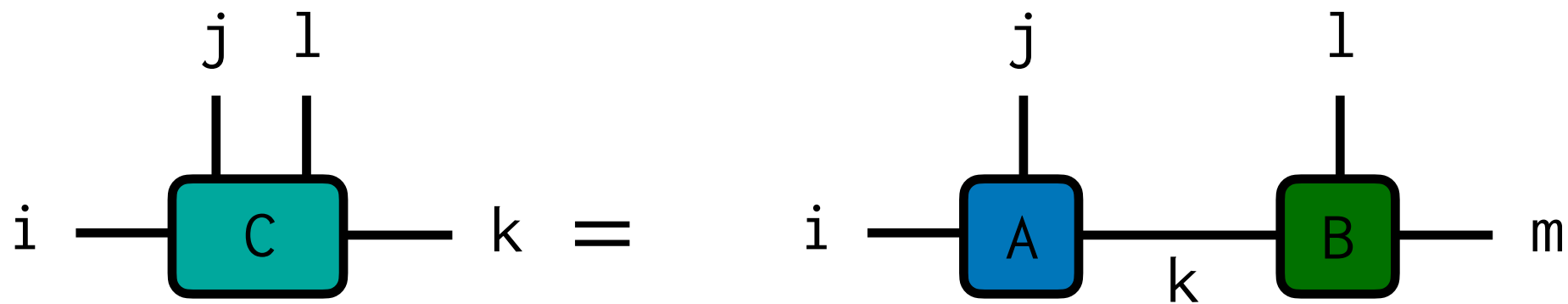


Matching indices recognized

Tensors permuted and contracted

General tensor contractions always

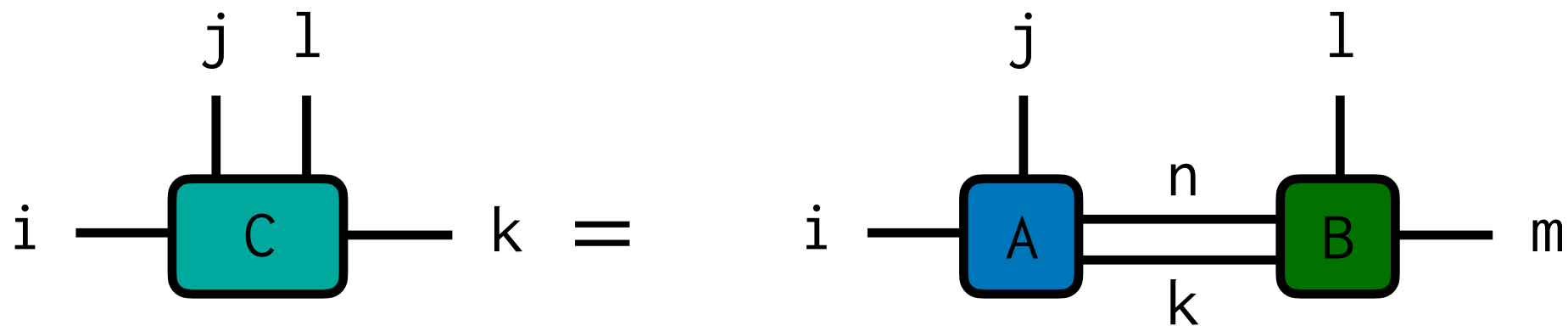
```
julia> C = A * B
```



Assuming all matching indices should be contracted

General tensor contractions always

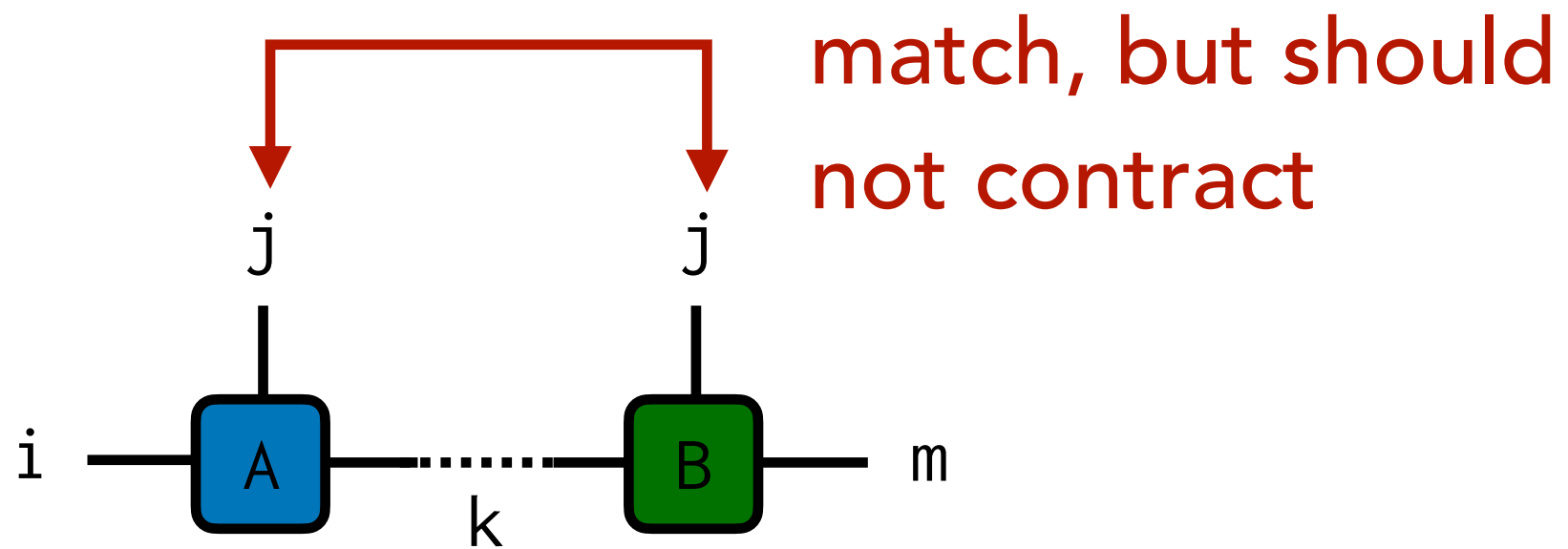
```
julia> C = A * B
```



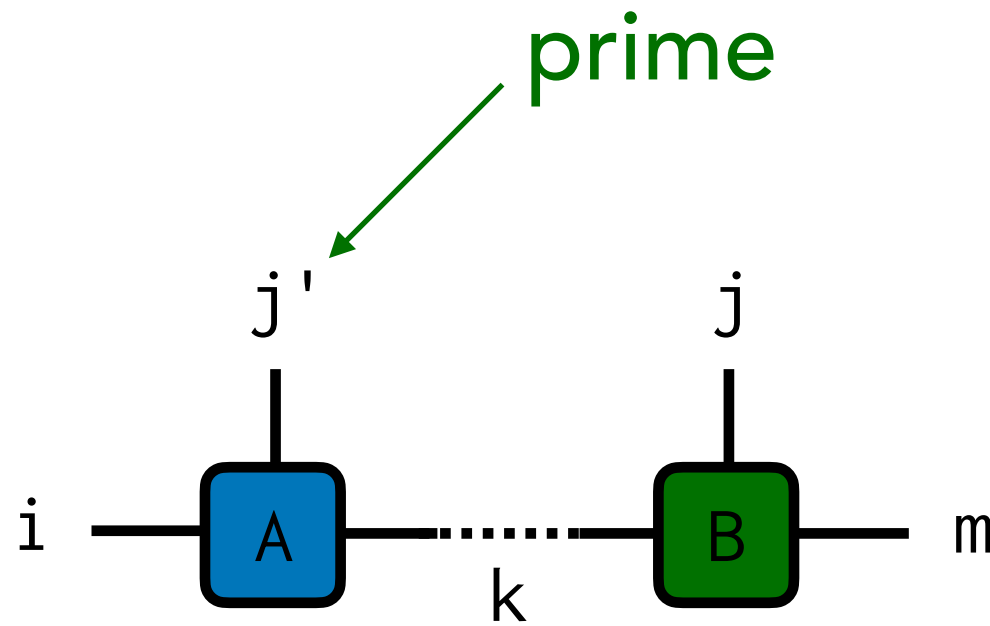
Works for multiple contracted indices, of course

Priming Indices

What if certain matching indices should stay uncontracted?

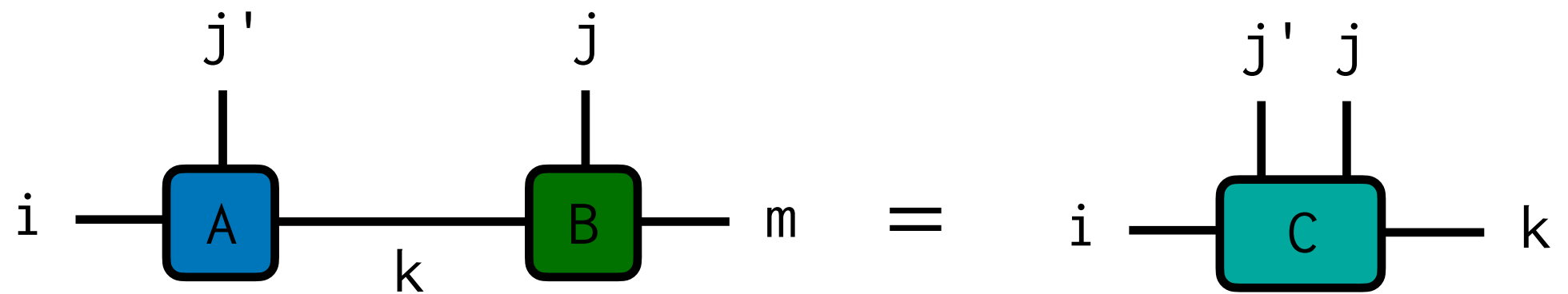


Prime an index to make it different



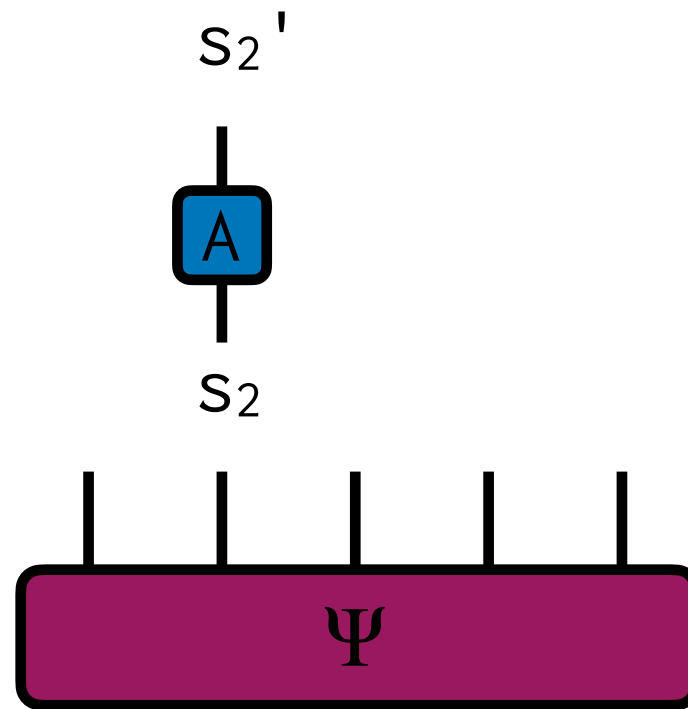
```
julia> prime(A,j) # prime index j only
```

Contracting now gives desired result



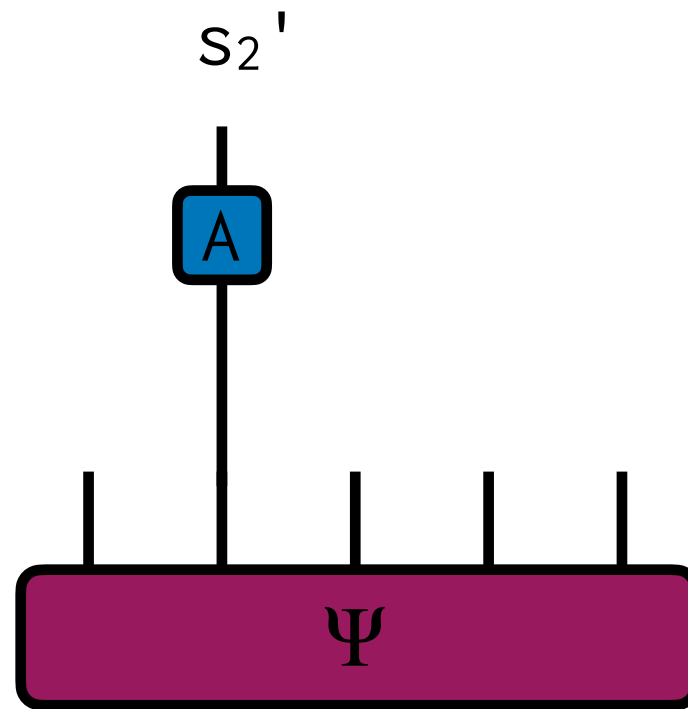
```
julia> Aprime = prime(A,j) # prime index j only  
julia> C = Aprime * B
```

Key use of priming is applying operators



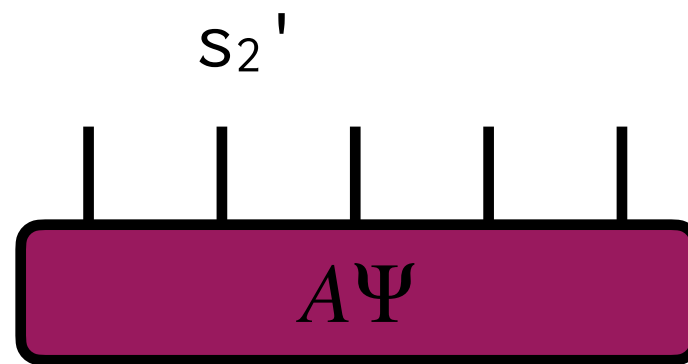
```
julia> A_psi = A * psi
```

Key use of priming is applying operators



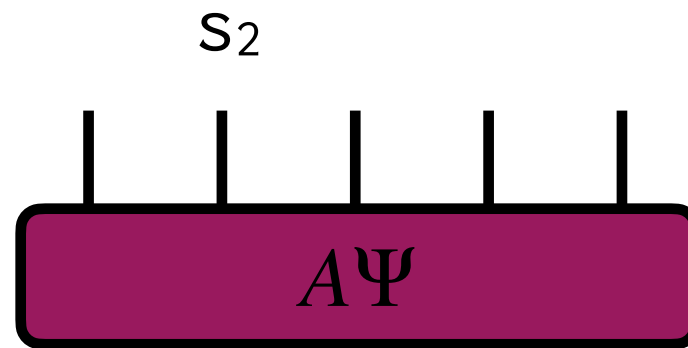
```
julia> A_psi = A * psi
```

Remove prime with `noprime` function



```
julia> A_psi = A * psi  
julia> A_psi = noprime(A_psi)
```

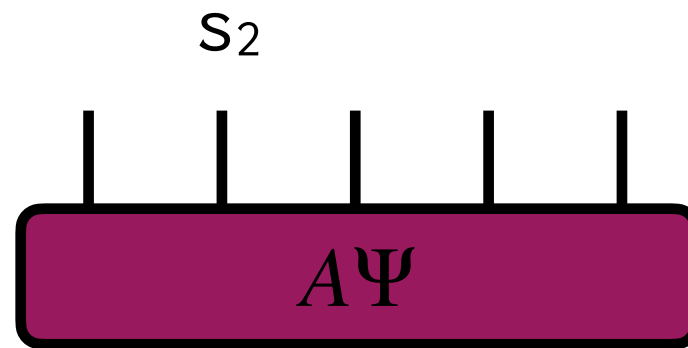
Remove prime with `noprime` function



```
julia> A_psi = A * psi  
julia> A_psi = noprime(A_psi)
```


Or use the shorthand

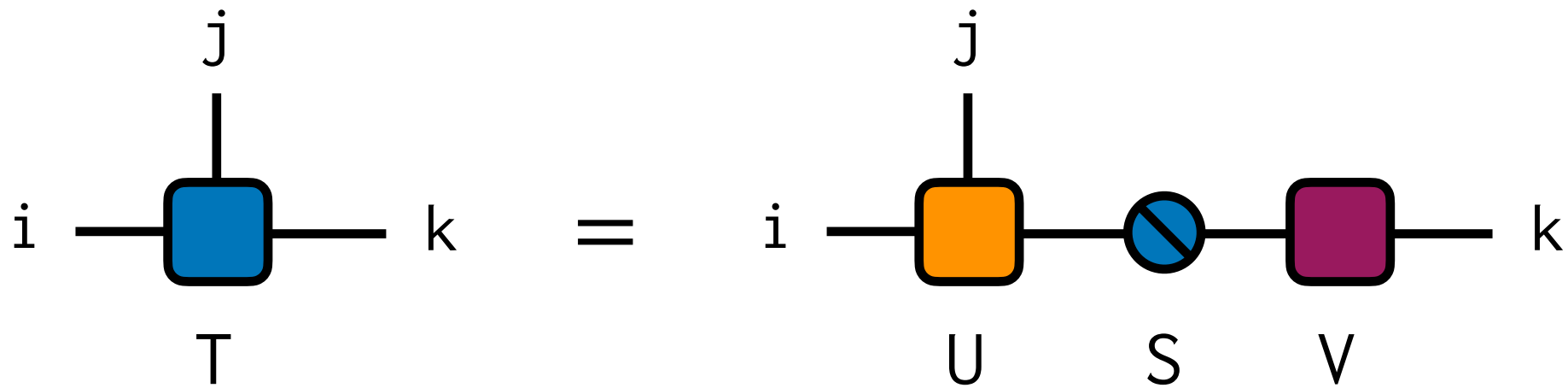
`apply`



```
julia> A_psi = apply(A, psi)
```


ITensor Decompositions

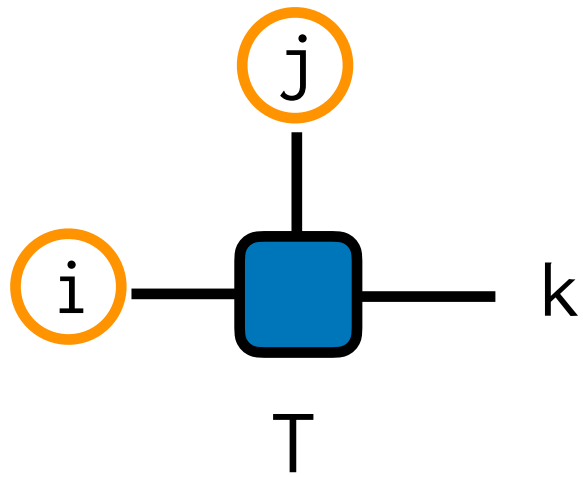
Recall we can SVD a tensor



```
julia> T ≈ U * S * V  
true
```

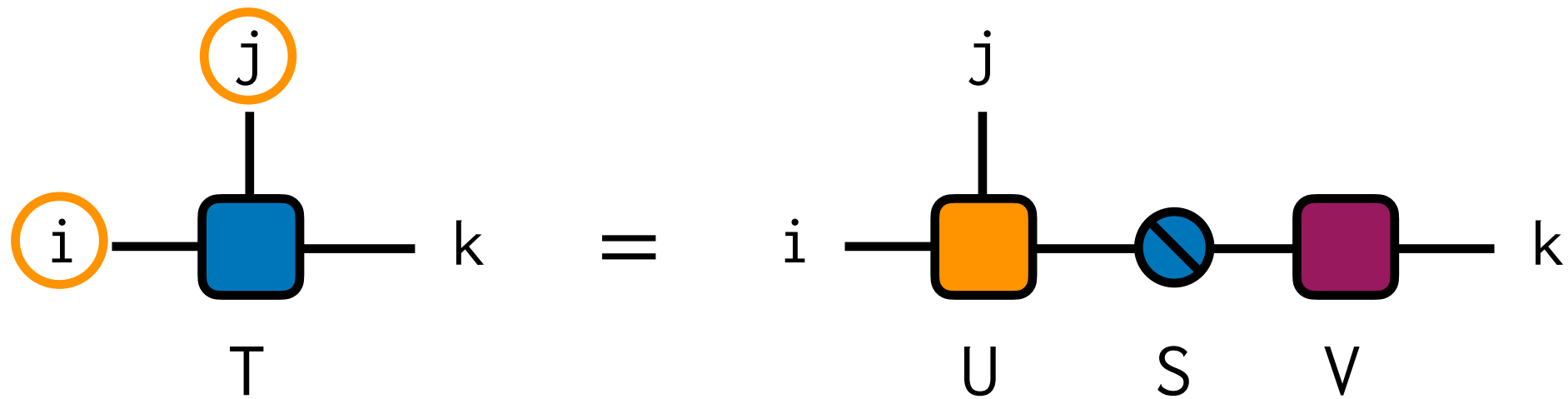
How to SVD an ITensor?

Select which indices go onto "U"



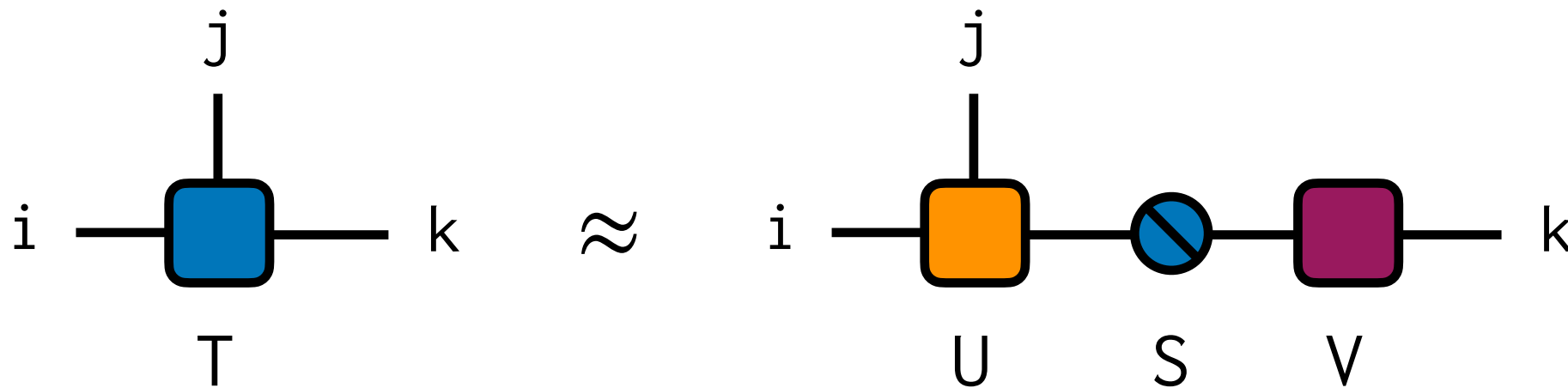
```
julia> u_inds = [i,j]
```

Call ITensor `svd` function



```
julia> u_inds = [i,j]
julia> U,S,V = svd(T, u_inds)
```

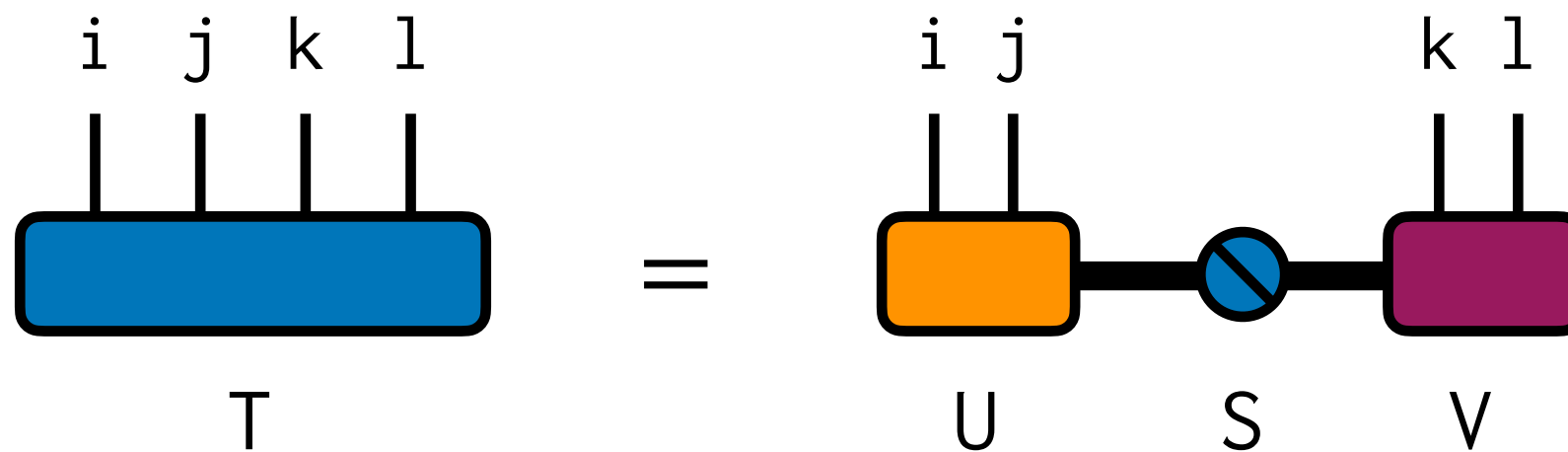
To compute truncated SVD, pass
truncation parameters: maxdim, cutoff



```
julia> u_inds = [i,j]
julia> U,S,V = svd(T, u_inds; maxdim=10, cutoff=1E-4)
```

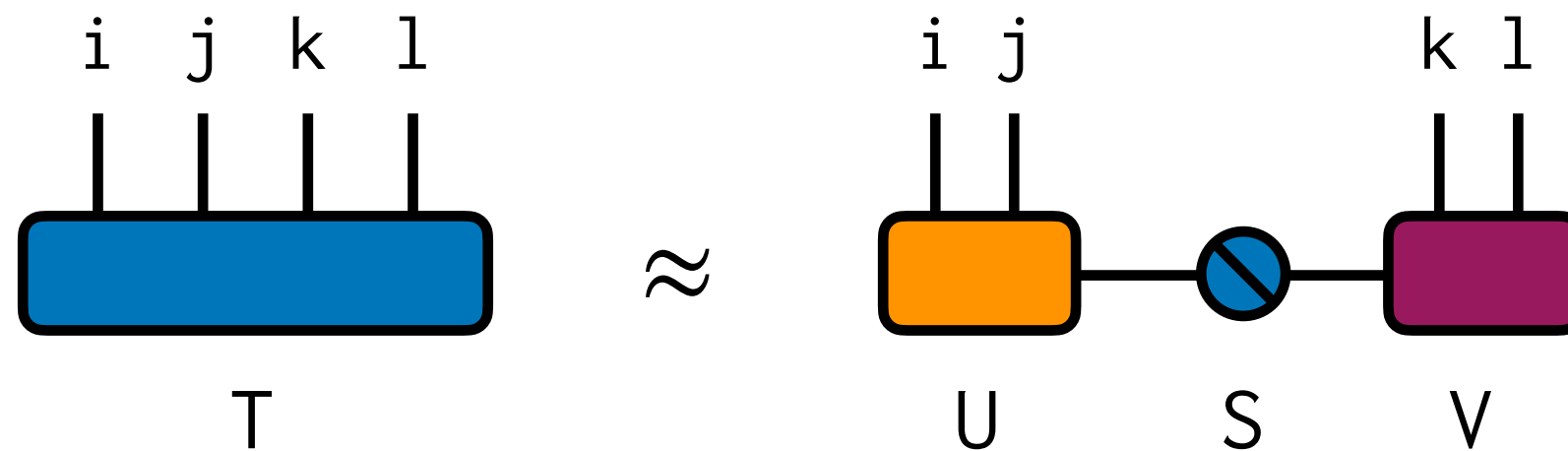
- maxdim: maximum dimension of internal bond
- cutoff ϵ : discard singular values λ_j such that $\sum_{j \in \text{discarded}} \lambda_j^2 \leq \epsilon$

Intuition about truncation - large outer spaces
yet tensor has hidden "low rank" property



```
julia> u_inds = [i,j]  
julia> U,S,V = svd(T, u_inds)
```

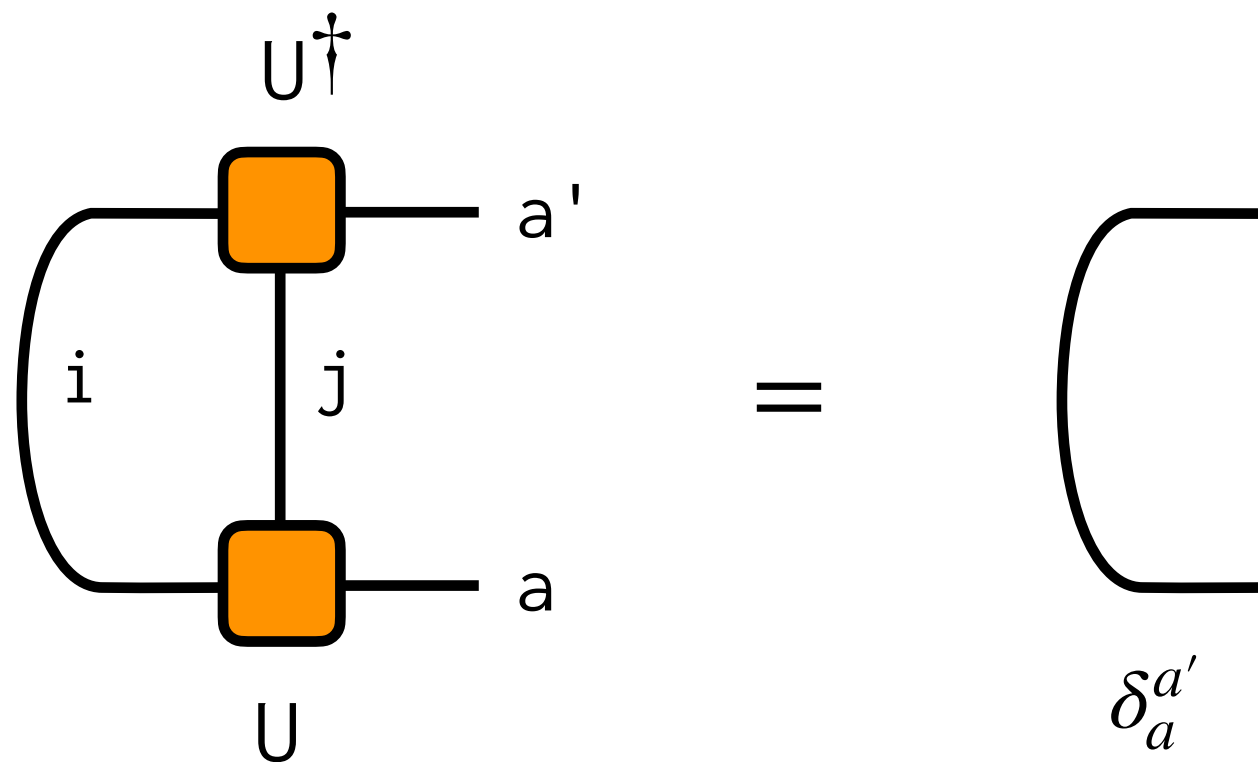
Intuition about truncation - large outer spaces
yet tensor has hidden "low rank" property



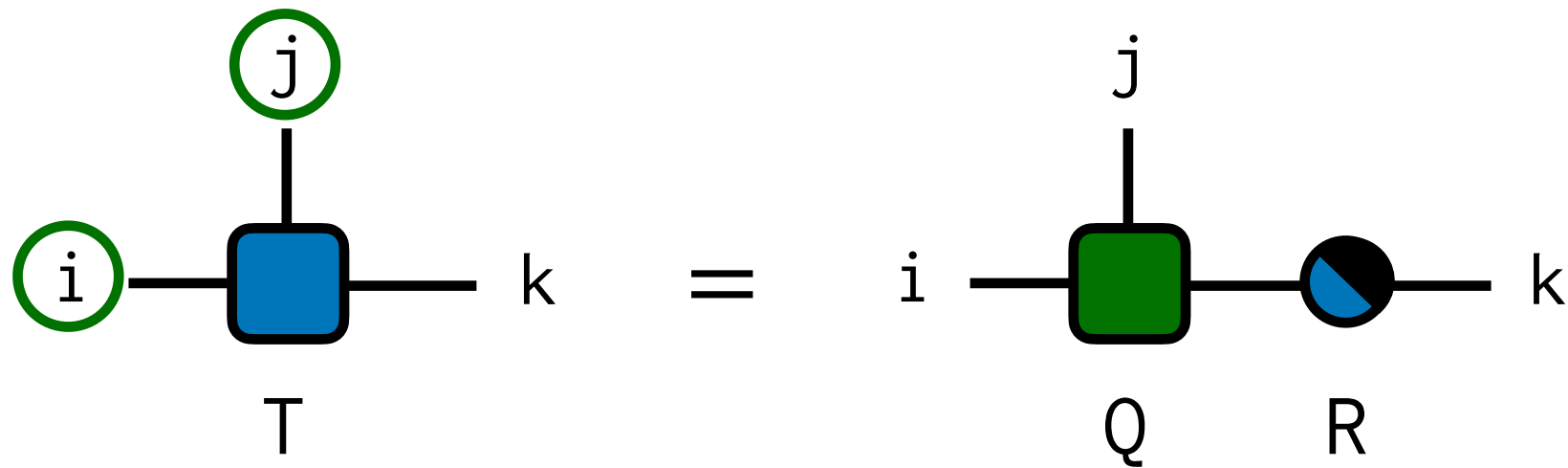
```
julia> u_inds = [i,j]
julia> U,S,V = svd(T, u_inds; maxdim=10, cutoff=1E-4)
```


U and V have "isometric" property

One-sided 'unitary':



Other decompositions work similarly
QR, for example



```
julia> q_inds = [i,j]
julia> Q,R = qr(T, q_inds)
julia> T ≈ Q * R
true
```

Summary

Tensor networks can compress many-body states by an exponential amount

Matrix product state (MPS) tensor networks offer many algorithms

The ITensor software makes it easy to perform basic tensor operations underlying tensor networks

Up Next

We will dive into the DMRG algorithm and see how to run it with ITensorMPS

Optional Bonus Slides

Gauging Tensor Networks

Gauging Tensor Networks

Recall gauging in electromagnetism ⚡

If one transforms:

$$\mathbf{A} \rightarrow \mathbf{A} + \nabla f$$

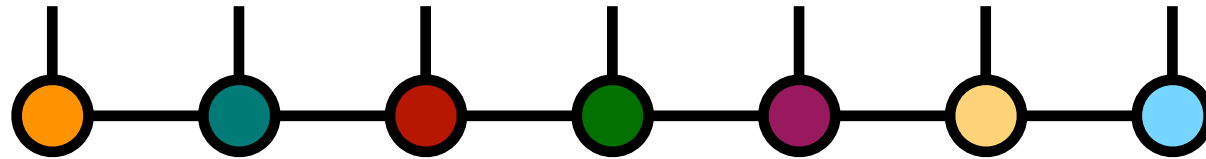
$$\varphi \rightarrow \varphi - \frac{\partial f}{\partial t}$$

Then physical observables **do not change**

Implies **redundancy** in the representation

Gauging Tensor Networks

Tensor networks likewise have a huge redundancy



They are "gauge theories" of quantum states

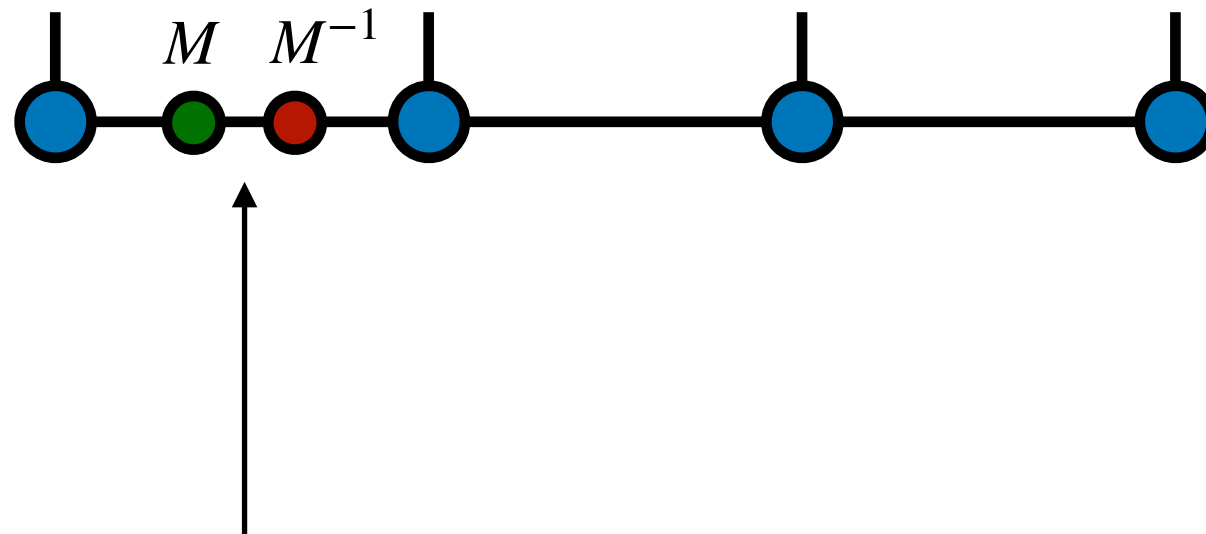
Gauging Tensor Networks

What are the gauge transformations of an MPS?



Gauging Tensor Networks

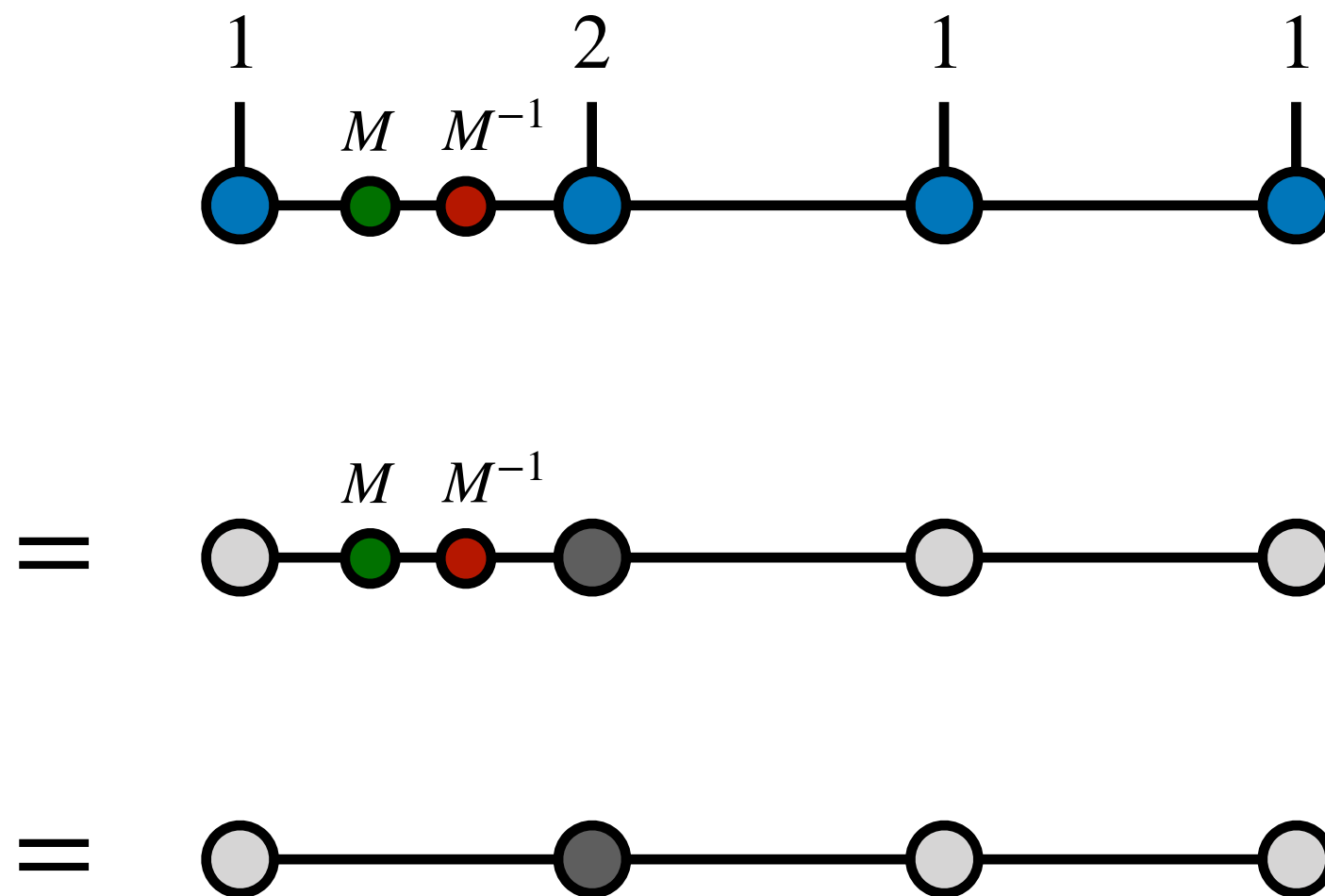
What are the gauge transformations of an MPS?



Insert resolution of identity $1 = MM^{-1}$

Gauging Tensor Networks

Check the tensor is unchanged – compute the T^{1211} element for example:



M and M^{-1} cancel out, proving each element unchanged

Gauging Tensor Networks

Can absorb "gauging" matrices into MPS



Different MPS representing the same tensor

Gauging Tensor Networks

What gauges are useful?

How does one compute useful gauges?

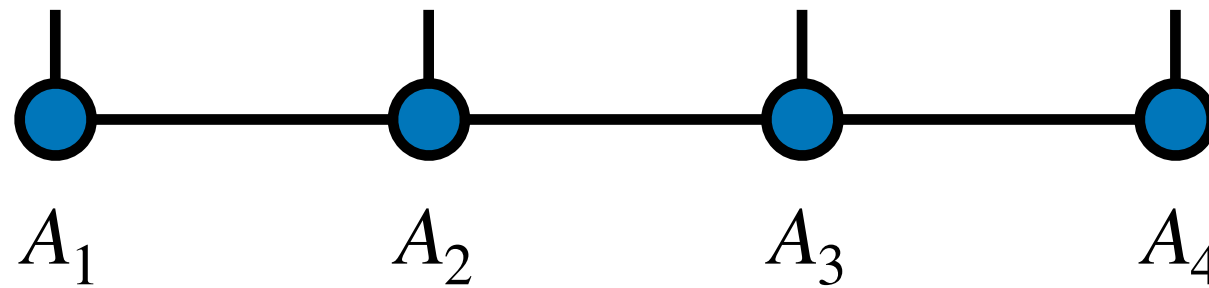
Interesting TN gauges related to:

- matrix factorizations
- quantum circuits

Orthogonal MPS Gauge

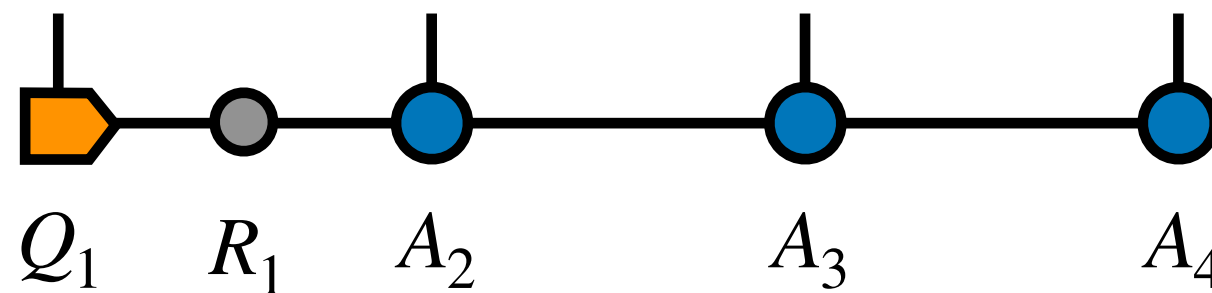
Let's find the "orthogonal" gauges of an MPS

Start from an arbitrary MPS

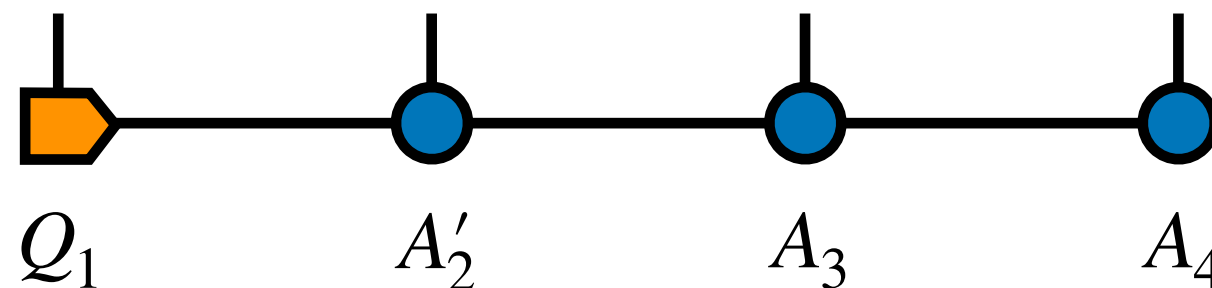


Orthogonal MPS Gauge

Now compute QR decomposition of $A_1 = Q_1 R_1$

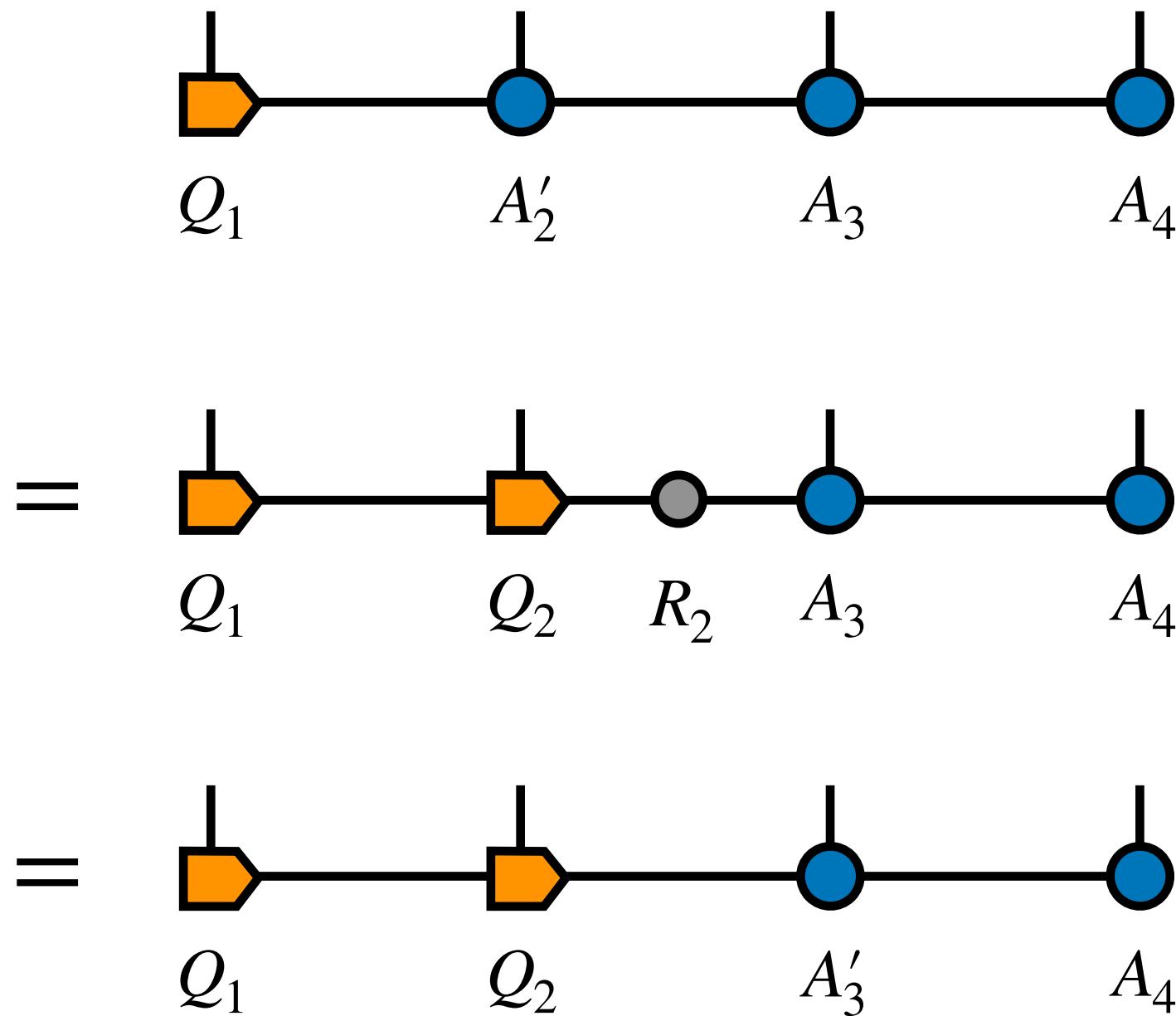


Multiply R_1 into A_2 to make A'_2



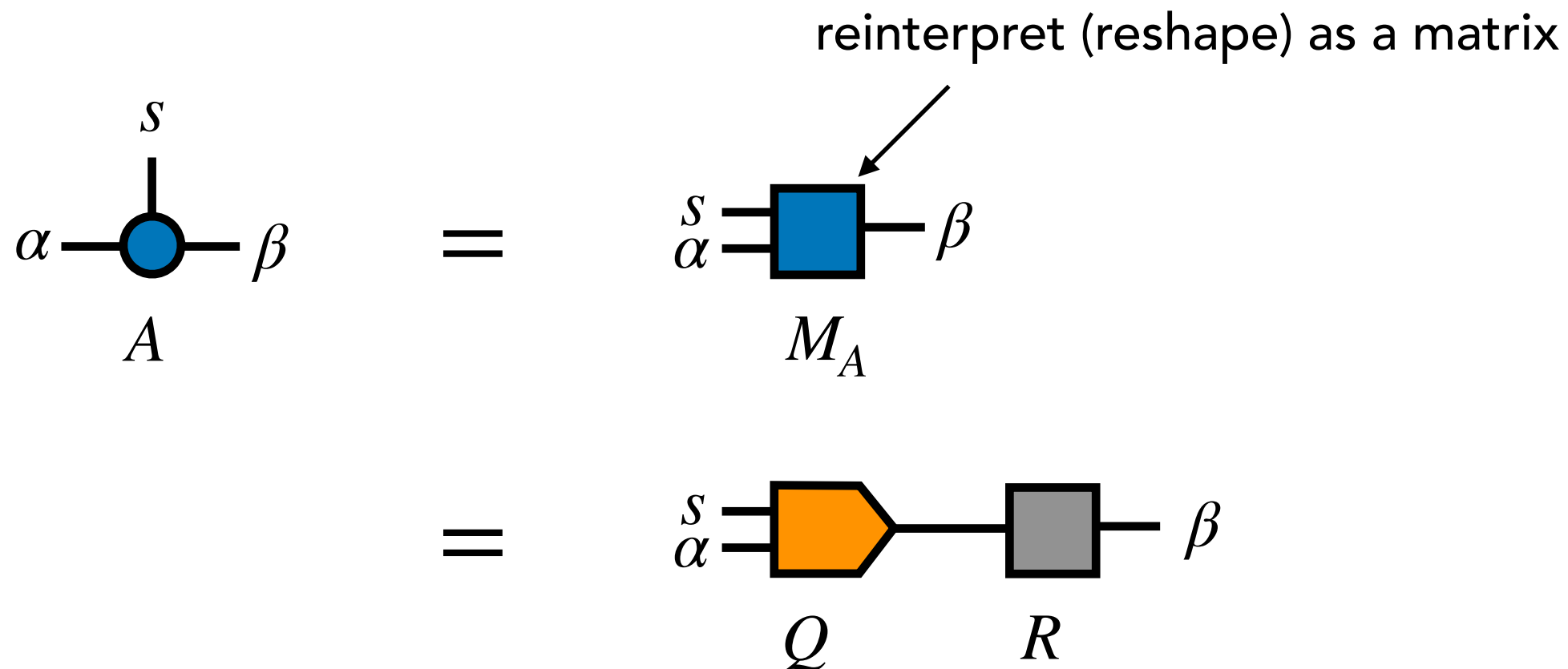
Orthogonal MPS Gauge

Next compute QR decomposition of $A'_2 = Q_2 R_2$

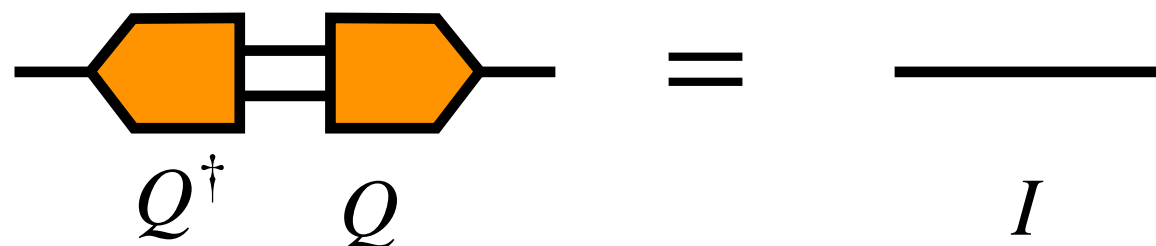


Orthogonal MPS Gauge

Recall QR of a tensor

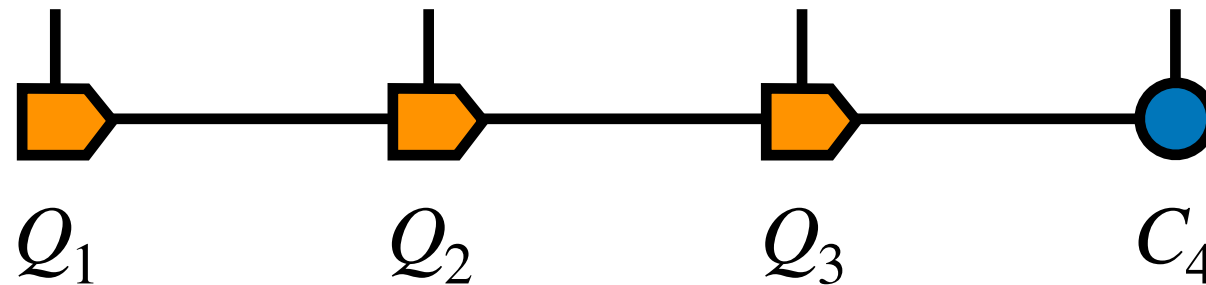


Q is an "isometric embedding" (isometry)



Orthogonal MPS Gauge

Continuing with more QR moves, one obtains



Call this "left orthogonal" gauge

Q tensors have "left orthogonality" property

A diagram illustrating the left orthogonality property of the Q tensors. On the left, two orange hexagonal tensors are stacked vertically. The top tensor is labeled $Q_2^\dagger$ and the bottom tensor is labeled Q_2 . A curved line connects the right side of the top tensor to the right side of the bottom tensor. This is followed by an equals sign and a single curved line, representing the identity matrix.

$$\sum_{s,a} Q_{ba}^{\dagger s} Q_{ab'}^s = I_{bb'}$$

Orthogonal MPS Gauge

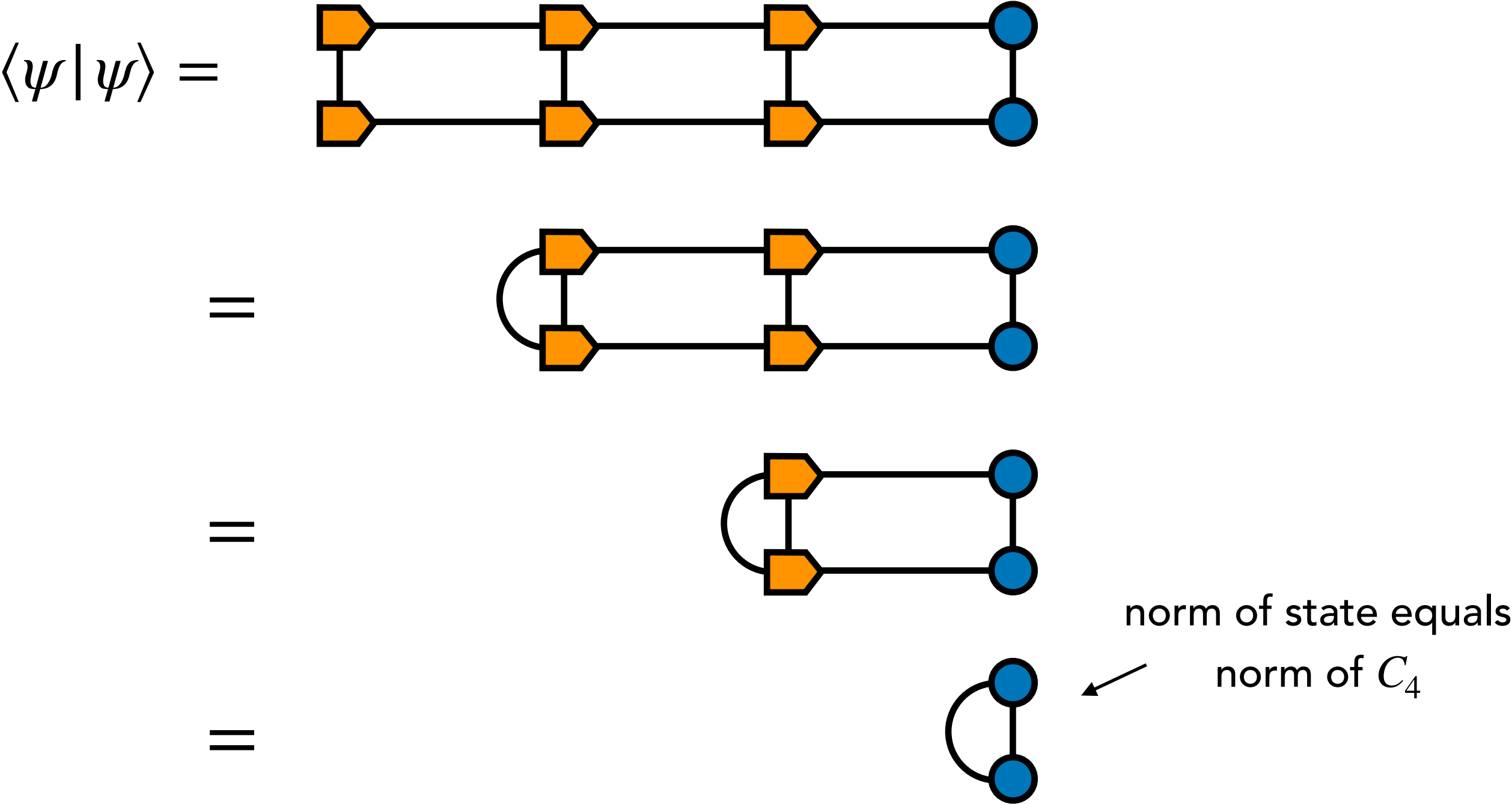
Challenge question:

Why QR and not SVD?

What advantages/disadvantages might one find?

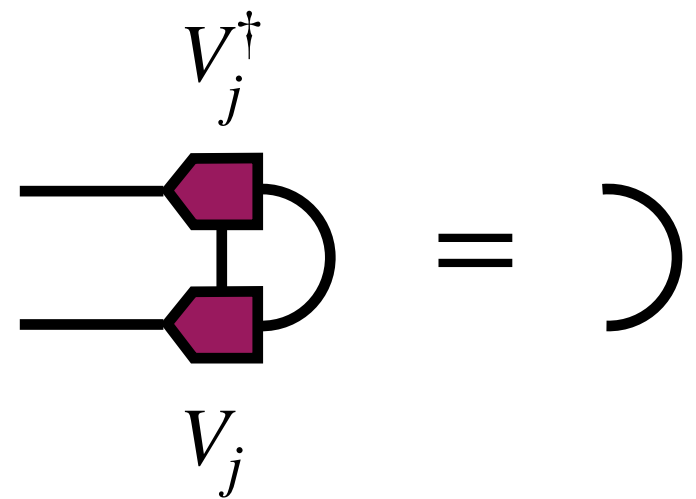
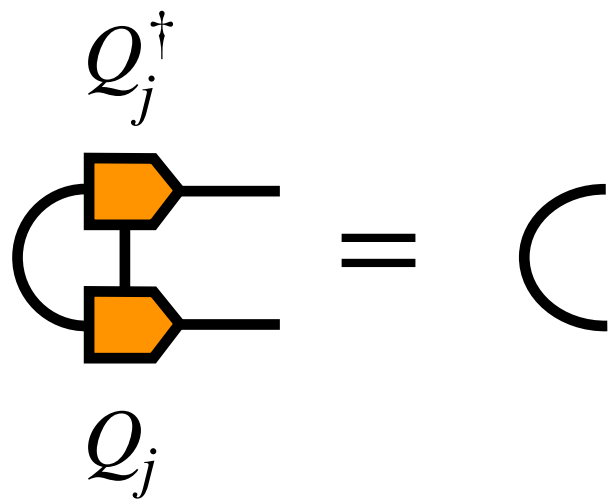
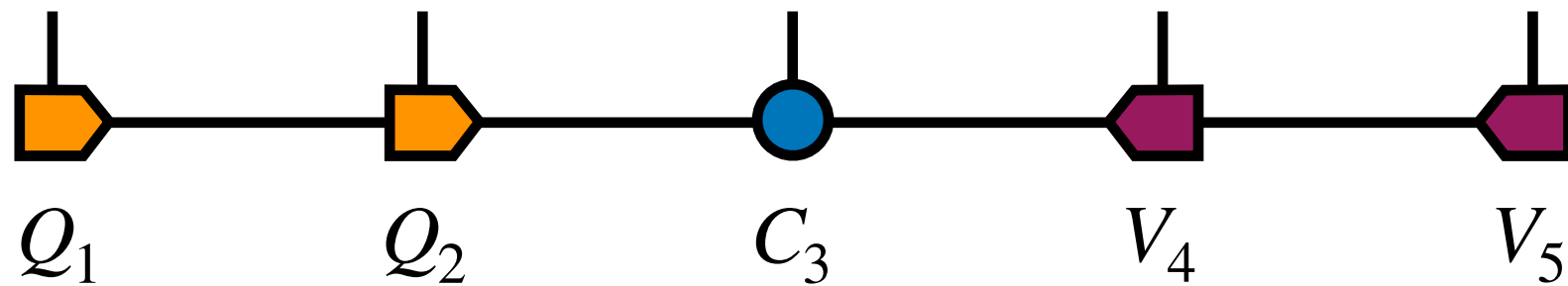
Orthogonal MPS Gauge

Can deduce certain properties



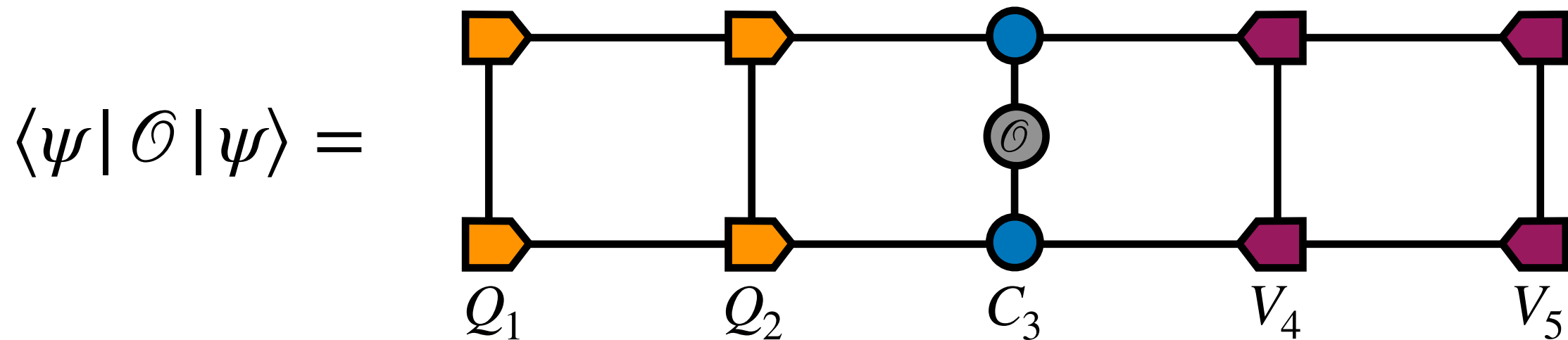
Orthogonal MPS Gauge

Can make "mixed" gauges too



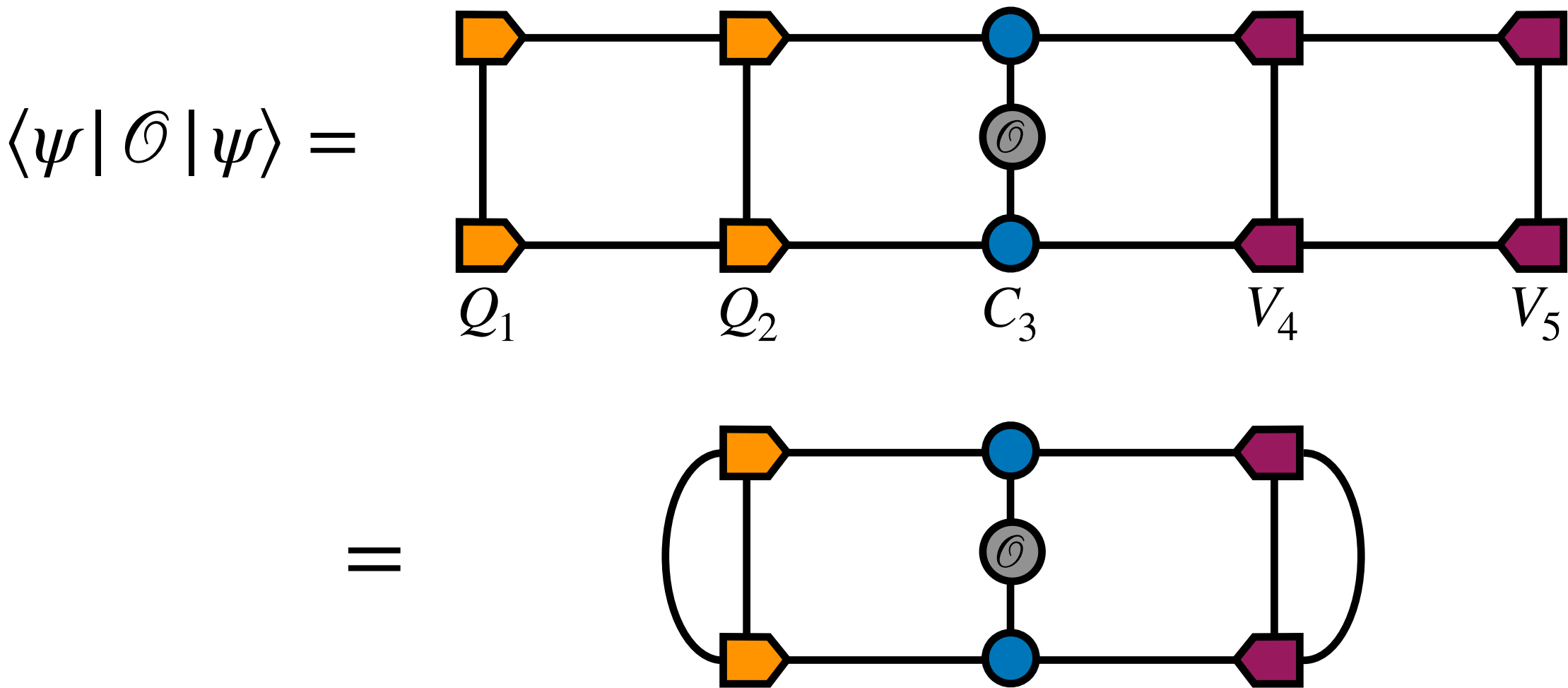
Orthogonal MPS Gauge

Mixed gauges useful for expected values



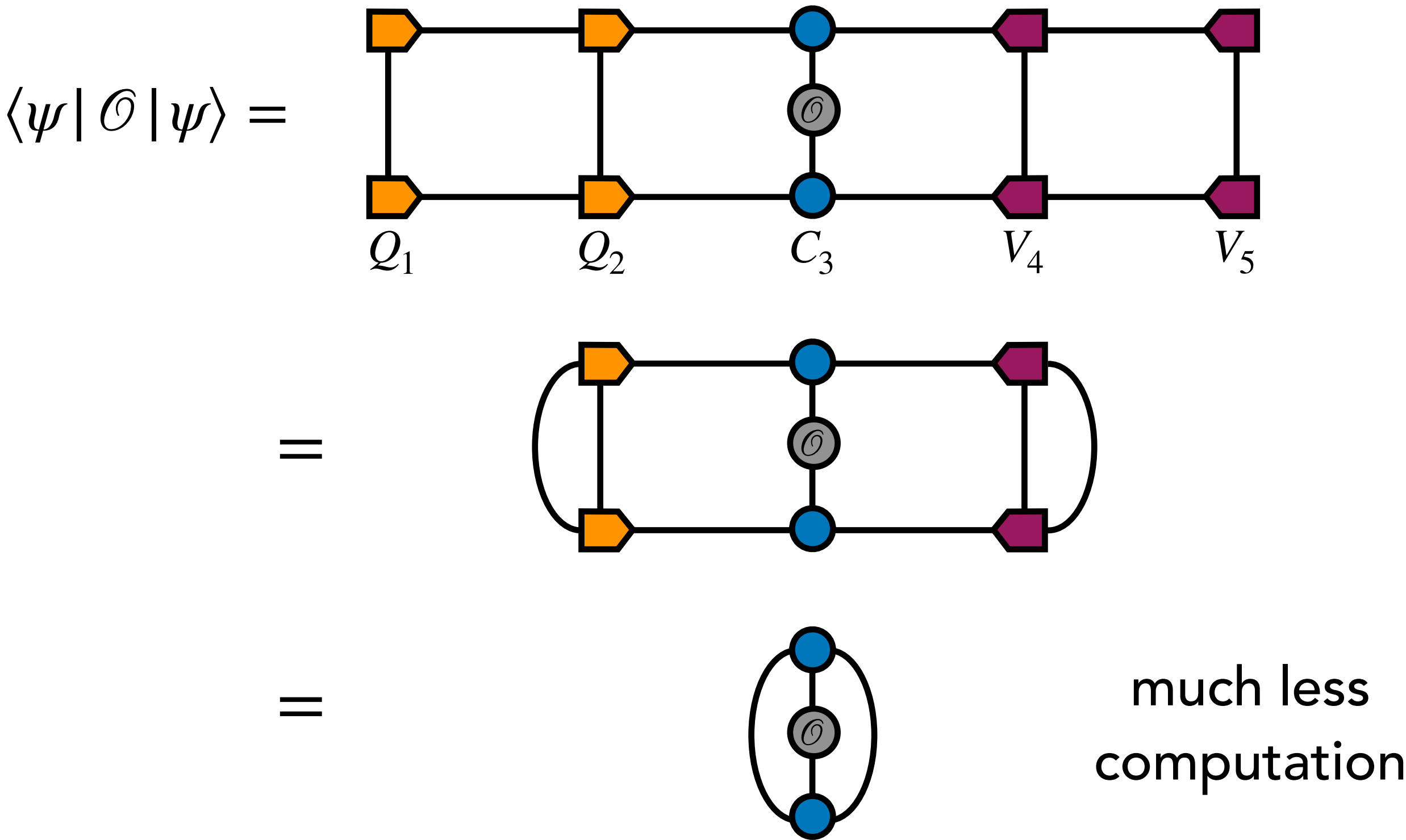
Orthogonal MPS Gauge

Mixed gauges useful for expected values



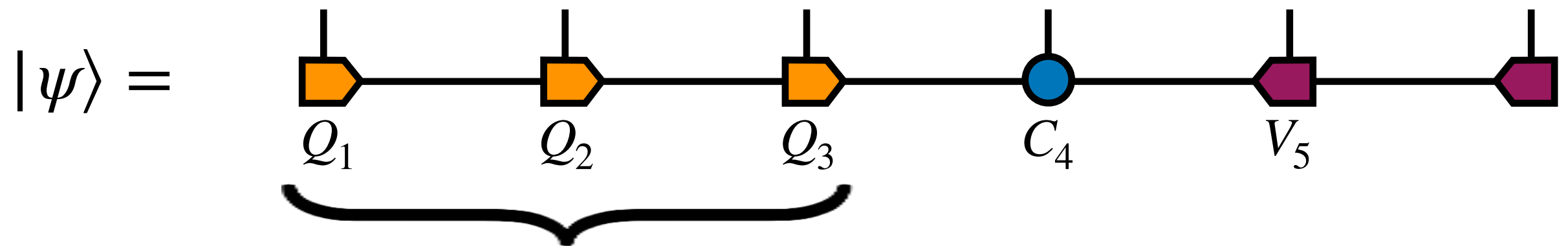
Orthogonal MPS Gauge

Mixed gauges useful for expected values



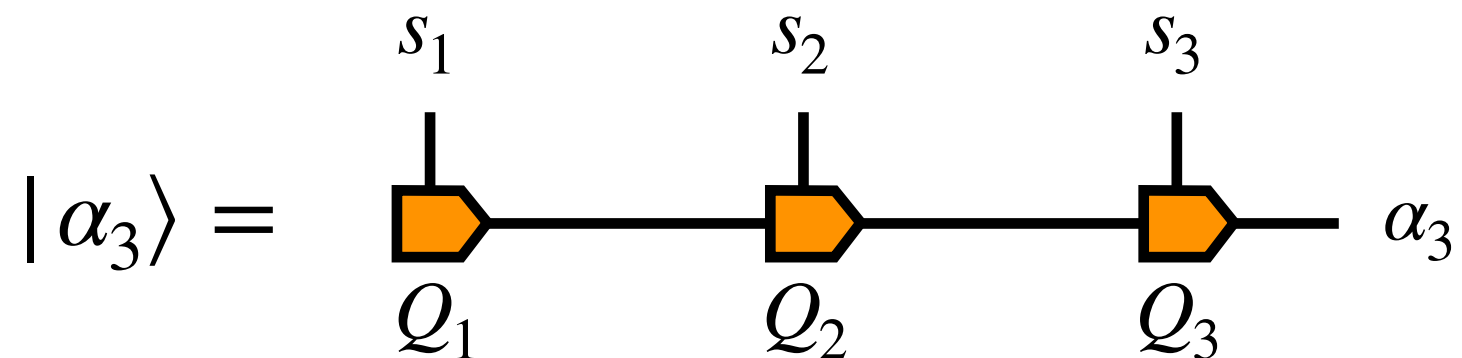
Orthogonal MPS Gauge

Mixed gauges also give interesting interpretation of MPS



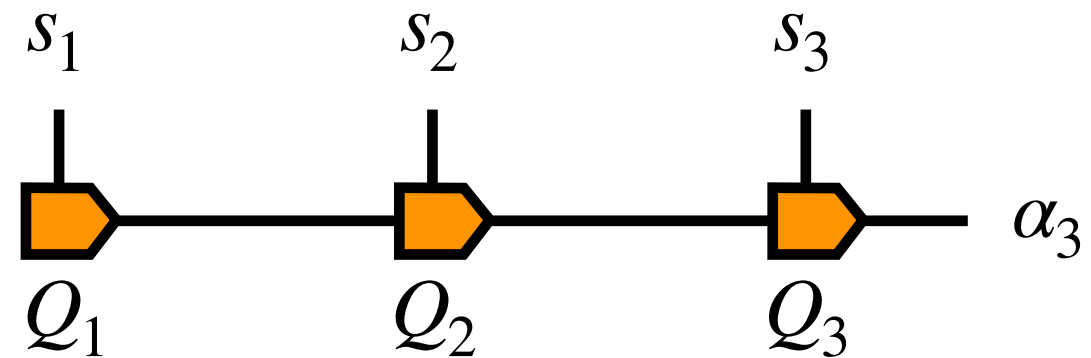
consider left part of 'gauged' MPS

It is an orthonormal basis – why?

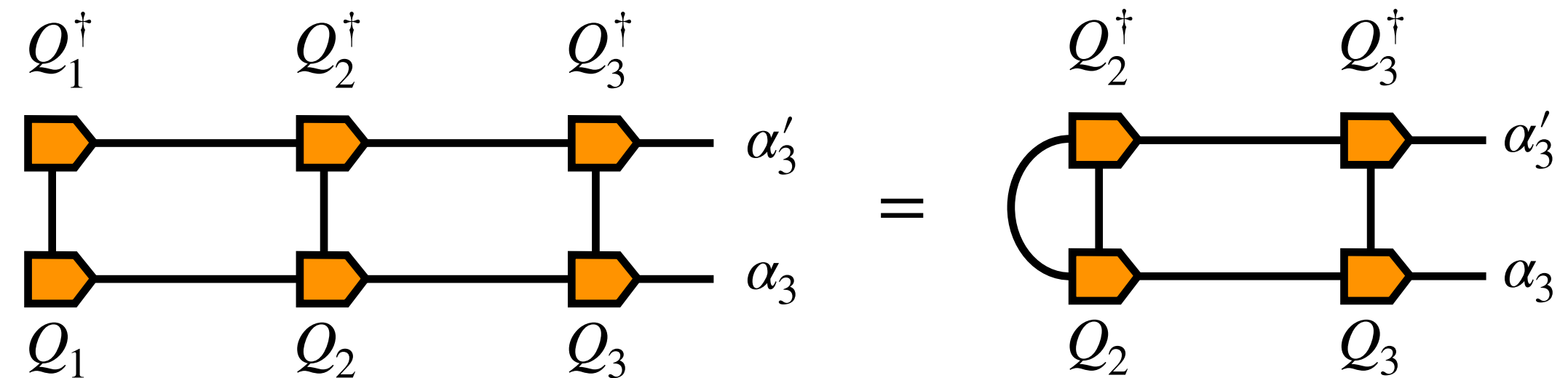


Orthogonal MPS Gauge

It is an orthonormal basis – why?

$$|\alpha_3\rangle =$$


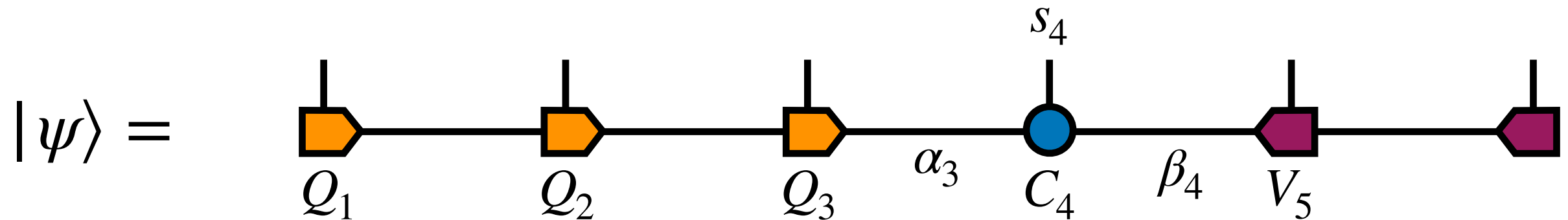
Compute overlap

$$\langle \alpha'_3 | \alpha_3 \rangle =$$


$$= \delta_{\alpha'_3 \alpha_3} \text{ orthonormal in the } \alpha'_3, \alpha_3 \text{ labels}$$

Orthogonal MPS Gauge

Thus entire MPS can be viewed as following

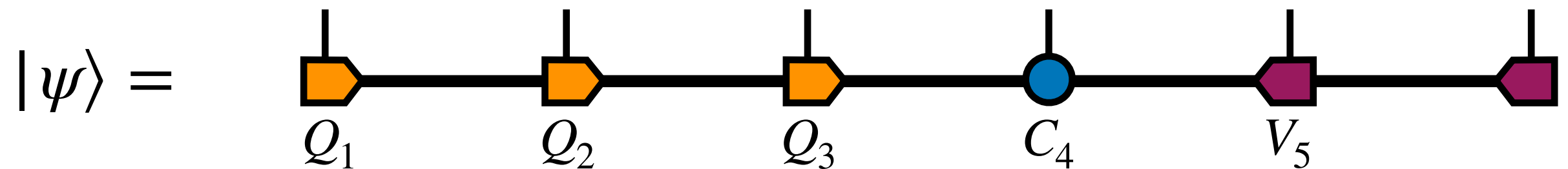


$$|\psi\rangle = \sum_{\alpha_3 \beta_4} C_{\alpha_3 \beta_4}^{s_4} |\alpha_3\rangle |s_4\rangle |\beta_4\rangle$$

In a sense, C_4 is the whole tensor or wavefunction just written in a specific "many-body" basis

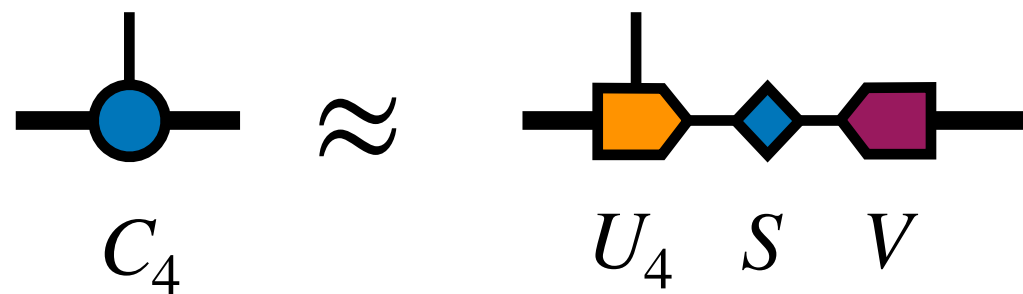
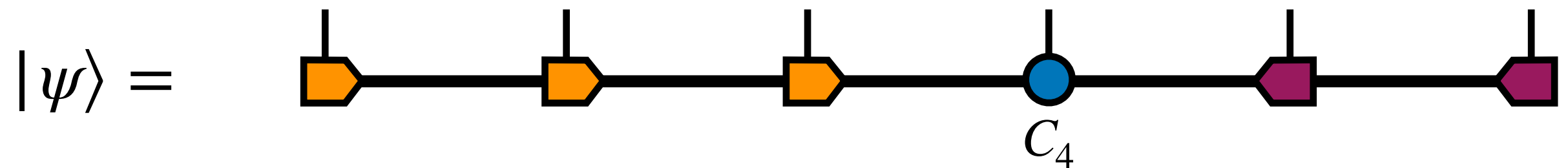
Orthogonal MPS Gauge

Using SVD instead of QR gives opportunity to "round" or truncate the bond dimension



Orthogonal MPS Gauge

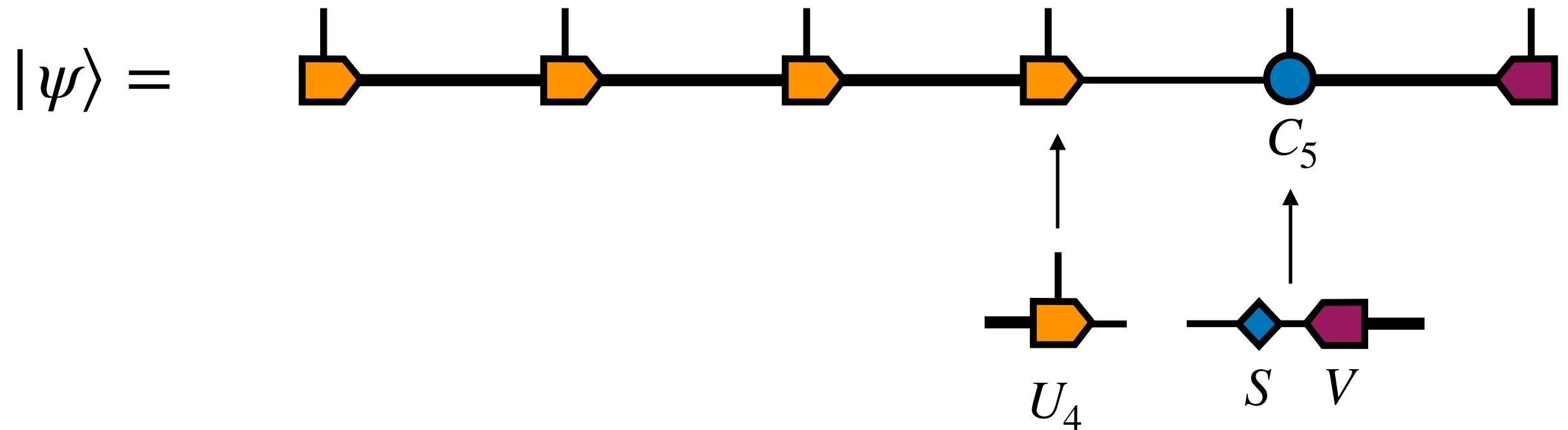
Using SVD instead of QR gives opportunity to "round" or truncate the bond dimension



Perform SVD of C_4 tensor – **truncate** small singular values

Orthogonal MPS Gauge

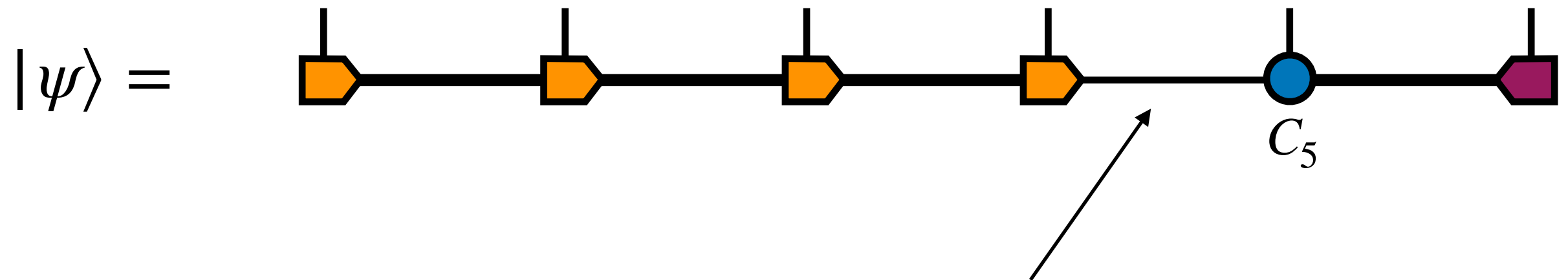
Using SVD instead of QR gives opportunity to "round" or truncate the bond dimension



Leave "U" behind, merge $S*V$ to form new "center"

Orthogonal MPS Gauge

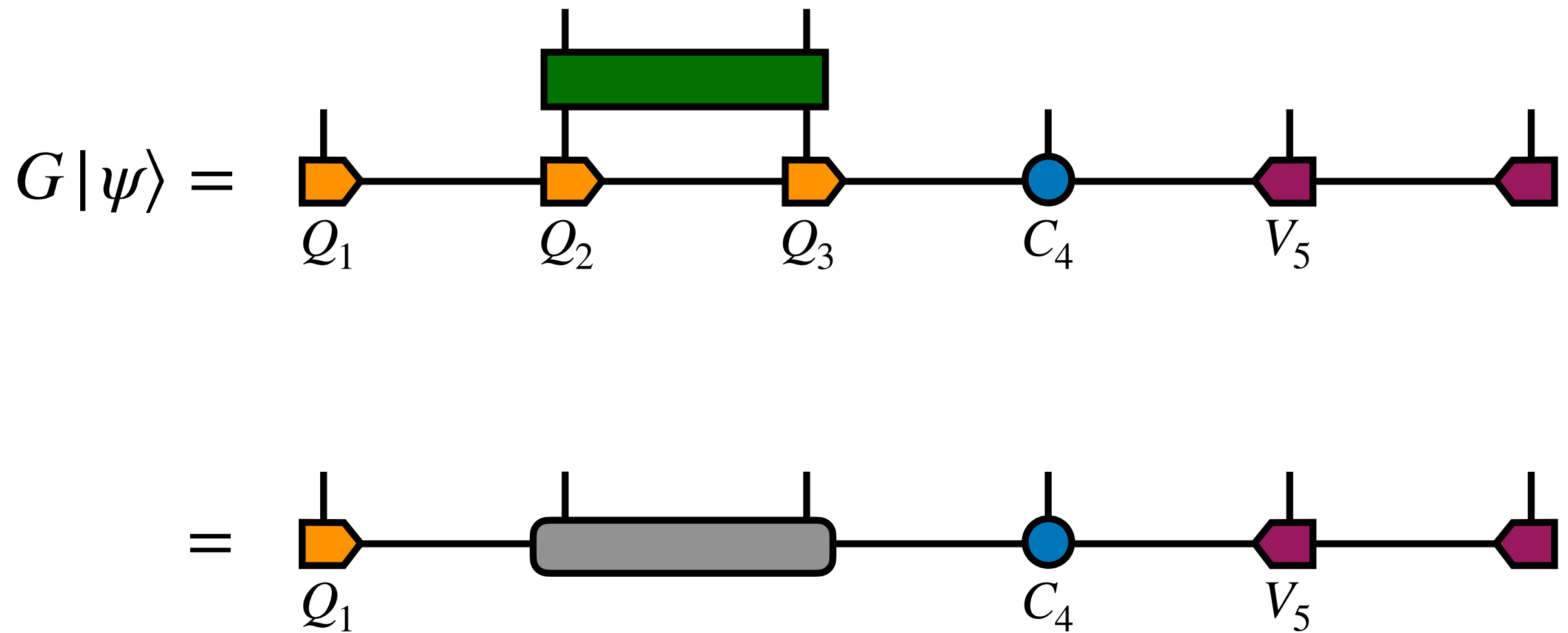
Using SVD instead of QR gives opportunity to "round" or truncate the bond dimension



Afterward, bond size has been reduced

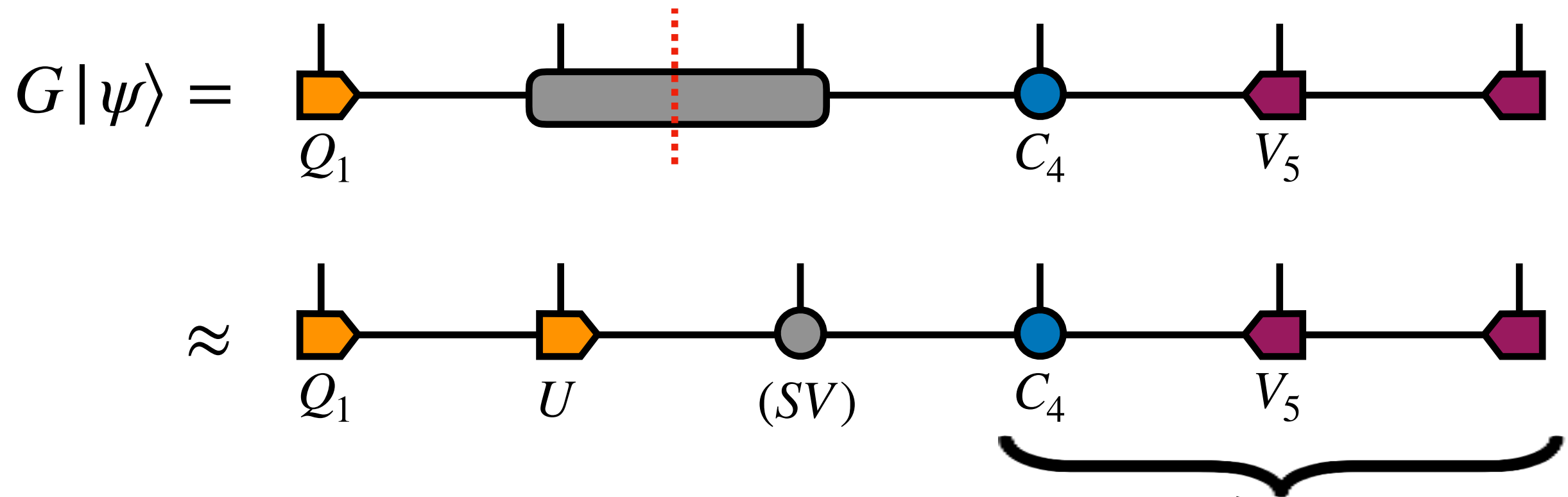
Importantly, truncation must happen within orthogonal gauge "center" – why?

Say we apply a gate away from the center



So far it is exact – no error

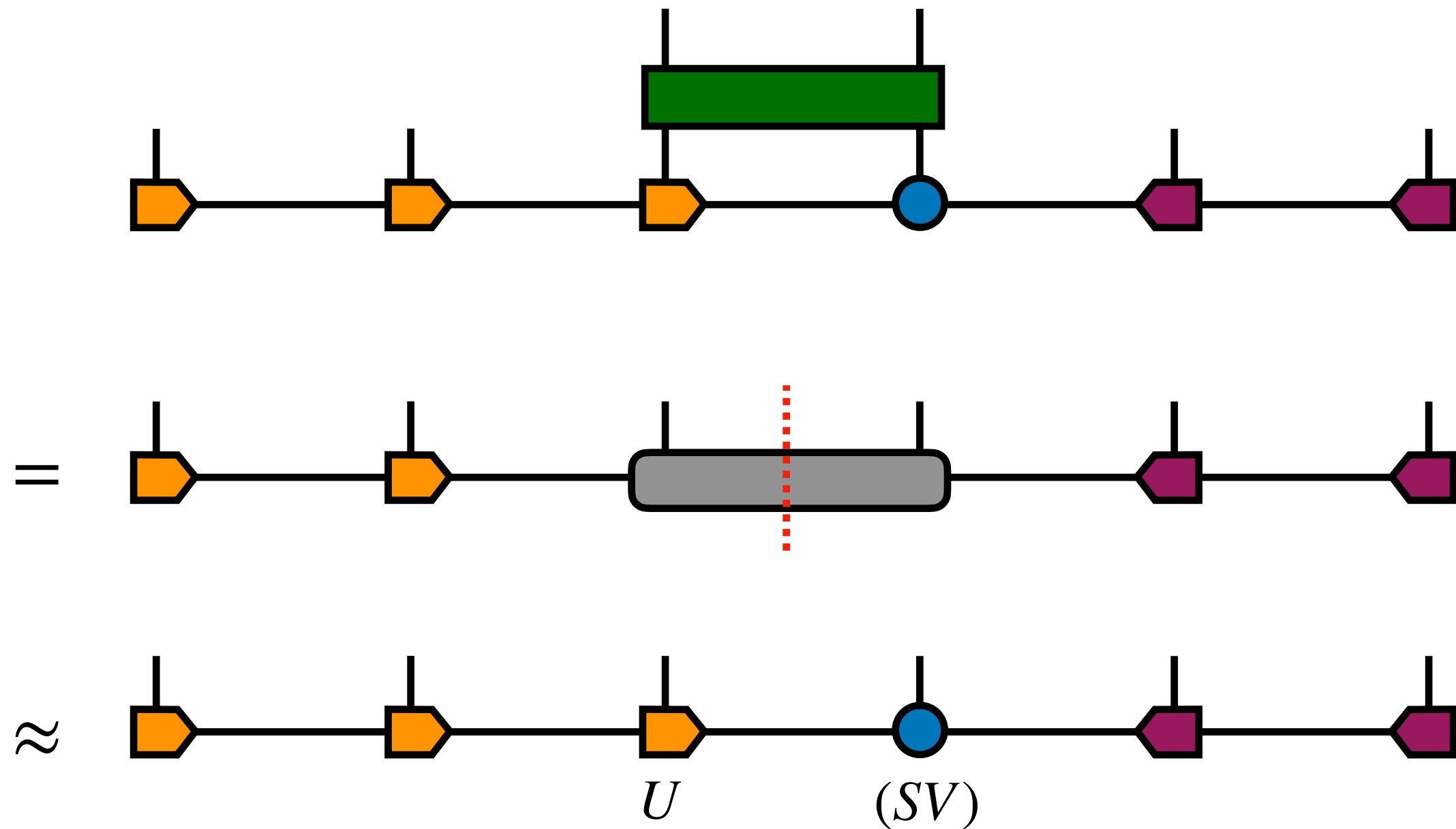
Now we can SVD & truncate small singular values



Not an orthonormal basis

Might make an **arbitrarily large** error

In contrast, truncating inside an orthonormal basis is controlled



local SVD error = global error